

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
филиал федерального государственного автономного
образовательного учреждения высшего образования
«Мурманский арктический университет» в г. Апатиты
(филиал МАУ в г. Апатиты)

КАФЕДРА ИНФОРМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ ДЛЯ ИССЛЕДОВАТЕЛЬСКОГО
ПОИСКА В МАССИВАХ СТРУКТУРИРОВАННЫХ ДАННЫХ С ПОМОЩЬЮ БОЛЬШИХ
ЯЗЫКОВЫХ МОДЕЛЕЙ

Выполнил обучающийся 4 курса
Фигуркин Даниил Сергеевич
Направление подготовки 09.03.02 Информационные
системы и технологии
Направленность (профиль): Программно-аппаратные
комплексы
очная форма обучения
группа 4БИСИТ-ПАК_АФ

Научный руководитель:
Федоров Андрей Михайлович – канд. техн. наук

г. Апатиты
2026 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	4
Глава 1. ТЕОРЕТИЧЕСКИЕ АСПЕКТЫ ПРЕДСТАВЛЕНИЯ И ПОИСКА В СТРУКТУРИРОВАННЫХ ДАННЫХ.....	7
1.1 Программный код как разновидность структурированных данных.....	7
1.2 Исследовательский поиск в программных данных	8
1.3 Подходы к поиску в программном коде	9
1.4 RAG и большие языковые модели в задачах исследовательского поиска	11
1.5 Представление информации для разных интерпретаторов	12
1.6 Выводы по главе 1	13
ГЛАВА 2. ОБЗОР ТЕХНОЛОГИЙ И АНАЛОГОВ.....	15
2.1. Требования к разрабатываемой информационной системе	15
2.2 Pascal/Delphi-кодовая база.....	16
2.3 Извлечение структуры кода через PasDoc.....	18
2.4 Markdown-wiki и Obsidian как представление для человека	19
2.5 Инфраструктура семантического поиска по Markdown-wiki	20
2.6 pasls как инструмент точной навигации по Pascal/Delphi-коду	22
2.7 Инструментальный доступ к кодовой базе: OpenCode, MCP и pasls	22
2.8 Средства воспроизводимости, управления и оптимизации.....	24
2.9 Аналоги и альтернативные подходы.....	26
2.10 Выводы по главе 2.....	27
ГЛАВА 3 РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	30
3.1. Постановка практической задачи	30
3.2 Архитектура и структура проекта finalversion.....	32
3.3. Конфигурационный слой системы	36
3.4 Сборка проекта	37
3.5 Генерация Markdown-wiki, RAG-ключи и AI-обогащение описаний	39
3.6 Индексирование wiki и организация семантического поиска.....	44
3.7 markdown-rag-mcp-wrapper.....	45
3.8 Настройка OpenCode-агента и подключение инструментов	47
3.9. Демонстрация и проверка работы системы.....	49
3.10 Ограничения и направления развития	53
3.11 Выводы по главе 3.....	57

ЗАКЛЮЧЕНИЕ	60
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	63
ПРИЛОЖЕНИЕ 1. Листинг ai-descr.local.conf и rag-pipeline.local.conf	65
ПРИЛОЖЕНИЕ 2. Листинг build-wiki-rag.sh.....	68
ПРИЛОЖЕНИЕ 3. Листинг ai-descr-ai-v3-config-rag-md-v3.sh.....	75
ПРИЛОЖЕНИЕ 4. Листинг pyproject.toml и server.py	107
ПРИЛОЖЕНИЕ 5. Листинг opencode.fragment_example.json и AGENTS.md ..	118

ВВЕДЕНИЕ

При сопровождении программных проектов разработчик часто сталкивается с большим объёмом, исчисляющимся тысячами строк кода и нелинейной топологией проекта. Сложность возрастает, если документация отсутствует или устарела. Особенно эта проблема заметна при сопровождении разработок, написанных на старых языках. Например, проекты, написанные на языке Pascal диалекта Delphi.

Актуальность темы обусловлена необходимостью упростить первичное изучение и сопровождение Pascal/Delphi-кодовых баз. Сегодня работа с Delphi часто связана не с выбором языка для новых проектов, а с необходимостью сопровождения ранее разработанных систем. В таких условиях разработчик сначала вынужден восстанавливать устройство проекта: искать основные модули, сопоставлять объявления и реализации, разбирать связи между классами и только после этого переходить к изменению кода. Как результат, сопровождение таких проектов может быть трудозатратным для разработчиков, которые не участвовали в их создании или не имеют достаточного опыта работы с Delphi-проектами и их типовой организацией.

В данном случае исходный код рассматривается как разновидность структурированных данных, поскольку его можно анализировать не только как последовательность строк, но и как набор программных сущностей с устойчивыми признаками. В случае Pascal/Delphi-кодовой базы основными сущностями анализа в данной работе являются unit, классы и методы.

Проблема, рассматриваемая в работе, заключается в том, что при ручной навигации по Pascal/Delphi-кодовой базе разработчик вынужден самостоятельно сопоставлять объявления, реализации, связи между модулями и назначение отдельных сущностей. Этот процесс требует времени и сильно зависит от предварительного знания проекта. Поэтому в работе предлагается система, которая не заменяет IDE или чтение исходников, а добавляет промежуточный навигационный слой между кодовой базой и разработчиком.

Цель работы — разработать информационную систему, которая преобразует Pascal/Delphi-кодovou базу в удобное для поиска и навигации представление, а так же позволяет разработчику выполнять исследовательский поиск по программным сущностям проекта.

Объектом исследования являются массивы структурированных данных, представленные кодовыми базами Pascal/Delphi-проектов.

Предметом исследования являются подходы к извлечению, представлению и поиску информации в кодовых базах Pascal/Delphi-проектов.

Для достижения требуемого результата необходимо решить следующие задачи:

- Рассмотреть программный код как разновидность структурированных данных и определить особенности его анализа.
- Рассмотреть основные подходы к поиску в программном коде.
- Проанализировать принципы представления информации о кодовой базе для разных акторов.
- Рассмотреть технологии, обеспечивающие извлечение сведений о кодовой базе, формирование промежуточного представления и выполнение поиска по подготовленным данным.
- Разработать архитектуру информационной системы, обеспечивающей извлечение сведений о кодовой базе, формирование промежуточного представления и выполнение поиска по подготовленным данным.
- Провести проверку работы системы.

Практическая значимость работы состоит в разработке локального инструмента, который формирует по Pascal/Delphi-проекту навигационное представление и уменьшает порог входа при работе с проектами.

Хронологические рамки исследования: в работе рассматриваются подходы, инструменты и open-source проекты в области интеграции LLM, актуальные в период 2020 — начало 2026 гг., поскольку именно в этот период

сформировался основной пласт прикладных библиотек и типовых решений для разработки [1; 22].

Выпускная квалификационная работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе рассматриваются теоретические аспекты представления и поиска в структурированных данных. Во второй главе проводится анализ технологий и аналогов, необходимых для построения системы. В третьей главе описывается фактическая разработка информационной системы

Глава 1. ТЕОРЕТИЧЕСКИЕ АСПЕКТЫ ПРЕДСТАВЛЕНИЯ И ПОИСКА В СТРУКТУРИРОВАННЫХ ДАННЫХ

1.1 Программный код как разновидность структурированных данных

Первый пункт рассмотрения применимости подхода начинается с установки доступного для обработки материала.

Программный код в данной работе рассматривается не как обычный текст, а как структурированные данные [5]. Его элементы и связи между ними определяются правилами языка программирования: синтаксисом объявлений, порядком размещения блоков, вложенностью конструкций и способом обращения одних частей программы к другим.

Для анализа кодовой базы этого достаточно, чтобы работать не только со строками файла, но и с более устойчивыми объектами: модулями, классами, методами, типами, процедурами, функциями и свойствами.

Набор признаков у этих сущностей различается. Имя и расположение в исходном файле является обязательными характеристиками, тогда как описание назначения, сведения о наследовании или связи с другими частями проекта могут отсутствовать либо извлекаться не полностью. Например, класс в Pascal/Delphi-проекте связан не только со своим именем. Для анализа важны модуль, в котором он объявлен, его методы, возможный предок, используемые типы и место реализации связанных процедур.

Для Pascal/Delphi-кодовых баз структурность особенно заметна на уровне unit. В одном модуле могут находиться объявления, открытые для других частей проекта, и реализации, которые показывают фактическое поведение методов и процедур. Из-за этого при сопровождении приходится сопоставлять несколько фрагментов одного файла: само объявление, принадлежность к классу, реализацию и связи с другими unit.

Поэтому поиск только по строкам недостаточен. Он помогает найти совпадение, но не показывает, какую роль найденный фрагмент играет в проекте.

Более полезным становится анализ программных сущностей: unit, классов, методов, типов и отношений между ними.

Такой анализ можно условно разделить на несколько уровней которые будут рассмотрены далее.

1.2 Исследовательский поиск в программных данных

В данной работе исследовательский поиск понимается как поиск «от задачи», а не «от имени» [6]. Разработчик может понимать, какую функциональность нужно найти, но не знать, как называется нужный unit, класс, метод или процедура.

Для Pascal/Delphi-проектов это обычная ситуация. Логика может быть распределена между несколькими модулями, а объявление и реализация одной сущности находятся в разных частях файла. Поэтому результатом такого поиска должно быть не одно случайное совпадение, а постепенное сужение области: сначала подходящая часть проекта, затем конкретная сущность, затем фрагмент исходного кода.

В кодовой базе объектом такого поиска могут быть:

- модуль, отвечающий за определённую часть функциональности;
- класс, реализующий нужное поведение;
- метод или процедура, выполняющие конкретное действие;
- тип данных, используемый в нескольких частях проекта;
- свойство или поле, влияющее на состояние объекта;
- связь между несколькими сущностями;
- группа элементов, совместно реализующих некоторый механизм.

Обычный поиск по строке хорошо работает, когда известен точный идентификатор: имя файла, класса, метода, переменной или константы. В исследовательском поиске такой информации часто нет. Запрос начинается с описания поведения: например, требуется найти место, где выполняется обработка очереди, поиск элемента, расчёт значения или обращение к внешнему ресурсу.

Поэтому здесь важен не только найденный текст. Нужно понимать ещё и что именно найдено: модуль, класс, метод, поле, тип или связь между несколькими элементами. Без этого результат остаётся набором совпадений, которые разработчику всё равно придётся вручную сопоставлять с устройством проекта.

Для плохо документированной кодовой базы это особенно заметно. Последовательное чтение всех файлов редко даёт быстрый результат: важная связь может находиться в другом unit, в объявлении класса или в реализации метода, к которой разработчик ещё не дошёл.

1.3 Подходы к поиску в программном коде

Поиск в программном коде нельзя рассматривать как одну универсальную операцию. В разных ситуациях разработчик ищет разные вещи: иногда ему известно точное имя метода или класса, иногда он помнит только примерное назначение функциональности, а иногда ему нужно не найти строку, а понять связь между несколькими частями проекта. Поэтому для работы с кодовой базой полезно разделять несколько уровней поиска:

а) текстовый поиск по файлам. Он применяется тогда, когда известно точное имя сущности или фрагмент строки. Например, разработчик может искать имя метода, переменной, класса, константы или текстовое значение. Такой поиск удобен своей простотой: он не требует предварительной обработки проекта и даёт быстрый результат. Однако его возможности ограничены. Если имя неизвестно или в разных частях проекта используются разные обозначения одной и той же логики, текстовый поиск не помогает. Кроме того, найденное совпадение не всегда относится к рабочему коду: оно может находиться в комментарии, строковом литерале или устаревшем фрагменте.

б) лексический поиск. В этом случае анализ выполняется не по отдельным символам, а по токенам языка: идентификаторам, ключевым словам, операторам и литералам. Такой подход меньше зависит от оформления исходного файла. Например, переносы строк, пробелы или иной стиль

форматирования не влияют на результат. При этом он сложнее обычного текстового поиска, поскольку требует предварительной обработки исходных файлов.

в) синтаксический поиск. Он учитывает структуру языка программирования и позволяет искать не просто совпадения, а конкретные конструкции. Для Pascal/Delphi это особенно важно, потому что в одном unit могут быть разделы interface и implementation, объявления классов, реализации методов, процедуры, функции и свойства. Синтаксический поиск позволяет, например, отличить объявление метода от его реализации или вызова. Это делает результат более полезным для анализа, но требует парсера или другого инструмента, который способен корректно разбирать Pascal/Delphi-код [5]. В старых проектах это может быть затруднено из-за условной компиляции, нестандартного оформления и устаревших языковых конструкций.

г) Отдельно следует выделить навигационный поиск. Он связан не столько с поиском текста, сколько с переходами между программными сущностями. К нему относятся переход к объявлению, реализации и поиск мест использования. Такой подход удобен, когда имя сущности уже известно, но требуется понять, где она определена и как связана с другими частями проекта. Для кодовой базы это особенно полезно при работе с unit-файлами, классами форм, методами и обработчиками событий. Навигационный поиск помогает уточнить найденный результат и проверить его по исходному коду и при этом даёт меньше нерелевантных совпадений относительно текстового поиска.

д) поиск по документации. В этом случае источником становятся не сами исходные файлы, а описания модулей, классов, методов и других элементов. Такой поиск ближе к тому, как человек формулирует задачу. Например, разработчик может искать не имя процедуры, а описание действия: “формирование отчёта”, “загрузка библиотек”, “проверка данных”. Для плохо документированных или старых проектов такой слой особенно полезен, потому что имена сущностей в коде могут быть сокращёнными или неочевидными. Но у документационного поиска есть слабое место: если описание неполное,

устаревшее или автоматически сформировано с ошибкой, результат поиска также может оказаться неточным [21].

е) Для исследовательского сценария наиболее важен семантический поиск. Он используется тогда, когда запрос задаётся через смысл, а не через точное имя. Например, разработчику нужно найти место, где рассчитывается значение, выполняется проверка клиента, происходит подключение к базе данных или формируется отчёт, но он не знает названия соответствующего метода. Семантический поиск позволяет найти близкие по смыслу описания или фрагменты [6].

В рамках данной работы эти подходы рассматриваются не как взаимоисключающие, а как дополняющие друг друга. Для Pascal/Delphi-кодовой базы наиболее целесообразным является сочетание обзорного поиска по подготовленному представлению и последующей проверки найденных сущностей по исходным файлам.

1.4 RAG и большие языковые модели в задачах исследовательского поиска

В задачах исследовательского поиска большие языковые модели целесообразно использовать как средство интерпретации найденного контекста и слой обработки запросов на естественном языке [2]. Они могут сопоставить несколько фрагментов и сформулировать связный ответ, однако сами по себе LLM не знают локальную кодовую базу. Поэтому ответ допустимо использовать только вместе с контекстом, полученным из исходников или подготовленного хранилища.

Для ограничения таких рисков используется подход Retrieval-Augmented Generation [22]. В этом случае языковая модель получает не только пользовательский запрос, но и фрагменты, найденные в подготовленном хранилище. В контексте исследовательского поиска это позволяет разделить два этапа: сначала поисковый механизм выбирает релевантные материалы, затем языковая модель объясняет их пользователю. Такой подход снижает затраты на

первичный анализ проекта и позволяет разработчику работать с уже дообработанными данными.

Качество RAG-поиска зависит от качества подготовленного хранилища информации и реализованных методов поиска. Если данные представлены как разрозненный набор фрагментов без указания имён сущностей, связей и контекста, то поиск может возвращать неполные или слабо релевантные результаты. Напротив, если представление содержит понятные описания, идентификаторы, сведения о типе сущности и связи с другими элементами, языковой модели проще интерпретировать найденный материал. Так же если поиск реализован через точное совпадение, а запрос неточен, то получение нужного фрагмента данных крайне маловероятно.

1.5 Представление информации для разных интерпретаторов

Одна и та же кодовая база может быть полезна в разных формах. В данной работе это разделение учитывается так как система должна быть удобна каждого из акторов процесса эксплуатации.

Исходный код остаётся основным источником достоверности [21]. Только в нём можно проверить фактическую реализацию метода, порядок вызовов, условия выполнения, обращения к данным и связи между сущностями. Фактор интерпретации информации неизбежно влечёт за собой её изменение, часть данных может быть упрощена, обобщена или представлена в другом порядке. Поэтому интерпретированное представление не должно заменять исходники. Его задача — помочь быстрее найти нужный участок и затем перейти к проверке.

При этом читать исходный код как единственный источник не всегда удобно. В крупном Pascal/Delphi-проекте сведения распределены между множеством unit-файлов. Объявление сущности может находиться в одной части файла, реализация — в другой, а используемые типы и связанные методы — в соседних модулях. Если разработчик впервые работает с таким проектом, ему сначала нужно понять общую структуру: какие unit существуют, какие классы в них находятся, какие методы относятся к этим классам и где искать нужную

функциональность. Для этой задачи удобнее не полный исходный код, а обзорное представление.

Для человека такое представление должно быть читаемым и навигационным. В нём полезны заголовки, списки сущностей, краткие описания, ссылки между связанными элементами и указание исходного файла. Например, страница unit может содержать перечень классов, процедур и функций, а страница класса — список его методов и краткое описание назначения. Такой формат не раскрывает всю реализацию, но помогает быстрее понять, куда смотреть дальше.

Для программы требования другие. Ей важна не удобочитаемость, а однозначность. Автоматическому обработчику нужны фиксированные поля: имя сущности, тип сущности, путь к файлу, принадлежность к unit или классу, границы расположения в исходном коде и связи с другими элементами.

Для языковой модели требуется промежуточный вариант [1; 22]. Если передать ей только технические поля, ответ может получиться формальным и плохо связанным с пользовательским запросом. Если передать только свободное описание без привязки к исходному коду, возникает риск «галлюцинаций». Поэтому представление для LLM должно сочетать естественно-языковое описание и проверяемые признаки: имя сущности, её тип, модуль, путь к исходному файлу и связи с другими элементами.

1.6 Выводы по главе 1

В первой главе были рассмотрены основания, на которых строится дальнейшая разработка системы исследовательского поиска по Pascal/Delphi-кодовой базе.

Программный код в работе рассматривается как разновидность структурированных данных, что обеспечивает дальнейшую разработку желаемого сценария работы с кодом упрощающего порог входа в исходный код.

Также было уточнено понятие исследовательского поиска. В отличие от обычного поиска по известному имени, он начинается с описания задачи или

функциональности. Поэтому результат поиска должен помогать переходить от общего описания к конкретной программной сущности и участку исходного кода.

Рассмотренные подходы к поиску показывают, что одного механизма недостаточно. Поэтому для выбранной задачи требуется сочетать несколько уровней поиска.

Отдельное значение имеет использование языковых моделей и RAG-подхода. Языковая модель может помочь пользователю сформулировать ответ на естественном языке и связать между собой найденные сведения [1]. Для корректной работы ей требуется подготовленный контекст, предоставление которого задача RAG.

В главе также показано, что кодовая база должна иметь несколько представлений. Для человека важно обзорное и удобное для навигации описание. Для программы требуется формализованная структура с однозначными полями. Для языковой модели нужен промежуточный вариант, где технические признаки сочетаются с естественно-языковым описанием. Такое разделение позволяет использовать одну кодовую базу в разных сценариях: для чтения, автоматической обработки, поиска и формирования ответа пользователю.

Весь этот набор выводов открывает возможность перехода к формированию требований к возможностям разрабатываемой системы.

ГЛАВА 2. ОБЗОР ТЕХНОЛОГИЙ И АНАЛОГОВ

2.1. Требования к разрабатываемой информационной системе

На основании выводов, сформированных в предыдущей главе, можно приступить к разработке требований к будущему проекту. Требования можно разделить на два уровня. Те, что носят обязательный характер исполнения и те, без удовлетворения которых проект работоспособен.

К обязательным требованиям относится:

а) возможность работать с кодовыми базами на Pascal/Delphi. Данное требование связано с тем, что именно этот тип проектов выбран в работе в качестве целевого источника данных;

б) на каждом этапе иметь подходящий для ведущих акторов вид представления. Это требование обусловлено пользой наличия эффективного источника информации для своего актора что обеспечивает его наилучшее применение в процессе работы программы. То есть детерминированному алгоритму требуется дать жёсткий шаблон чтобы тот был пригоден к обработке, человеку нужна иерархическая структура с грамотным распределением смысловой информации, а LLM — промежуточный вариант, сочетающий связное описание с проверяемыми техническими признаками;

в) поддержка семантического поиска [6]. Он обеспечивает возможность формирования неточных поисковых запросов;

г) возможность интеграции с AI-агентом [1]. Данное требование обусловлено тем, что LLM это эффективный промежуточный слой между человеком и программой, позволяющий автоматизировать рутинные операции анализа и дать человеку более нужную для него информацию;

д) возможность обращения к исходному коду проекта. Данное требование проистекает из необходимости иметь гарантированный источник истины, который обеспечит точное понимание искомого в случаях, когда дообработанные варианты представления противоречат или не дают нужного уровня понимания.

Так же есть ряд требований чьё воплощение не является критически важным для базовой работоспособности системы, но являются желательными и помогают или уменьшают сложность различных процессов. К ним относятся следующие требования:

а) поддерживать локальную работу. Итоговый проект должен быть способен на офлайн работу обеспечивающую бесперебойность. Это особенно важно в связи с тем, что сеть может быть не стабильной и доступ к различным ресурсам, обеспечивающим функционал из облака, будет затруднён. Так же важности этому добавляет текущая ситуация с доступом русскоязычных провайдеров к заграничным ресурсам;

б) иметь логирование результатов и процессов. Это позволяет ускорять процессы отладки и выявление багов при разработке и эксплуатации;

в) иметь отладочный инструментарий для тестирования различных уровней функционала что обеспечивает более точное понимание источника и уровня ошибки;

г) позволять проверять источник получения информации. В случае если только LLM будет иметь доступ к источнику информации или если источники человека и большой языковой модели будут различаться, то увеличивается риск того, что ошибочный ответ модели будет не обнаружен до самого последнего момента [22];

д) Иметь грамотную архитектуру. Такая архитектура упрощает сопровождение системы, поиск нужных компонентов внутри проекта и последующее расширение под новые сценарии работы.

Данный набор требований обеспечивает понимание необходимого к реализации функционала и позволяет перейти к поиску нужных, а не похожих компонентов для его воплощения.

2.2 Pascal/Delphi-кодовая база

Pascal/Delphi-кодовая база в данной работе рассматривается как основной источник данных для разрабатываемой системы. Из неё необходимо извлекать

не только текст исходных файлов, но и сведения о программных сущностях. Такой подход связан с тем, что при сопровождении проекта разработчику важно видеть не отдельные совпадения в файлах, а структуру, в которой эти совпадения находятся.

Для Pascal/Delphi-проектов важным уровнем организации кода является unit [19]. В нём располагаются объявления типов, классов, процедур, функций, констант, переменных и других элементов программы.

При этом содержимое unit обычно разделяется на две основные части: interface и implementation. В разделе interface находятся объявления, доступные другим частям проекта, а в разделе implementation размещается фактическая реализация. Поэтому анализ только одного фрагмента файла не всегда позволяет корректно понять назначение сущности. Например, объявление метода позволяет определить его имя, параметры и принадлежность к классу, но не раскрывает выполняемые действия. Реализация метода, наоборот, показывает последовательность операций, обращения к другим сущностям и работу с данными, но без связи с объявлением и классом может восприниматься как отдельный участок кода. Следовательно, при обработке Pascal/Delphi-кода необходимо сохранять связь между unit, классом, методом, его объявлением и реализацией.

В данной работе основными объектами анализа выбраны unit, классы и методы. Такой уровень детализации является рабочим компромиссом. Если ограничиться только файлами или модулями, представление получится слишком общим: разработчик увидит перечень частей проекта, но не сможет быстро перейти к конкретной логике. Если же анализировать каждую строку или каждый оператор отдельно, объём данных станет избыточным для навигации и последующего поиска. Уровень unit, класса и метода позволяет описать структуру проекта достаточно подробно, но без превращения представления в полный пересказ исходного кода.

Дополнительная сложность связана с тем, что Pascal/Delphi-проекты часто имеют длительный срок сопровождения. В таких кодовых базах могут

встречаться участки, написанные в разное время, неодинаковый стиль оформления, неполные комментарии, сокращённые имена, устаревшие решения и зависимости, которые становятся понятными только при просмотре связанных модулей. Для разработчика, который не участвовал в создании проекта, это создаёт необходимость сначала восстановить общую структуру системы, а уже затем переходить к изменению конкретного участка кода.

С точки зрения разрабатываемой системы это означает, что Pascal/Delphi-кодовая база должна быть преобразована в промежуточное представление. В таком представлении необходимо сохранить имена сущностей, их тип, принадлежность к unit и классу, расположение в исходном файле, а также связи с другими элементами проекта. Эти данные могут использоваться для навигации, поиска по сущностям и передачи релевантного контекста языковой модели.

При этом производное представление не должно отрываться от исходного кода. Любое описание unit, класса или метода должно сохранять связь с файлом и фрагментом, из которого оно получено. Это необходимо для проверки результата: если описание оказалось неполным или неточным, разработчик должен иметь возможность обратиться к источнику и самостоятельно проверить фактическую реализацию.

2.3 Извлечение структуры кода через PasDoc

Для извлечения структуры кода выбран внешний инструмент PasDoc [7]. Разработка собственного парсера для Pascal/Delphi-файлов потребовала бы отдельно обрабатывать синтаксис языка, секции `interface` и `implementation`, объявления классов, методов, свойств и типов. Это сместило бы акцент работы с построения системы поиска и навигации на задачу синтаксического анализа. В данной работе.

В системе используется структурированный вывод PasDoc в формате SimpleXML. Этот файл служит промежуточным представлением между исходным кодом и генератором wiki: из него извлекаются имена unit, классов, методов, свойства, поля и связи между элементами. Для пользователя

SimpleXML не является итоговым результатом, но для детерминированного алгоритма генерации он удобнее исходного текста, так как уже содержит выделенные элементы кода.

При этом важно учитывать ограничения такого подхода. PasDoc остаётся внешним инструментом, поэтому полнота результата зависит от того, насколько корректно он разобрал конкретный код. В старых Pascal/Delphi-проектах могут встречаться нестандартные конструкции, неодинаковый стиль оформления, неполные комментарии и участки, которые плохо подходят для автоматического анализа. Поэтому SimpleXML следует рассматривать как производное представление, которое помогает извлечь структуру, но не заменяет исходные файлы.

2.4 Markdown-wiki и Obsidian как представление для человека

После извлечения структуры кода эти данные необходимо представить в виде, с которым сможет работать разработчик. Формат XML удобен для автоматической обработки, но для ручного просмотра он мало подходит. В нём можно хранить имена сущностей, связи и технические признаки, однако человеку сложно использовать такой файл как карту проекта. Поэтому в системе дополнительно формируется Markdown-wiki.

Markdown выбран из-за простоты и читаемости. Он позволяет представить сведения из SimpleXML в виде отдельных страниц с заголовками, списками, краткими описаниями и ссылками между сущностями. В результате разработчик работает с набором страниц, посвящённых unit и классам. Поэтому в wiki должны попадать прежде всего структура, имена сущностей, краткие описания, связи и сведения для возврата к источнику: имя unit, путь к файлу, имя класса или метода.

Задача wiki-представления — не заменить исходный код, а дать более удобный путь к нему. Страница unit может содержать список найденных классов, процедур и функций, а страница класса — его методы и связи с другими элементами проекта. За счёт этого разработчик сначала получает общее

представление о структуре, а уже затем переходит к конкретному исходному файлу.

Для просмотра такой wiki используется Obsidian [12]. Он воспринимает папку с Markdown-файлами как хранилище заметок и позволяет переходить по внутренним ссылкам, искать по содержимому и просматривать связи между страницами. Для данной работы это удобно, потому что сформированная wiki остаётся обычной папкой с файлами, но при этом может использоваться как навигационная среда. Ещё у такого подхода есть практическое преимущество. Markdown-файлы легко переносить между средами, хранить в системе контроля версий и просматривать без специализированного программного обеспечения.

Markdown-wiki также используется как материал для последующего семантического поиска. Заголовки, имена сущностей, краткие описания и связи между страницами образуют более удобный корпус, чем исходный XML. Поэтому в системе wiki выполняет две роли: помогает человеку ориентироваться в проекте и одновременно подготавливает данные для дальнейшей индексации.

2.5 Инфраструктура семантического поиска по Markdown-wiki

После формирования Markdown-wiki одной навигации по страницам недостаточно. Разработчик может открыть нужный unit или класс вручную, но при исследовательском поиске он часто не знает, где находится нужная функциональность. Поэтому по подготовленной wiki необходимо выполнять поиск по неточным запросам.

Для этой задачи используется семантический поиск. Его смысл состоит в том, что Markdown-фрагменты преобразуются в векторные представления, а затем пользовательский запрос сравнивается с ними по близости. Если в wiki есть описание сущности, её назначение, имя и связи с другими элементами, то такой фрагмент может быть найден даже без точного совпадения по имени. Например, запрос может быть сформулирован через действие или назначение, а не через конкретный идентификатор из кода.

Векторный поиск в данной системе рассматривается как возможность сузить область анализа и найти несколько вероятно подходящих фрагментов. Если запрос слишком общий или нужной функциональности в проекте нет, система всё равно может вернуть ближайшие по смыслу результаты, которые одновременно будут совершенно не подходящими. Поэтому найденные фрагменты должны использоваться как отправная точка, после чего актер проверяет сведения по исходному коду.

Для хранения векторных представлений используется Milvus [8]. Он выполняет роль векторного хранилища: сохраняет подготовленные векторы и возвращает ближайшие результаты для пользовательского запроса.

Связующим компонентом между Markdown-wiki и Milvus выступает markdown-rag-mcp [11]. В рамках данной работы он используется как backend для индексации и поиска. Он читает Markdown-документы, разбивает их на фрагменты, формирует embedding-представления, сохраняет метаданные и выполняет поиск по подготовленному корпусу. Важно, что этот инструмент работает не с .pas-файлами напрямую, а уже с wiki, сформированной на предыдущем этапе.

Такое разделение выбрано намеренно. Исходный Pascal/Delphi-код сначала преобразуется в более удобное промежуточное представление, а уже затем это представление индексируется. За счёт этого один и тот же слой используется и человеком, и поисковым механизмом. Человек просматривает Markdown-страницы в Obsidian, а семантический поиск использует эти же страницы как корпус.

Качество поиска зависит не только от Milvus или выбранной embedding-модели. На результат сильно влияет качество самой wiki: насколько полно извлечены сущности, есть ли у них понятные описания, сохранены ли связи с исходными файлами. Если PasDoc не извлёк часть структуры или описание получилось слишком общим, эта проблема перейдёт и на этап поиска. Поэтому важно сохранять возможность вернуться от найденного Markdown-фрагмента к исходному файлу с исходным кодом.

2.6 pasls как инструмент точной навигации по Pascal/Delphi-коду

В разрабатываемой системе остаётся открытой задача точного перехода к языковым символам. Если пользователь нашёл в wiki имя класса, метода, функции или типа, ему всё равно требуется проверить как он реализован в исходном коде. Для этой цели в проекте используется pasls [13].

Pasls в контексте проекта является специализированным инструментом навигации по Pascal/Delphi-коду. Его задача — обеспечить точную работу с символами языка программирования. К таким действиям относится переход к объявлению функции, типа, класса, константы или другого элемента, а также получение места, где находится соответствующий исходный код.

В типовом сценарии сначала используется поиск по wiki, чтобы определить структуру. После этого pasls позволяет перейти к точному месту объявления или реализации найденного символа. Такая связка уменьшает риск ошибки: семантический поиск помогает найти направление анализа, а pasls даёт возможность подтвердить результат по исходному коду.

Следует уточнить что позднее рассмотренный OpenCode так же имеет инструменты чтения файлов, однако они являются менее специализированными и оптимальными в контексте больших проектов.

2.7 Инструментальный доступ к кодовой базе: OpenCode, MCP и pasls

После выбора средств извлечения структуры, формирования Markdown-wiki и организации семантического поиска необходимо определить, как пользователь будет обращаться к этим возможностям. Разработчик может работать с wiki напрямую, открывая страницы unit и классов в Obsidian, но такой вариант подходит скорее для ручной навигации. Для исследовательского поиска требуется другой сценарий: пользователь формулирует вопрос на естественном языке, а система подбирает связанные с ним сведения из подготовленного представления и исходного кода.

В данной работе для этого используется OpenCode [9]. Он выступает рабочей оболочкой, через которую пользователь задаёт вопросы по кодовой базе, а AI-агент получает доступ к доступным инструментам проекта. При этом сам доступ к файлам не решает всю задачу. Агент может открыть .pas-файл и прочитать его содержимое, однако это остаётся низкоуровневой операцией. Такой доступ даёт текст файла, но не показывает заранее, где находятся нужные unit, классы, методы и связи между ними.

Поэтому OpenCode используется вместе с внешними инструментами. Их подключение выполняется через MCP [10], который позволяет агенту обращаться не только к файлам, но и к специализированным возможностям сборной системы: поиску по Markdown-wiki и навигации по Pascal/Delphi-символам через pasls.

Для поиска по Markdown-wiki требуется отдельная обёртка над markdown-rag-mcp. Сам markdown-rag-mcp выполняет индексацию и поиск по подготовленной wiki, но в данном проекте он рассматривается как backend из-за отсутствия предоставления MCP инструментов для агента. В таких условиях требуется программа связка чтобы предоставить его функции в виде инструмента, который может быть вызван из OpenCode через MCP.

После первичного поиска может использоваться pasls. Его задача отличается от семантического поиска: wiki и RAG помогают найти вероятно подходящую область проекта, а pasls позволяет уточнить найденное на уровне исходного кода. С его помощью можно перейти к объявлению, реализации или использованию Pascal/Delphi-символа, если найденный результат требует проверки.

В рабочем сценарии пользователь задаёт вопрос о функциональности проекта. Агент обращается к поиску по Markdown-wiki и получает несколько релевантных фрагментов. Если этого недостаточно, он дополнительно проверяет результат через исходные файлы или pasls. После этого ответ формируется на основе найденных данных, а не только на основе предположений языковой модели.

Такой инструментальный слой связывает несколько разных способов работы с кодовой базой. OpenCode обеспечивает среду взаимодействия, MCR подключает внешние инструменты, wrapper открывает доступ к поиску по wiki, а pasls используется для проверки найденных сущностей в исходниках.

2.8 Средства воспроизводимости, управления и оптимизации

Каждый следующий этап использует результат предыдущего: данные PasDoc становятся основой для генерации wiki, wiki используется как корпус для индексирования, индексированные фрагменты попадают в векторное хранилище, а затем становятся доступными AI-агенту через инструментальный слой. Поэтому ошибка в настройке одного этапа может проявиться уже на другом уровне — например, в виде пустых результатов поиска или неверных ссылок на исходные файлы.

Воспроизводимость в данном случае означает возможность повторить работу системы без ручного восстановления всех параметров. Для этого требуется также зафиксировать используемые пути, команды запуска, адреса сервисов, параметры индексирования и настройки внешних инструментов. Иначе одна и та же система может работать по-разному на разных машинах или после переноса проекта в другой каталог.

Одним из средств воспроизводимости является Docker [3]. В рамках данной системы он нужен для запуска Milvus, так как он работает как отдельное векторное хранилище и требует своего окружения.

Отдельное значение имеют конфигурационные файлы [21]. Система работает с несколькими каталогами: исходниками Pascal/Delphi-проекта, результатами PasDoc, сформированной wiki, индексом, логами и служебными данными. Если эти пути жёстко записывать в коде, то любое изменение расположения проекта потребует правки программы. Поэтому такие параметры должны быть вынесены в настройки.

Переменные окружения также полезны для параметров, которые зависят от конкретной среды запуска. Например, через них можно задавать путь к

backend-у и парсеру, то есть к таким объектам, которые не всегда имеют строго ожидаемое расположение.

Ещё одной частью сопровождения являются логи. В проекте много этапов, и ошибка не всегда находится там, где она проявилась. Например, плохой результат поиска может быть связан не с Milvus, а с неполной wiki, ошибкой при генерации embeddings или неправильным путём к каталогу документов. Поэтому системе нужны журналы выполнения, по которым можно понять, какой этап был запущен, какие данные использовались и где возникла проблема.

Для разработки также полезны отдельные проверочные команды и отладочные сценарии. Они позволяют проверить не всю систему сразу, а конкретный участок. Без таких проверок поиск ошибки превращается в последовательный ручной перебор всех компонентов.

Кэширование в данном случае относится прежде всего к средствам оптимизации, но оно также влияет на предсказуемость работы системы. В системе есть операции, которые нет смысла выполнять повторно при каждом запуске, если исходные данные не изменились. Например, уже подготовленные промежуточные результаты можно сохранить и использовать повторно. Это ускоряет работу и уменьшает количество лишних обращений к внешним компонентам. При этом кэш должен обновляться при изменении исходных данных, иначе система может использовать устаревшие сведения.

Конфигурационное управление особенно важно на стыке OpenCode, связки и markdown-tag-mcp. Эти компоненты должны использовать согласованные пути и команды. Если OpenCode вызывает один инструмент, связка обращается к другому каталогу, а backend индексирует третью папку, то система формально будет запускаться, но результат поиска окажется неправильным.

Поэтому для такой системы недостаточно просто подобрать рабочие инструменты. Нужно также зафиксировать порядок их запуска, входные и выходные каталоги, параметры индексирования и способ передачи данных между компонентами. В противном случае основная сложность будет возникать

не в самом поиске, а в попытке понять, какой именно этап использовал неправильные данные.

2.9 Аналоги и альтернативные подходы

К аналогам разрабатываемой системы можно отнести несколько групп инструментов. Они решают близкие задачи, но закрывают только часть выбранного сценария.

Первая группа — классические IDE для Pascal/Delphi-разработки. Основным примером является RAD Studio [19], поскольку эта среда ориентирована непосредственно на Delphi-проекты. Она предоставляет редактирование, отладку, поиск по проекту, переходы к объявлениям и реализациям, работу с формами и другие средства разработки. Такой подход удобен, когда разработчик уже понимает структуру проекта или знает примерное имя нужной сущности. Однако задача данной работы отличается от задачи IDE. Разрабатываемая система не предназначена для редактирования или отладки кода. Её роль находится на более раннем этапе: помочь разработчику разобраться в незнакомой Pascal/Delphi-кодовой базе и найти область проекта, связанную с нужной функциональностью. После этого проверка и дальнейшая работа всё равно могут выполняться в IDE.

Вторая группа — современные агентные среды работы с кодом. К ним можно отнести Cline, Cursor, Continue и GitHub Copilot coding agent [20]. Общий принцип таких инструментов состоит в том, что пользователь задаёт вопрос или задачу на естественном языке, а агент читает файлы проекта и использует доступные инструменты. Такой сценарий близок к цели данной работы, поскольку тоже предполагает взаимодействие с кодом через естественный язык. Отличие разрабатываемой системы состоит в предварительной подготовке контекста. Агентные среды обычно работают с кодовой базой как с набором файлов и текущим контекстом редактора. В данной работе перед использованием агента создаётся отдельное представление Pascal/Delphi-

проекта. Поэтому агент получает не только доступ к файлам, но и заранее подготовленный слой сведений о проекте.

Отдельно можно выделить средства генерации документации по коду. Они близки к данной системе тем, что преобразуют исходный код в более удобное описание. В разрабатываемой системе эта идея также используется, но документация не является конечным результатом. Markdown-wiki включается в общий процесс: она используется человеком для навигации, затем индексируется для семантического поиска и становится источником контекста для AI-агента.

Из этого следует что разрабатываемая система не заменяет IDE, агентную среду или генератор документации. Она соединяет несколько подходов в один локальный сценарий: Pascal/Delphi-кодовая база преобразуется в wiki, wiki используется для ручной навигации и семантического поиска, а агент получает доступ к подготовленному контексту через инструментальный слой. За счёт этого система занимает промежуточное положение между документацией, поиском по коду и AI-ассистентом.

Подход	Что решает	Отличие от разрабатываемой системы
Классическая IDE для Delphi/Pascal	Редактирование, отладка, точная навигация по исходному коду, переходы к объявлениям и реализациям	Работает напрямую с исходниками. Разрабатываемая система добавляет отдельный wiki-слой и семантический поиск по подготовленному представлению
Агентная среда работы с кодом	Позволяет задавать вопросы по проекту на естественном языке, читать файлы и использовать инструменты	Обычно опирается на доступные файлы и контекст редактора. В разрабатываемой системе перед агентом создаётся подготовленное представление Pascal/Delphi-кодовой базы
документации по коду	Преобразует код в более удобное для чтения описание	В разрабатываемой системе документационный слой не является финальным результатом: он используется как wiki для человека и как источник данных для семантического поиска

2.10 Выводы по главе 2

Во второй главе были рассмотрены требования к разрабатываемой системе и инструменты, которые могут быть использованы для их выполнения.

Главный вывод состоит в том, что исследовательский поиск по Pascal/Delphi-кодовой базе нельзя построить на одном отдельном инструменте. Для выбранной задачи требуется цепочка обработки: сначала нужно извлечь

структуру исходного кода, затем представить её в читаемом виде, после этого подготовить данные для семантического поиска и только затем подключать результат к агентной среде.

В качестве исходного материала рассматривается Pascal/Delphi-кодовая база. Её особенность состоит в том, что значимая информация распределена между `unit`, классами, методами, объявлениями и реализациями. Для извлечения такой структуры выбран PasDoc, который формирует промежуточное представление, используемое при генерации `wiki`.

Markdown-`wiki` в выбранной архитектуре выполняет две функции. Она даёт разработчику обзорный слой для изучения проекта, где страницы `unit` и классов можно открывать в Obsidian, переходить между ними и использовать как карту кодовой базы. Так же `wiki` становится корпусом для семантического поиска. Это позволяет не создавать два независимых представления, а использовать один промежуточный слой и для человека, и для дальнейшей обработки.

Для поиска по неточным запросам используется векторный подход. Milvus выполняет роль хранилища `embedding`-представлений, а `markdown-rag-mcp` используется как `backend` для индексации Markdown-документов и поиска по ним.

OpenCode выступает средой, через которую AI-агент получает доступ к инструментам проекта. Поиск по `wiki` подключается через `MCP-wrapper`, а `pasls` используется для точной навигации по Pascal/Delphi-символам. Такое разделение позволяет совместить естественно-языковой запрос, семантический поиск и проверку результата по исходному коду.

Также были рассмотрены средства, необходимые для воспроизводимости и сопровождения системы: Docker, конфигурационные файлы, переменные окружения, логи, проверочные команды и кэширование.

Рассмотрение аналогов показало, что существующие инструменты закрывают отдельные части задачи. IDE удобны для разработки и точной навигации по известным сущностям. Агентные среды позволяют работать с

кодом через естественный язык, но не всегда имеют заранее подготовленное представление проекта. Генераторы документации помогают получить описание кода, однако сами по себе не образуют полный сценарий поиска и проверки. Разрабатываемая система же строится как связка этих подходов: извлечение структуры, Markdown-wiki, семантический поиск и инструментальный доступ для агента.

Полученные выводы позволяют перейти к практической реализации системы и описанию структуры проекта.

ГЛАВА 3 РАЗРАБОТКА ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1. Постановка практической задачи

На основании выводов, полученных в первой и второй главах, практическая задача разработки заключается в создании локальной системы, которая преобразует Pascal/Delphi-кодovou базу в представление, удобное для изучения, поиска и последующей работы AI-агента. Её назначение состоит в том, чтобы подготовить промежуточный слой между исходным кодом и разработчиком: извлечь основные сущности проекта, оформить их в виде Markdown-wiki, проиндексировать это представление и предоставить к нему инструментальный доступ, где пользователь сможет задавать вопросы по проекту через агента.

Работа системы делится на два основных этапа. Первый этап связан с подготовкой данных. На нём исходные .pas-файлы обрабатываются внешним инструментом PasDoc, после чего полученное структурное представление используется генератором Markdown-wiki. В wiki должны попасть сведения о unit, классах, методах, связях между сущностями и путях к исходным файлам. При необходимости описания сущностей дополняются с помощью AI-обогащения.

Второй этап связан с использованием подготовленных данных. Сформированная Markdown-wiki открывается человеком как Obsidian-хранилище и одновременно индексируется как корпус для RAG-поиска. После индексации OpenCode-агент получает возможность обращаться к поиску через MCP-wrapper, находить релевантные Markdown-фрагменты и использовать их при ответе на вопросы пользователя. Если найденного описания недостаточно, результат должен проверяться по исходным файлам или через инструменты точной навигации по Pascal/Delphi-символам.

Основная цепочка обработки данных в системе имеет следующий вид:

- а) исходная Pascal/Delphi-кодová база передаётся на обработку;

б) PasDoc формирует структурное представление исходников в формате SimpleXML;

в) генератор преобразует полученные сведения в Markdown-wiki;

г) при необходимости выполняется AI-обогащение описаний unit, классов и методов;

д) сформированная wiki индексируется средствами семантического поиска;

е) найденные фрагменты становятся доступными AI-агенту через MCP-инструмент;

ж) при необходимости информация уточняется по исходному коду.

Входными данными для системы являются:

а) Pascal/Delphi-кодовая база, содержащая .pas и связанные с ними файлы проекта;

б) конфигурационные параметры, определяющие пути к исходникам, wiki, кэшу, логам и внешним инструментам;

в) настройки AI-обогащения, включая выбранного провайдера, модель и режимы обработки описаний;

г) внешние инструменты, необходимые для отдельных этапов работы: PasDoc, xmlstarlet, jq, markdown-rag-mcp, Milvus, pasls и MCP-wrapper.

Результатом работы системы являются несколько видов артефактов. К артефактам подготовки относятся:

а) Markdown-wiki, совместимая с Obsidian и сформированная на основе структуры Pascal/Delphi-кода;

б) векторная база данных, построенная по содержимому wiki;

в) кэш AI-обогащения, позволяющий повторно использовать уже полученные описания;

г) логи генерации, индексации, поиска и работы wrapper-а.

Отдельно следует выделить пользовательский результат. Разработчик получает возможность просматривать wiki вручную, искать по ней релевантные сущности и задавать вопросы по кодовой базе через OpenCode. Ответ AI-агента

в таком сценарии должен строиться не только на языковой модели, а на найденных фрагментах wiki и, при необходимости, проверяться по исходным файлам проекта.

3.2 Архитектура и структура проекта finalversion

Архитектура разрабатываемой системы построена как последовательная цепочка обработки Pascal/Delphi-кодовой базы. Каждый этап получает результат предыдущего этапа и подготавливает данные для следующего. Такой подход выбран потому, что системе требуется получить сразу несколько разных представлений: структурное представление для алгоритмической обработки, Markdown-wiki для человека, индекс для семантического поиска и инструментальный доступ для AI-агента.

Общая логика работы системы выглядит следующим образом:

- а) исходная Pascal/Delphi-кодовая база передаётся на обработку;
- б) PasDoc формирует промежуточное представление в формате SimpleXML;
- в) генератор wiki извлекает из SimpleXML сведения о unit, классах, методах, связанных элементах и комментариях;
- г) на основе этих данных создаётся Markdown-wiki, пригодная для просмотра в Obsidian;
- д) сформированная wiki индексируется средствами markdown-rag-mcp и сохраняется в векторном хранилище Milvus;
- е) OpenCode-агент получает доступ к поиску по wiki через MCP-wrapper;
- ж) при необходимости найденные сведения уточняются по исходным файлам.

Такое построение позволяет разделить две части работы системы. Первая часть относится к подготовке данных: извлечение структуры, генерация wiki, AI-обогащение описаний и индексация. Вторая часть относится к использованию

этих данных: поиск по подготовленной wiki, передача найденных фрагментов агенту и проверка результата по исходному коду.

Практически это разделение отражено в структуре проекта `finalversion`. В проекте выделены четыре основные области: конфигурация, собственный код, рабочие данные и внешние инструменты. Такое устройство помогает не смешивать скрипты разработки, подключённые программы и результаты выполнения.

Верхний уровень проекта имеет следующий вид:

а) `.config` — каталог локальных настроек. В нём размещаются параметры, которые зависят от конкретной машины и выбранного сценария запуска. Файл `rag-pipeline.local.conf` задаёт пути к исходному проекту, wiki, логам, `Milvus`, backend-у и wrapper-у. Файл `ai-descr.local.conf` отвечает за параметры AI-обогащения: выбранного провайдера, модель и режимы генерации описаний;

б) `AGENTS.md` — файл инструкций для OpenCode-агента. В нём задаются правила работы с проектом: когда использовать RAG-поиск, когда проверять результат через исходники или `pasls`, а также стиль взаимодействия с пользователем;

в) `code` — каталог собственного кода проекта. В `code/wiki` находится генератор Markdown-wiki, который запускает `PasDoc`, обрабатывает `SimpleXML` и создаёт страницы `unit` и `class`. В `code/pipeline` находятся скрипты общей сборки: `build-wiki-rag.sh` выполняет полный процесс от генерации wiki до индексации, а `check-rag.sh` позволяет проверить уже построенный индекс без повторного запуска всей цепочки. В `code/mcp` размещается `wrapper`, который предоставляет OpenCode доступ к RAG-поиску как к MCP-инструменту. В `code/agent` находится пример конфигурации для подключения инструментов к OpenCode;

г) `data` — каталог входных и выходных данных. В `data/input_project` размещается обрабатываемая Pascal/Delphi-кодовая база. В `data/wiki_out/current` сохраняется текущая Markdown-wiki, которую можно открыть как Obsidian-

хранилище. В data/cache находятся кэшированные результаты AI-обогащения, а в data/logs — журналы генерации, индексации, поиска и работы wrapper-a;

д) tools — каталог внешних инструментов. В него вынесены PasDoc, xmlstarlet, jq, pasls, markdown-rag-mcp и вспомогательные исполняемые файлы. Такое разделение показывает, какие части являются собственной разработкой, а какие используются как внешние компоненты/

За счёт такого разделения архитектура системы не превращается в один монолитный инструмент. Каждый компонент отвечает за свой участок. Основная структура проекта представлена на рисунке 1.

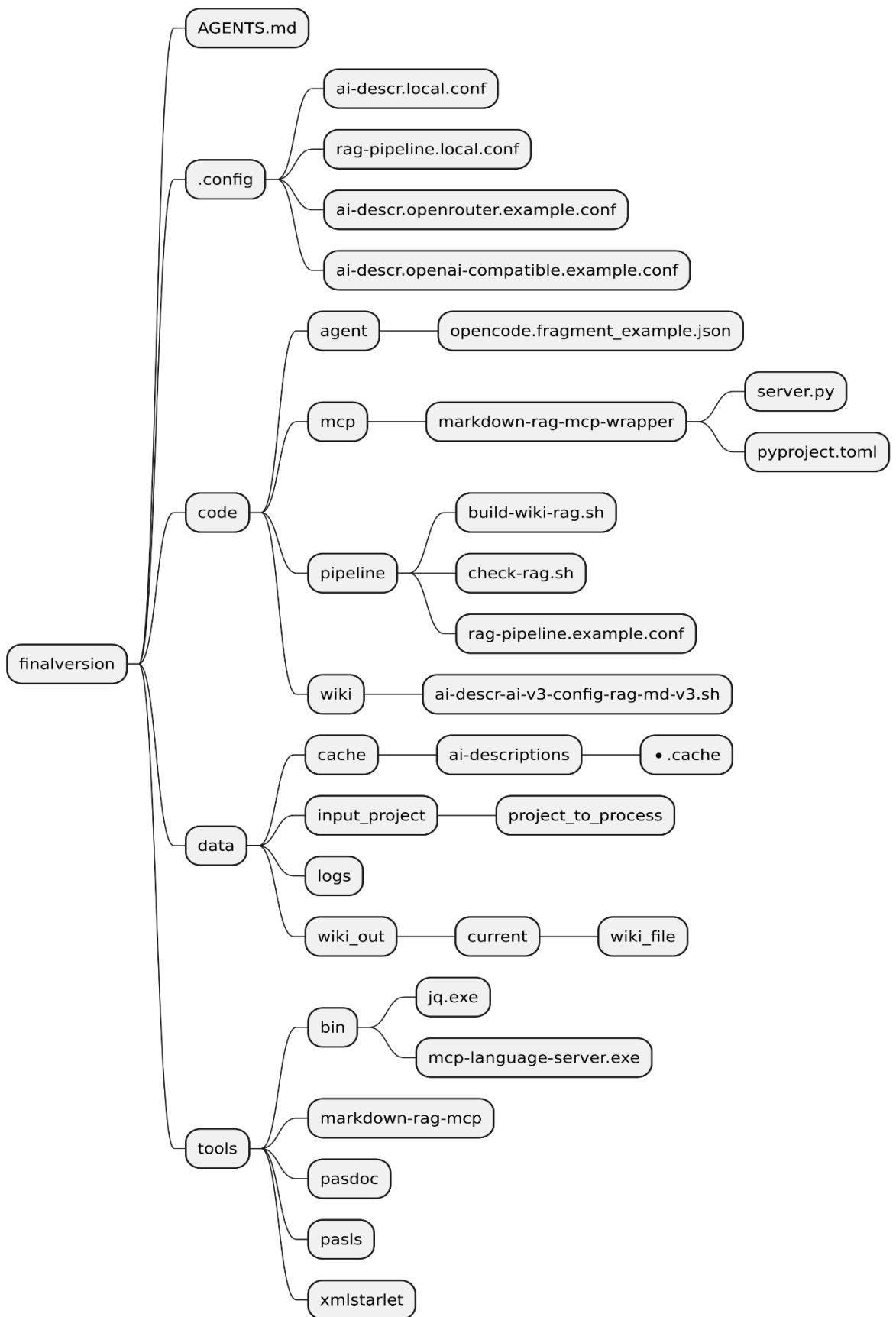


Рисунок 1. Структура проекта *finalversion*

3.3. Конфигурационный слой системы

Конфигурационный слой нужен для того, чтобы отделить изменяемые параметры запуска от программной логики. Причиной создания набора конфигов стало разрастание проекта до обилия путей и настроек, которые нецелесообразно хранить в родных файлах.

В проекте конфигурационные файлы вынесены в каталог `.config`. Основными локальными файлами являются `rag-pipeline.local.conf` и `ai-descr.local.conf`. Первый отвечает за параметры общей цепочки обработки и поиска. Второй используется для настройки AI-обогащения описаний.

Файл `rag-pipeline.local.conf` содержит параметры, связанные с запуском проекта. В нём задаются пути к исходной кодовой базе, каталогу выходной `wiki`, внешним инструментам и компонентам семантического поиска. К таким параметрам относятся расположение `PasDoc`, `markdown-rag-mcp`, `Milvus`, `wrapperr-a` и служебных каталогов таких как места хранения логов и других выходных данных. За счёт этого основной скрипт сборки может работать с разными проектами без изменения кода.

Файл `ai-descr.local.conf` относится к этапу AI-обогащения. В нём задаются параметры подключения к модели, режим использования AI-описаний и настройки, влияющие на повторное применение уже полученных результатов.

Помимо локальных конфигураций в проекте присутствуют примерные файлы: `ai-descr.openrouter.example.conf`, `ai-descr.openai-compatible.example.conf` и `rag-pipeline.example.conf`. Они нужны не как рабочие настройки, а как шаблоны. Пользователь может взять такой файл за основу чтобы имея пример написать нужные данные.

Практическая польза такого слоя состоит в том, что изменения окружения не требуют переписывания скриптов. Можно заменить обрабатываемый проект, изменить каталог `wiki`, перенести `backend` или включить другой вариант AI-подключения через настройки. Ознакомиться с листингом `ai-descr.local.conf` и `rag-pipeline.local.conf` можно в ПРИЛОЖЕНИИ 1.

3.4 Сборка проекта

Процесс сборки реализован в скрипте `build-wiki-rag.sh`. Его задача — выполнить полный цикл подготовки `wiki` и поискового индекса без ручного запуска каждого этапа отдельно. Он связывает существующие компоненты в один сценарий выполнения и относится к `build-time` части системы.

Работа начинается с загрузки конфигурации. По умолчанию используется файл `.config/rag-pipeline.local.conf`. После загрузки проверяется наличие обязательных параметров: путей к исходному проекту, `wiki`-генератору, выходной `wiki`, AI-конфигурации, `PasDoc`, `xmlstarlet`, `jq`, `markdown-rag-mcp`, `Docker Compose`-файлу `Milvus`, `uv`, каталогам логов и кэша, а также `wrapper-y`. Если один из обязательных параметров отсутствует, скрипт завершает работу с ошибкой.

Следующий этап — подготовка выходных каталогов. Скрипт создаёт папку логов, папку кэша и каталог для файла-маркера последней `wiki`. Если в конфигурации включён режим `CLEAN_WIKI_DIR`, текущая папка `wiki` предварительно очищается. После этого создаётся новый каталог `WIKI_DIR`, в который будет записан результат генерации.

Далее выполняется запуск или проверка `Milvus`. `pipeline` проверяет наличие команды `docker`, наличие `compose`-файла `Milvus` и запускает контейнер через `Docker Compose`. После запуска скрипт ожидает, пока контейнер перейдёт в состояние `healthy` или `running`. Если за заданное время `Milvus` не становится доступным, выполнение останавливается с ошибкой.

После подготовки окружения запускается генерация `Markdown-wiki`. На этом этапе `build-wiki-rag.sh` переходит в каталог генератора `wiki` и вызывает основной скрипт генерации `ai-descr-ai-v3-config-rag-md-v3.sh`. В него через переменные окружения передаются пути. Результат выполнения записывается в лог. После завершения скрипт проверяет, что папка `wiki` действительно создана и что в ней присутствует `Home.md`. Дополнительно путь к текущей `wiki` записывается в файл-маркер `LAST_WIKI_DIR_FILE`.

Следующий этап — индексация сформированной wiki, в ходе которой `markdown-rag-mcp` подготавливает фрагменты документов и сохраняет их в векторном индексе. Pipeline переходит в каталог `markdown-rag-mcp` и запускает команду `markdown-rag-mcp index` через `uv`. В качестве индексируемого каталога используется `WIKI_DIR`. Для Windows-окружения путь предварительно может быть приведён к Windows-формату через `cugpath`. Лог этого этапа сохраняется в файл.

После индексации выполняется проверка результата. Скрипт ищет в логе признаки того, что количество неуспешно обработанных файлов равно нулю. Проверяются разные варианты вывода, так как `markdown-rag-mcp` может отображать результат в текстовом, табличном или JSON-подобном виде. Если подтверждение успешной индексации не найдено, pipeline завершает работу с ошибкой и указывает путь к логу. Логирование выполняется в каталоге, заданном параметром `LOG_DIR`. При генерации — `wiki` создаётся лог вида `wiki-build-*.log`, при индексации — `rag-index-*.log`, при контрольном поиске — `rag-search-check-*.log`. Имена файлов содержат дату и время запуска, поэтому результаты разных запусков не перезаписывают друг друга. Запись выполняется через `tee`: вывод одновременно показывается в консоли и сохраняется в файл.

Последним этапом является контрольный поисковый запрос. Для него используется значение `RAG_TEST_QUERY`, а если оно не задано, применяется запрос `FindBucket`. Также из конфигурации берутся параметры `RAG_SEARCH_THRESHOLD` для указания порога близости при сравнении векторов и `RAG_SEARCH_LIMIT` для указания количества возвращаемых результатов. pipeline запускает `markdown-rag-mcp search` с выводом в JSON и проверяет, что в ответе есть поле `results`, а сам список результатов не пустой. Этот этап не оценивает качество найденного ответа, но подтверждает, что индекс создан и поисковый backend способен возвращать данные.

В конце успешного выполнения pipeline выводит основные пути: каталог `wiki`, файл-маркер последней `wiki`, каталог `RAG-backend-a` и каталог `wrapper-a`. Ознакомиться с листингом `build-wiki-rag.sh` можно в ПРИЛОЖЕНИИ 2

3.5 Генерация Markdown-wiki, RAG-ключи и AI-обогащение описаний

Генерация Markdown-wiki выполняется скриптом `ai-descr-ai-v3-config-rag-md-v3.sh`. Этот скрипт преобразует Pascal/Delphi-кодovou базу в набор Markdown-файлов. Его входом является каталог с исходными `.pas` и `.pp` файлами, а выходом — wiki-каталог, пригодный для просмотра в Obsidian и последующей индексации.

Перед началом работы скрипт загружает конфигурацию. Перед началом работы скрипт загружает конфигурацию. Если задана переменная `AI_CONFIG`, используется указанный в ней файл; иначе скрипт пытается прочитать `ai-descr.local.conf` из своего каталога.. Далее выполняется проверка зависимостей. Скрипт проверяет наличие `PasDoc`, `xmlstarlet`, `jq`, `awk`, `curl`, `cygpath`, `find`, `grep`, `md5sum`, `mktemp` и `sed`.

Отдельно создаются временные каталоги `DOC_DIR` и `STAGE_DIR`. Они размещаются во временном ASCII-пути. Это сделано для решения встреченной проблемы Windows-окружения с русским языком, так как некоторые внешние исполняемые файлы могут нестабильно работать с путями, содержащими кириллицу. Перед передачей файла в `PasDoc` исходный файл копируется во временный staging-каталог. Имя временного файла дополняется хэшем исходного пути, что предотвращает конфликт одноимённых файлов.

Основная обработка начинается с поиска Pascal/Delphi-файлов. Скрипт получает список, затем обрабатывает их по одному. Для каждого файла определяется имя `unit`. В его роли выступает либо найденное объявление `unit`, иначе берётся имя файла. После этого `PasDoc` запускается в режиме `simplexml` и формирует XML-представление структуры найденного `unit`.

Полученный SimpleXML разбирается с помощью `xmlstarlet` [14]. Из него извлекаются описания, классы, методы, свойства, поля, типы, переменные уровня `unit` и сведения о наследовании. Перед записью в Markdown текст очищается: удаляются XML/HTML-фрагменты, декодируются сущности,

нормализуются пробелы и переносы строк. Если описание отсутствует, вместо него пишется «Нет описания».

Для каждого unit создаётся отдельная Markdown-страница. В начале страницы указывается имя и путь к исходному файлу. Далее добавляется блок RAG-поисковые ключи, где сохраняются технические признаки. Для страницы unit в этот блок помещаются `kind`, `id`, `source_path`, а также списки найденных классов, типов, переменных и routines. Для страницы класса сохраняются `kind`, `id`, `unit`, `structure_type`, `source_path`, сведения о предке, методах, полях и свойствах.

Такой блок выполняет две функции. Для человека он является краткой технической сводкой по сущности. Для поискового backend-а он добавляет в индекс точные имена и связи в контексте одного чанка в векторном хранилище.

Для классов создаются отдельные Markdown-страницы. Имя файла класса строится как сочетание unit и имени класса, по схеме `UnitName.ClassName`. Это снижает риск коллизий, если в разных unit встречаются классы с одинаковыми именами. На странице класса указываются сведения о наследовании, видимость, ссылка на исходный unit, путь к исходнику и RAG-ключи класса. Затем формируются разделы с описанием, иерархией наследования, методами, полями и свойствами.

Методы внутри класса записываются отдельными подразделами. Для метода сохраняется ID, объявление, видимость и описание. Если в классе есть несколько перегрузок метода, скрипт добавляет счётчики и формирует уточнённые идентификаторы вида `Name#2`. Это нужно для отделения повторяющихся элементов, чтобы они не сливались в один объект при чтении wiki. В итоге ID может иметь вид `UnitName.ClassName.MethodName#2` в случае перегрузки и `UnitName.ClassName.MethodName` в случае её отсутствия.

При этом каждая сущность отделена от соседей своим уровнем иерархии задаваемым через «##...» это обусловлено обнаруженным ограничением markdown-rag-мср. Он не может создавать чанки размером более чем 1000 токенов и чанки нарезаются на основе иерархических знаков.

AI-обогащение используется для ситуаций, когда исходные комментарии отсутствуют или слишком короткие. Скрипт проверяет описание через функцию `needs_description`: пустое описание или слишком короткий текст считаются требующими дополнения. Присутствует возможность указывать обогащаемые сущности. Можно выборочно включать или отключать обогащение для `unit`, `class` и `method`. Это сделано так как обработка полной кодовой базы занимает значительное время, что затрудняет тестирование, а используемая для обогащения модель может быть ограничена по запросам. Разные объекты имеют разный порог слишком малого описания, после которого запускается AI-обогащение. Для `unit` и `class` порог установлен в 20 символов, так как зачастую меньшее описание малополезно и его лучше заменить. Для методов установлен порог в 12 символов. Такое значение меньше других порогов так как суть метода, может быть, в достаточной детальности описана малым количеством символов, а само значение имеет такое состояние для случаев, чтобы дата в комментарии не считалась достаточным описанием.

Для AI-обогащения поддерживаются два варианта подключения: `openrouter` и `openai-compatible`. В первом случае используется endpoint OpenRouter [17], во втором — произвольный OpenAI-compatible endpoint [18], например локальная или развёрнутая отдельно модель. Запрос к модели формируется через `jq` [15]: в него включается промпт и полный исходный файл Pascal/Delphi. После ответа из результата удаляются возможные reasoning-блоки и markdown-обёртки.

Результаты AI-обогащения кэшируются. Путь к кэш-файлу строится на основе имени `unit`, типа элемента, имени элемента, хэша исходного файла, версии схемы кэша, версии промпта, провайдера, модели, температуры и лимита токенов. Поэтому при изменении исходного файла, модели или промпта старое описание не должно ошибочно использоваться как актуальное. Так кэш обеспечивает, чтобы повторный запуск генерации не создавал заново уже полученные описания.

После обработки всех файлов создаётся индексная страница Home.md. В неё добавляются ссылки на сформированные unit-страницы и страницы классов. Благодаря этому wiki получает начальную точку входа, удобную для просмотра в Obsidian. В конце скрипт выполняет проверку результата: выводит предупреждения о пропущенных описаниях, считает количество страниц unit и классов, а также сообщает путь к созданной wiki. Диаграмма логики работы скрипта представлена на рисунке 2. Ознакомиться с листингом кода можно в ПРИЛОЖЕНИИ 3.

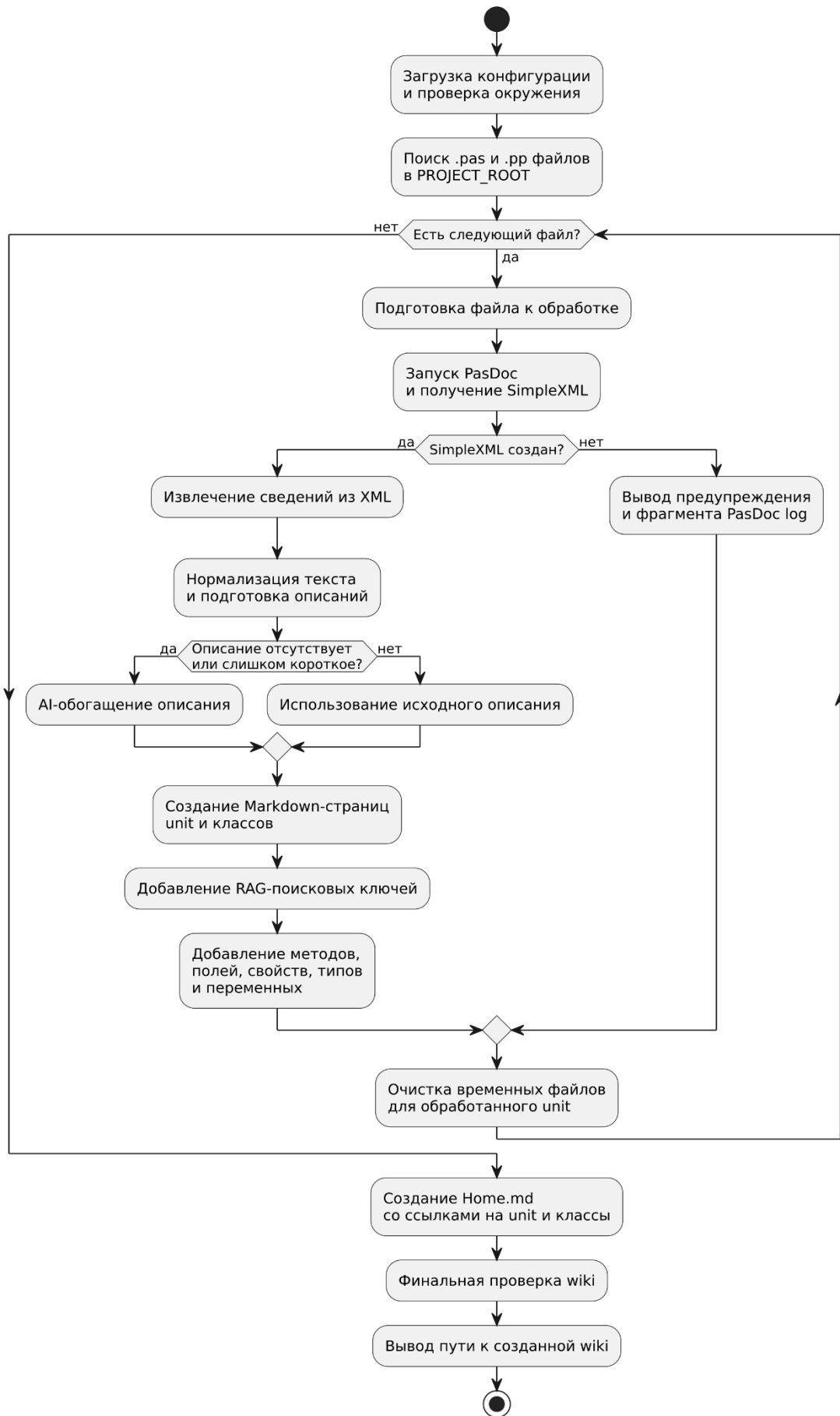


Рисунок 2. Логика скрипта *ai-descr-ai-v3-config-rag-md-v3.sh*.

3.6 Индексирование wiki и организация семантического поиска

После генерации Markdown-wiki требуется подготовить её к поиску по смысловой близости. На этом этапе система работает с готовыми Markdown-страницами.

Индексирование выполняется средствами `markdown-rag-mcp`. Его задача — принять каталог wiki, обработать Markdown-файлы, разделить их содержимое на пригодные для поиска фрагменты и сформировать для них векторные представления. После этого полученные векторы сохраняются в Milvus.

В индексирование запускается через `uv [16] run markdown-rag-mcp index` с передачей каталога сформированной wiki. В используемой конфигурации каталог передаётся параметром `--index-dir`, а также добавляются параметры рекурсивной обработки и принудительной переиндексации при необходимости.

В качестве `WIKI_DIR` передаётся каталог, созданный на этапе генерации wiki. Опциональный параметр `RAG_RECURSIVE` позволяет индексированию выполняться рекурсивно для вложенных каталогов. Параметр `RAG_FORCE_REINDEX` управляет добавлением флага `--force`, который используется для принудительной переиндексации уже обработанных документов. Это нужно в случаях, когда wiki уже была обработана ранее, но требуется полностью обновить поисковый индекс.

В Milvus попадают уже подготовленные `embedding`-векторы и связанные с ними данные. При поиске Milvus сравнивает вектор запроса с сохранёнными векторами и возвращает наиболее близкие элементы.

Для выполнения поиска используется команда: `markdown-rag-mcp search "<запрос>" --limit N --threshold T --include-metadata --format json`

Значение `limit` определяет максимальное количество возвращаемых результатов. Порог `threshold` ограничивает выдачу по степени близости. Параметр `include-metadata` нужен для того, чтобы вместе с найденным текстом получить служебные сведения о фрагменте. Вывод в JSON используется потому, что такой формат удобнее передавать следующему компоненту системы.

Содержательно поиск работает следующим образом: Пользовательский запрос преобразуется в embedding-вектор. Затем markdown-rag-mcp обращается к Milvus и получает фрагменты wiki, близкие к запросу по векторному представлению. Индекс при этом является производным результатом: если исходный код или wiki изменяются, поисковую базу требуется перестроить.

3.7 markdown-rag-mcp-wrapper

Причиной создания wrapper-а стало то, что используемый backend markdown-rag-mcp, несмотря на название, фактически предоставляет CLI и Python API [4] для индексирования и поиска, но не предоставляет готовый MCP-инструмент, который можно напрямую подключить к OpenCode. Поэтому для интеграции семантического поиска с агентной средой был создан компонент markdown-rag-mcp-wrapper. Wrapper является read-only Он предполагает, что индексация уже выполнена.

Wrapper расположен в каталоге code/mcp/markdown-rag-mcp-wrapper. Его состав: файл server.py содержит реализацию MCP-сервера, а pyproject.toml описывает зависимости. В качестве зависимостей используются markdown-rag-mcp и пакет mcp.

Основной файл wrapper-а создаёт MCP-сервер с именем markdown-rag-mcp-wrapper через FastMCP. Внутри него объявлены инструменты, которые доступны для вызова агентом:

а) `markdown_rag_ping` — быстрая проверка доступности wrapper-а. Инструмент не принимает параметров и возвращает статус success, если MCP-сервер отвечает на вызов;

б) `markdown_rag_warmup` — предварительная инициализация backend-а поиска. При вызове он создаёт и кэширует объект RAGEngine, чтобы первый реальный поисковый запрос не тратил время на инициализацию backend-а;

в) `markdown_rag_search` — основной инструмент поиска по уже построенному индексу. Он принимает параметр `query`, содержащий текст поискового запроса, `limit`, задающий максимальное количество возвращаемых

результатов, `threshold`, определяющий минимальный порог близости, и `include_metadata`, указывающий, нужно ли возвращать служебные сведения о найденных фрагментах. Перед обращением к backend-у `wrapper` проверяет параметры: `query` не должен быть пустым, `limit` должен быть больше нуля, а `threshold` должен находиться в диапазоне от 0.0 до 1.0. Если проверка не пройдена, инструмент возвращает структурированную ошибку и не выполняет поиск. Поиск выполняется через Python API `markdown-rag-mcp`. Для этого `wrapper` импортирует `get_config` и `RAGEngine`, создаёт объект движка и вызывает его метод `search`. Полученные результаты затем приводятся к компактному виду. В ответе сохраняются `confidence_score`, заголовок найденной секции, текст фрагмента, путь к файлу и часть метаданных: идентификатор документа, идентификатор секции, номер `chunk`-а, позиции начала и конца, количество токенов. Такой формат удобнее для агента, чем сырой объект backend-а. Отдельно реализовано ограничение длины возвращаемого текста. Параметр `MARKDOWN_RAG_MAX_SECTION_CHARS` задаёт максимальный размер секции, передаваемой в ответе. По умолчанию используется значение 4000 символов. Если найденный фрагмент длиннее этого ограничения, он обрезается и помечается как `truncated`. Это нужно для того, чтобы один слишком большой результат не занимал лишний контекст агента;

г) `markdown_rag_reload_engine` — сброс кэшированного объекта `RAGEngine`, то есть главного рабочего объекта `markdown-rag-mcp`, через который `wrapper` получает доступ к уже реализованной логике поиска. При первом обращении backend инициализируется, после чего объект сохраняется в глобальной переменной. Повторные поисковые запросы используют уже подготовленный движок. Инструмент нужен после переиндексации `wiki` или изменения параметров backend-а. Он позволяет заставить `wrapper` заново инициализировать поисковый движок без перезапуска `OpenCode` и всего `MCP`-сервера.

`Wrapper` также проверяет корректность расположения backend-а. Каталог `markdown-rag-mcp` определяется через переменную окружения

MARKDOWN_RAG_BACKEND_DIR, а при её отсутствии используется значение по умолчанию. Перед импортом backend-а проверяется наличие каталога, файла pyproject.toml и пакета markdown_rag_mcp. Если структура не соответствует ожидаемой, возвращается ошибка.

Поскольку MCP-сервер запускается через stdio, стандартный вывод используется не как обычная консоль, а как канал протокольного обмена с OpenCode. Поэтому любые посторонние сообщения, напечатанные backend-ом, могут нарушить JSON-RPC обмен. Для предотвращения этого wrapper на время инициализации, поиска и завершения RAGEngine перенаправляет stdout и stderr backend-а в отдельный debug-лог. Благодаря этому OpenCode получает только корректные MCP-сообщения, а диагностическая информация сохраняется в файле журнала. Путь к логу задаётся переменной MARKDOWN_RAG_DEBUG_LOG_PATH. С кодом wrapper можно ознакомиться в ПРИЛОЖЕНИИ 4

3.8 Настройка OpenCode-агента и подключение инструментов

После подготовки wiki и поискового индекса пользовательская работа с системой выполняется через OpenCode. Подключение инструментов задаётся в opencode.json. В конфигурации используются два MCP-сервера: markdown_rag_wrapper и pasls.

markdown_rag_wrapper запускается через uv из окружения markdown-rag-mcp. Это позволяет wrapper-у импортировать Python API backend-а и предоставить OpenCode инструменты markdown_rag_ping, markdown_rag_warmup, markdown_rag_search и markdown_rag_reload_engine. Для этого сервера задан увеличенный timeout, так как первый запуск backend-а может занимать больше времени из-за загрузки инициации работы в виде загрузки библиотек и запуска подскриптов

pasls подключается через mcp-language-server.exe. В конфигурации указывается рабочая папка Pascal/Delphi-проекта и путь к pasls.exe. Этот сервер

нужен для точной работы с Pascal/Delphi-символами, когда найденную через wiki сущность требуется проверить по исходному коду.

Поведение агента задаётся файлом AGENTS.md, который подключается в opencode.json через поле instructions. В нём описан порядок работы с источниками данных. Если вопрос касается архитектуры, unit, class, method, назначения сущностей или связей в проекте, агент сначала обращается к markdown_rag_wrapper.markdown_rag_search. Для точных имён методов и классов используется короткий запрос с идентификатором который возвращает 4 чанка. Для смысловых вопросов используется запрос на русском или английском языке рекомендовано получать от 5 до 7 чанков. В обоих случаях базовым порогом выбран threshold = 0.1, а include_metadata включается для получения служебных сведений о найденных фрагментах. Такое количество чанков обусловлено логикой разбиения у markdown-rag-mcp. Каждый отдельный чанк зачастую мал и не способен дать полноценной картины.

Перед первым поиском агент должен вызвать markdown_rag_warmup. Первый вызов может завершиться timeout-ошибкой, что связано с холодным запуском backend-а. После этого повторный вызов поиска выполняется быстрее.

Результаты RAG-поиска используются как навигационный контекст. Если нужен ответ по реализации, агент должен обратиться к исходному коду через встроенное чтение файлов OpenCode или через pasls. Для pasls в инструкции отдельно указано, что перед использованием его инструментов нужно запустить diagnostics. Это обусловлено столкновением с внутренними процессами pasls где для корректной работы других tool требуется предварительный прогрев через diagnostics. Также запрещены инструменты редактирования pasls и references, так как в рамках данной системы они либо дублируют существующий функционал, либо не работают. Последнее выяснено в результате тестирования.

Отдельное правило относится к неполным описаниям. Если в wiki указано Нет описания, агент не должен считать это ошибкой поиска. В этом случае используются технические поля страницы: ID, объявление, видимость,

source_path и связи с другими сущностями. Затем сведения уточняются по исходному коду.

Встроенные инструменты OpenCode используются для чтения файлов и, при явном запросе пользователя, для внесения правок. Если пользователь просит изменить код, агент сначала кратко обозначает план изменения, затем вносит минимальные правки. Если пользователь просит только поиск, объяснение или анализ, файлы изменяться не должны.

В AGENTS.md также заданы правила ответа: русский язык, краткость, опора на найденные сущности и явное обозначение сомнений. В ответах по коду агент должен указывать найденную сущность, её уровень unit/class/method, краткое назначение и, при необходимости, место в исходном коде.

Итоговый сценарий работы выглядит так: пользователь задаёт вопрос, агент обращается к RAG-поиску, получает релевантные фрагменты wiki, при необходимости проверяет их через исходники или pasls, после чего формирует ответ. Ознакомиться с листингом конфигурационного файла и системного промта модели можно в ПРИЛОЖЕНИИ 5.

3.9. Демонстрация и проверка работы системы

Демонстрация работы системы выполнялась по основной цепочке, описанной в предыдущих разделах.

Первым является этап сборки wiki и поискового индекса. Для этого запускается скрипт build-wiki-rag.sh. Представление работы скрипта показано на рисунке 3 и рисунке 4. На рисунке 3 представлена демонстрация успешного запуска скрипта и первые сообщения.

```
MINGW64/d/Works/yue6a/exp/finalversion
$ cd "/d/works/yue6a/exp/finalversion"
$ bash "code/pipeline/build-wiki-rag.sh"
[Pipeline] Очистка WIKI_DIR: D:/works/yue6a/exp/finalversion/data/wiki_out/current
[Pipeline] CONFIG_FILE: /d/works/yue6a/exp/finalversion/.config/rag-pipeline.local.conf
[Pipeline] FINAL_ROOT: D:/works/yue6a/exp/finalversion
[Pipeline] PROJECT_ROOT: D:/works/yue6a/exp/finalversion/data/input_project/src
[Pipeline] WIKI_DIR: D:/works/yue6a/exp/finalversion/data/wiki_out/current
[Pipeline] Занесем в репозиторий Milvus вексы Docker Compose
times="2026-05-23T15:23:44+03:00" level=warning msg="D:/works/yue6a/exp/finalversion/tools/markdown-rag-mcp/docker/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 6/8
✔ Network docker_milvus Created 0.1s
✔ Volume docker_milvus Created 0.0s
✔ Volume docker_etcd Created 0.0s
✔ Volume docker_minio Created 0.0s
✔ Container milvus-etcd Started 0.6s
✔ Container milvus-minio Started 0.7s
✔ Container milvus-standalone Started 0.8s
✔ Container milvus-attu Started 1.0s
[Pipeline] Milvus rotom: milvus-standalone (healthy)
[Pipeline] Генерируем Markdown wiki
[Pipeline] WIKI_DIR: D:/works/yue6a/exp/finalversion/data/wiki_out/current
[Pipeline] Лог wiki: D:/works/yue6a/exp/finalversion/data/logs/wiki-build-2026-05-23-15-23-52.log
CONFIG_FILE: D:/works/yue6a/exp/finalversion/.config/at-descr.local.conf
CONFIG_LOADED: true
PROJECT_ROOT: D:/works/yue6a/exp/finalversion/data/input_project/src
PASDOC_PROG: D:/works/yue6a/exp/finalversion/tools/pasdoc/bin/pasdoc.exe
XMLSTARLET_BIN: D:/works/yue6a/exp/finalversion/tools/xmlstarlet/xmlstarlet-1.6.1/xm1.exe
ML_BIN: D:/works/yue6a/exp/finalversion/tools/bin/js.exe
WORK_ROOT: /tmp/pasdoc_wiki_2026-05-23-15-23-52_wdkiB
DOC_DIR: /tmp/pasdoc_wiki_2026-05-23-15-23-52_wdkiB/docs_tmp
STAGE_DIR: /tmp/pasdoc_wiki_2026-05-23-15-23-52_wdkiB/stage
WIKI_DIR: D:/works/yue6a/exp/finalversion/data/wiki_out/current
AI_PROVIDER: openrouter
AI_URL: https://openrouter.ai/api/v1/chat/completions
AI_MODEL: poolside/lapuna-m.1:free
AI_ENRICH_UNITS: true
AI_ENRICH_CLASSES: true
AI_ENRICH_METHODS: false
CACHE_DIR: D:/works/yue6a/exp/finalversion/data/cache/ai-descriptions
[1] Ошибка: D:/works/yue6a/exp/finalversion/data/input_project/src/GpLockFreeQueue.pas
AI generate: GpLockFreeQueue/class/TGpLockFreeQueue
OK: GpLockFreeQueue
[2] Ошибка: D:/works/yue6a/exp/finalversion/data/input_project/src/GpProperty.pas
AI generate: GpProperty/unit/GpProperty
OK: GpProperty
[3] Ошибка: D:/works/yue6a/exp/finalversion/data/input_project/src/GpStringHash.pas
AI generate: GpStringHash/class/TGpStringHash
AI generate: GpStringHash/class/TGpStringHashEnumerator
AI generate: GpStringHash/class/TGpStringHash
AI generate: GpStringHash/class/TGpStringObjectHash
AI generate: GpStringHash/class/TGpStringObjectHashEnumerator
AI generate: GpStringHash/class/TGpStringInterfaceHash
AI generate: GpStringHash/class/TGpStringInterfaceHashEnumerator
AI generate: GpStringHash/class/TGpStringInterfaceHash
AI generate: GpStringHash/class/TGpStringTablekv
```

Рисунок 3. Успешный старт build-wiki-rag.sh

На рисунке 4 показано успешное завершение работы скрипта. После индексации выполнялся контрольный поиск. Для него использовалась команда `markdown-rag-mcp search` с тестовым запросом. В проверочном сценарии важно не содержание найденного фрагмента, а факт, что backend корректно возвращает JSON-ответ.

```
MINGW64/d/Works/yue6a/exp/finalversion
Metadata: Yes
2026-05-23 15:35:27,671 - markdown_rag_mcp.core.rag_engine - INFO - Initializing RAG engine components...
2026-05-23 15:35:27,671 - markdown_rag_mcp.embeddings.embedder - INFO - Initializing HuggingFace embedder: sentence-transformers/all-MiniLM-L6-v2 on cpu
2026-05-23 15:35:27,671 - markdown_rag_mcp.embeddings.embedder - INFO - Loading embedding model: sentence-transformers/all-MiniLM-L6-v2
2026-05-23 15:35:30,696 - sentence_transformers.SentenceTransformer - INFO - Load pretrained SentenceTransformer: sentence-transformers/all-MiniLM-L6-v2
2026-05-23 15:35:33,865 - markdown_rag_mcp.embeddings.embedder - INFO - Model loaded successfully on device: cpu
2026-05-23 15:35:33,865 - markdown_rag_mcp.core.rag_engine - INFO - LangChain embedding adapter initialized
2026-05-23 15:35:33,865 - markdown_rag_mcp.core.rag_engine - INFO - Database 'markdown_rag' already exists
2026-05-23 15:35:34,190 - markdown_rag_mcp.storage.milvus_store - INFO - Milvus vector store initialized successfully
2026-05-23 15:35:34,190 - markdown_rag_mcp.core.rag_engine - INFO - LangChain vector store initialized
2026-05-23 15:35:34,190 - markdown_rag_mcp.core.rag_engine - INFO - Document parser initialized
2026-05-23 15:35:34,190 - markdown_rag_mcp.core.rag_engine - INFO - Query processor initialized
2026-05-23 15:35:34,190 - markdown_rag_mcp.core.rag_engine - INFO - Document indexer initialized with frontmatter enhancement
2026-05-23 15:35:34,190 - markdown_rag_mcp.core.rag_engine - INFO - Incremental indexer initialized with change detection
2026-05-23 15:35:34,190 - markdown_rag_mcp.core.rag_engine - INFO - Monitoring coordinator initialized with incremental updates
2026-05-23 15:35:34,190 - markdown_rag_mcp.core.rag_engine - INFO - RAG engine initialization complete
2026-05-23 15:35:34,190 - markdown_rag_mcp.core.rag_engine - INFO - RAG engine initialization complete
Initializing search engine...
Searching documents... 0:00:00
Batches: 100% 1/1 [00:00:00:00, 49.96it/s]
{
  "results": [
    {
      "confidence_score": 0.4754,
      "file_path": "doc_9d677b8-eb8a-40de-9ff1-a988cab3def",
      "section_heading": "Method: TGpStringDictionary.FindBucket",
      "section_text": "Method: TGpStringDictionary.FindBucket\n\n**ID:** 'GpStringHash.TGpStringDictionary.FindBucket'\n\n**Function:** 'function FindBucket(const key: string): cardinal: 'n''\n\n**ID:** 'GpStringHash.TGpStringDictionary.FindBucket'\n\n**Protected:** 'n''\n\n**ID:** 'GpStringHash.TGpStringDictionary.FindBucket'\n\n**Created at:** '2026-05-23T12:35:15.496021+00:00'\n\n**pk:** 466495478649064899,\n\n**token_count:** 50,\n\n**heading_level:** 3,\n\n**file_path:** 'doc_9d677b8-eb8a-40de-9ff1-a988cab3def',\n\n**chunk_index:** 4,\n\n**section_type:** 'heading',\n\n**start_position:** 1033,\n\n**end_position:** 1239\n    }\n  ]
}
```

Рисунок 4. Успешное завершение build-wiki-rag.sh

После выполнения `build-wiki-rag.sh` доступна сформированная Markdown-wiki. В выходном каталоге должны появиться файл `Home.md`, страницы `unit` и

The screenshot shows the Graph Explorer application interface. On the left, a sidebar lists the nodes in the graph, including `GpLockFreeQueue`, `GpStringHash`, `GpStringHash.TGpStringDictionary`, `GpStringHash.TGpStringDictionaryEnum...`, `GpStringHash.TGpStringDictionaryKV`, `GpStringHash.TGpStringHash`, `GpStringHash.TGpStringHashEnumerator`, `GpStringHash.TGpStringHashKV`, `GpStringHash.TGpStringInterfaceHash`, `GpStringHash.TGpStringInterfaceHashEn...`, `GpStringHash.TGpStringInterfaceHashKV`, `GpStringHash.TGpStringObjectHash`, `GpStringHash.TGpStringObjectHashEnu...`, `GpStringHash.TGpStringObjectHashKV`, `GpStringHash.TGpStringTable`, `GpStringHash.TGpStringTableEnumerator`, `GpStringHash.TGpStringTableKV`, and `Home`. The main area displays a graph visualization of these nodes and their relationships. The graph shows a central node `Home` connected to several other nodes, including `GpLockFreeQueue`, `GpStringHash`, `GpStringHash.TGpStringDictionary`, `GpStringHash.TGpStringDictionaryEnum...`, `GpStringHash.TGpStringDictionaryKV`, `GpStringHash.TGpStringHash`, `GpStringHash.TGpStringHashEnumerator`, `GpStringHash.TGpStringHashKV`, `GpStringHash.TGpStringInterfaceHash`, `GpStringHash.TGpStringInterfaceHashEn...`, `GpStringHash.TGpStringInterfaceHashKV`, `GpStringHash.TGpStringObjectHash`, `GpStringHash.TGpStringObjectHashEnu...`, `GpStringHash.TGpStringObjectHashKV`, `GpStringHash.TGpStringTable`, `GpStringHash.TGpStringTableEnumerator`, `GpStringHash.TGpStringTableKV`, and `GpProperty`. The graph is a complex network of interconnected nodes, with `Home` acting as a central hub.

Дополнительно доступна к просмотру структура отдельной страницы wiki. На рисунке 6 показано начало страницы unit.

GpProperty

Unit: GpProperty

Исходник: `src/GpProperty.pas`

RAG-поисковые ключи

- kind: unit
- id: GpProperty
- source_path: src/GpProperty.pas
- routines: CreateGpProperty, CreateGpProperty, GetFloatProp, SetFloatProp, GetWideStrProp, SetWideStrProp, SetToString, StringToSet

Общее описание

GpProperty

Назначение: Упрощённый доступ к опубликованным (published) свойствам объектов Delphi.

Рисунок 6. Фрагмент страницы unit в Markdown-wiki

Отдельно происходит использование OpenCode в пользовательском сценарии. На рисунках 7 и 8 показан пример вызова RAG-поиска из OpenCode через MCP-warpper. На рисунке 7 находится запрос пользователя и начало работы агента

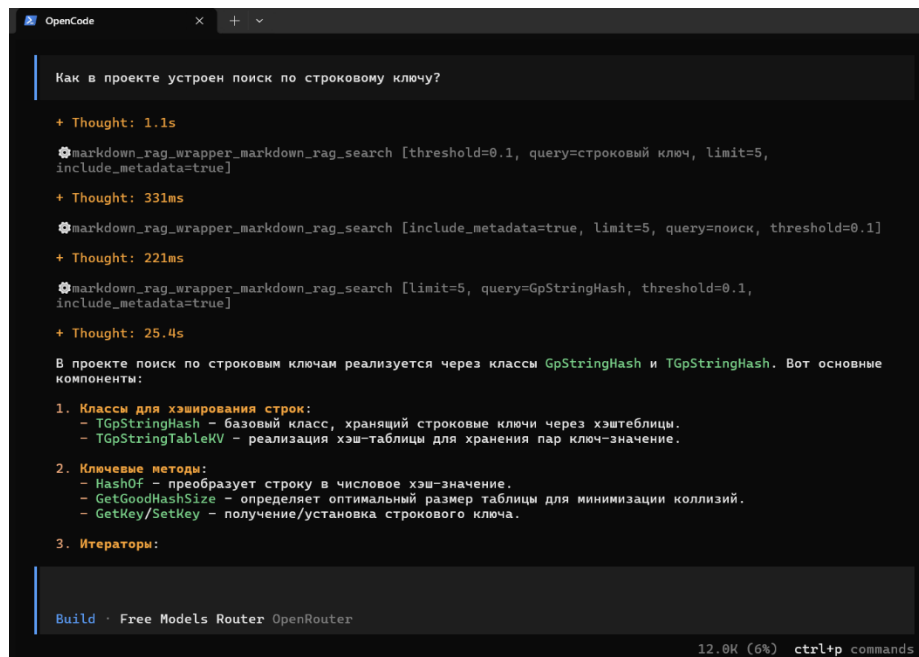


Рисунок 7. Использование AI-агента для получения информации по коду, начало

Далее на рисунке 8 представлен полный ответ модели.

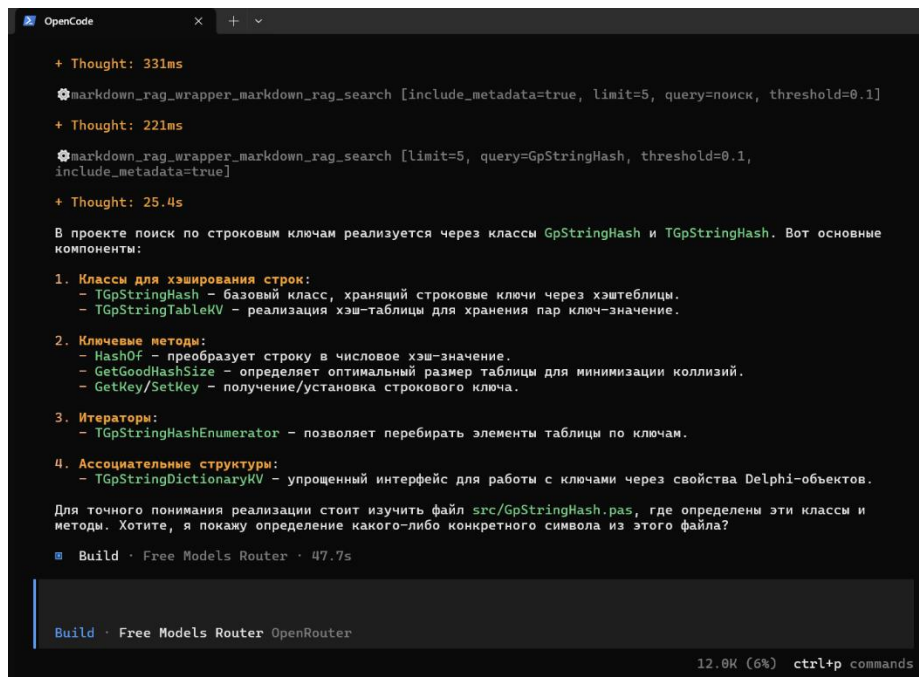


Рисунок 8. Использование AI-агента для получения информации по коду, полный ответ

3.10 Ограничения и направления развития

Разработанная система выполняет основную практическую задачу, однако её работу нельзя рассматривать как полностью автоматический анализ

Pascal/Delphi-проекта без ограничений. Часть ограничений связана с внешними инструментами, часть — с выбранной схемой представления данных.

К основным ограничениям относятся:

а) зависимость от результата PasDoc. Система строит wiki не напрямую из исходного текста, а из SimpleXML, сформированного PasDoc. Поэтому полнота wiki зависит от того, насколько корректно PasDoc обработал конкретные .pas-файлы. Если в проекте используются нестандартные конструкции, сложные директивы компиляции или участки, плохо поддерживаемые инструментом, часть сведений может быть извлечена неполно. В таком случае wiki будет отражать только ту структуру, которая попала в SimpleXML;

б) ограниченность семантического поиска. Векторный поиск возвращает фрагменты, близкие к запросу по embedding-представлению, но такая близость не гарантирует совпадение с реальной логикой программы. Если запрос слишком общий или нужная функциональность отсутствует в проекте, поиск всё равно может вернуть ближайшие по смыслу, но неверные фрагменты и занять контекст модели нерелевантной информацией. Поэтому результат RAG-поиска используется как направление анализа, а не как доказательство правильности найденной реализации;

в) зависимость результата от разбиения Markdown на фрагменты. Wiki индексируется не как цельная документация, а как набор секций. Поэтому при поиске возвращается не вся страница unit или класса, а отдельный фрагмент Markdown. Это влияет и на RAG-ключи: они полезны только в том случае, если попали в индексируемую секцию, которая затем вернулась в результате поиска. Если технические ключи находятся отдельно от содержательного описания, агент может получить неполный контекст и ему потребуется дополнительный переход к странице wiki или исходному файлу;

г) время подготовки данных. Генерация wiki, AI-обогащение и индексация занимают время, особенно если включено обогащение методов. Поэтому в системе предусмотрено выборочное включение AI-обогащения для unit, классов и методов. Это ускоряет тестовые запуски, но влияет на полноту

описаний. В тестовом проекте объёмом 98 КБ кода подготовка без AI-обогащения занимала примерно 1–2 минуты, с обогащением только unit — около 3 минут, с обогащением классов — до 20 минут, а с обогащением методов — до часа;

д) сложность локальной инфраструктуры. Система использует несколько внешних компонентов: PasDoc, xmlstarlet, jq, Docker, Milvus, markdown-rag-mcp, wrapper, OpenCode и pasls. Из-за этого требуется корректная настройка путей, окружения и зависимостей. Ошибка в одном параметре может проявиться не сразу, а на более позднем этапе, например при индексации или вызове поиска из OpenCode. В процессе работы уже встречался случай, когда после серии неудачных запусков требовалась очистка сохранённых данных Milvus и связанных кэшей;

е) ограничения pasls. В текущем варианте pasls используется для точной навигации по Pascal/Delphi-коду, но не все его возможности применимы. Часть дублирует существующий функционал, часть не работает. Поэтому в инструкции агента предусмотрен предварительный запуск diagnostics, а часть инструментов pasls запрещена к использованию. Это снижает риск некорректной работы, но ограничивает автоматизированные сценарии анализа;

ж) ориентация системы прежде всего на поиск, а не на редактирование. Проект помогает найти и объяснить нужную область кодовой базы, но не предназначен для активного цикла. После крупных изменений в исходниках требуется заново выполнить цепочку подготовки: сгенерировать wiki и обновить векторную базу.

з) необходимость предварительных действий при работе с агентом. Первый запуск поиска по векторной базе требует прогрева, который требуется вызывать агенту и который увеличивает время, через которое пользователь может приступить к работе с агентом. То же самое касается pasls.diagnostic без его запуска остальные инструменты pasls не работают.

На основании этих ограничений можно выделить следующие направления развития:

а) проверка и дополнение результата PasDoc. Можно добавить отдельный этап валидации SimpleXML: проверять, какие .pas-файлы не дали XML, какие unit не содержат ожидаемых сущностей, где отсутствуют классы или методы. Собрав статистику необрабатываемого функционала, можно разработать дополнительный мини-парсер, закрывающий выявленные пробелы обработки PasDoc;

б) улучшение структуры Markdown-wiki с учётом будущей индексации. Сейчас RAG-ключи добавляются в страницы, но эффективность их использования зависит от того, как Markdown затем режется на секции. Поэтому развитие wiki должно учитывать не только удобство чтения в Obsidian, но и поведение индексатора. Например, можно размещать краткие технические ключи ближе к соответствующим описаниям, добавлять компактные summary-блоки для класса или unit, а также продумать отдельные страницы или секции для методов, процедур и типов;

в) расширение инструментов wrapper-a. Сейчас основной инструмент wrapper-a — векторный поиск по wiki. В дальнейшем можно добавить более точные инструменты: получить страницу по точному совпадению с объектами в запросе, искать только по unit, только по классам, вернуть соседние секции страницы, показать связанные сущности или открыть полный Markdown-файл по результату поиска. Это уменьшит ситуацию, когда агент получил только небольшой фрагмент и вынужден самостоятельно догадываться, где искать продолжение. Иными словами, развитием решения к проблеме неточности поиска является создание гибридной, настраиваемой системы поисковых запросов;

г) улучшение AI-обогащения. Можно доработать промпты, добавить разные режимы описаний для unit, классов и методов, а также отдельную проверку качества сгенерированных описаний и способ нарезания кода чтобы модель гарантированно не получила больше, чем допускает её контекст и не дала ложного ответа на неполностью воспринятом сообщении;

д) упрощение развёртывания. Текущая система зависит от локальных путей и набора внешних программ. В дальнейшем можно упростить начальную настройку: добавить единый сценарий проверки окружения, расширить Docker-конфигурацию, подготовить шаблоны конфигов и сделать диагностику типовых ошибок более явной. Сделать часть путей автоматически настраиваемыми.

е) Развитие методов неполного обновления. Проблему невозможности работать с кодовой базой для полноценного редактирования без потери актуальности wiki и векторной базы можно решить точечным обновлением данных. Это упростит затраты на актуализацию данных

ж) переход к более структурированному хранилищу контекста для RAG. Текущий RAG ограничен логикой чанкенизации и доступом только к векторному поиску. Эту проблему можно решить, создавая улучшенные версии уже существующего функционала, но так же доступен переход на более совершенное, в контексте исходного кода, хранилище информации для RAG такие как графовые базы данных и базы знаний.

з) автоматизация прогревочных действий. Можно автоматизировать запуск прогревов, чтобы пользователю ненужно было держать в голове и тратить время на этот аспект эксплуатации.

3.11 Выводы по главе 3

В третьей главе была описана практическая реализация системы для исследовательского поиска по Pascal/Delphi-кодовой базе. Основная задача реализации состояла в том, чтобы связать исходный код, автоматически сформированную Markdown-wiki, семантический поиск и AI-агента в один рабочий сценарий.

В ходе разработки была реализована цепочка подготовки данных. Исходные Pascal/Delphi-файлы обрабатываются через PasDoc, полученное SimpleXML-представление используется генератором wiki, после чего создаются Markdown-страницы unit и классов. В них сохраняются описания, связи, технические признаки и сведения, необходимые для возврата к исходному

коду. Для неполных описаний предусмотрено AI-обогащение, а повторное использование уже полученных результатов обеспечивается кэшем.

Для поиска по подготовленной wiki используется отдельный этап индексации. Markdown-файлы передаются в markdown-rag-mcp, где разбиваются на фрагменты и преобразуются в векторные представления [22]. Эти данные сохраняются в Milvus и затем используются при семантическом поиске.

Для подключения поиска к агентной среде был реализован markdown-rag-mcp-wrapper. Он предоставляет OpenCode набор MCP-инструментов для проверки доступности, прогрева backend-a, выполнения поиска и сброса кэшированного объекта RAGEngine. Отдельное внимание было уделено защите MCP-обмена через stdio: лишний вывод backend-a перенаправляется в debug-лог, чтобы не нарушать JSON-RPC обмен между OpenCode и MCP-сервером.

Пользовательская работа с системой организована через два сценария. Первый сценарий связан с ручным изучением результата: сформированную Markdown-wiki можно открыть в Obsidian и использовать как навигационное представление проекта. Второй сценарий связан с OpenCode: агент получает доступ к RAG-поиску, использует найденные фрагменты wiki как навигационный контекст и при необходимости уточняет результат по исходным файлам или через pasls.

Проверка работы показала, что реализованная цепочка выполняет полный цикл: запускается сборка, формируется Markdown-wiki, выполняется индексация, контрольный поиск возвращает результаты, а агент получает доступ к найденным фрагментам через MCP-wrapper. Это подтверждает работоспособность выбранной архитектуры на уровне основного сценария.

Вместе с тем были выявлены ограничения системы. Наиболее существенными являются зависимость от качества разбора PasDoc, особенности разбиения Markdown на фрагменты, неточность семантического поиска, время AI-обогащения и сложность локальной инфраструктуры. Также система в текущем виде больше ориентирована на поиск и анализ, чем на постоянное редактирование кода с автоматическим обновлением wiki и индекса.

По результатам разработки можно сделать вывод, что поставленная практическая задача выполнена.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы была разработана информационная система для исследовательского поиска по Pascal/Delphi-кодовой базе.

В теоретической части работы рассмотрены предпосылки позволяющие организовать созданную логику работы с кодовой базой. Было показано, что для Pascal/Delphi-проектов важны не только строки исходного файла, но и устойчивые сущности и связи между ними. На этой основе было уточнено понятие исследовательского поиска.

Также были рассмотрены разные подходы к поиску в программном коде. Было установлено, что ни один из них не решает задачу полностью. Текстовый и лексический поиск для известных идентификаторов, синтаксический и навигационный — для структуры и связей, документационный и семантический — для поиска по назначению. Отдельно было определено, что языковая модель должна выступать в роли интерпретатора найденного контекста, а достоверность результата должна обеспечиваться связью с исходным кодом.

Во второй главе были сформулированы требования к разрабатываемой системе и выбран набор технологий для их выполнения. В качестве источника структурных данных была выбрана Pascal/Delphi-кодовая база, а для извлечения сведений о ней — PasDoc. Для человекочитаемого представления Markdown-wiki, пригодный для просмотра в Obsidian. Для семантического поиска markdown-rag-mcp и Milvus, а для агентного сценария — OpenCode, MCP-wrapper и pasls. Рассмотрение аналогов показало, что существующие инструменты закрывают отдельные части задачи, но не образуют такой же цепочки, которая была реализована в ходе работы.

Практическая часть была выполнена в виде проекта finalversion. В нём реализована цепочка обработки данных: исходные Pascal/Delphi-файлы передаются в PasDoc, результат SimpleXML используется генератором Markdown-wiki, полученные страницы дополняются техническими признаками

и при необходимости AI-описаниями, после чего wiki индексируется для семантического поиска. Для повторяемости процесса создан pipeline сборки, использующий конфигурационные файлы, логи, кэш и проверочные команды.

Отдельным результатом разработки является MCP-wrapper над markdown-rag-mcp. Он позволяет подключить поиск по wiki к OpenCode как инструмент агента. Благодаря этому пользователь может задавать вопросы по кодовой базе на естественном языке, а агент — получать релевантные фрагменты wiki, использовать их как навигационный контекст и при необходимости уточнять сведения по исходным файлам или через pasls.

Проверка работы показала, что разработанная система выполняет полный цикл подготовки и использования данных: запускается оркестратор, формируется Markdown-wiki, выполняется индексация, контрольный поиск возвращает результаты, а OpenCode-агент получает доступ к найденному контексту через MCP-wrapper. Это подтверждает выполнение практической задачи, поставленной в работе.

При этом система имеет ограничения. Качество результата зависит от полноты разбора PasDoc, структуры исходных комментариев, особенностей разбиения Markdown на фрагменты, корректности локальной конфигурации и ограничений используемых внешних инструментов. Кроме того, текущая реализация больше подходит для поиска и анализа проекта, чем для постоянного редактирования.

В дальнейшем развитие системы связано с улучшением связности производных форматов и исходного кода, расширением методов поиска, частичным обновлением wiki и векторной базы, а также упрощением развёртывания. Эти направления позволят повысить точность поиска, уменьшить зависимость от ручной проверки и сделать систему удобнее при работе с более крупными Pascal/Delphi-проектами. Отдельным масштабным улучшением является полный переход на другой носитель данных для RAG такой как графовая база данных или база знаний.

Из этого следует что в работе была создана локальная информационная система, которая формирует навигационно-поисковый слой над Pascal/Delphi-кодовой базой. Полученное решение снижает сложность первичного изучения проекта за счёт уникальной связки существующих технологий и подходов в единый процесс.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Намиот Д.Е., Ильюшин Е.А. Архитектура LLM агентов /International Journal of Open Information Technologies. 2025. Т. 13. № 1. с. 67–74.
2. Brown T.B., Mann B., Ryder N. и др. Language Models are Few-Shot Learners / Advances in Neural Information Processing Systems (NeurIPS). 2020. Режим доступа: <https://arxiv.org/abs/2005.14165>
3. Docker. Get started. — Режим доступа: <https://docs.docker.com/>
4. Python Software Foundation.— Режим доступа: <https://docs.python.org/>
5. Алпатов А.Н., Соловьев Р.В. Архитектура программной системы автоматизации рефакторинга кода на основе метода комбинаторных парсеров / Современные наукоемкие технологии. 2022. № 4. с. 9–15
6. Антонов И. В., Бруттан Ю. В. Применение больших языковых моделей для интеллектуального поиска и извлечения информации в корпоративных информационных системах / Компьютерные исследования и моделирование. 2025. Т. 17, № 5. с. 871–888.
7. PasDoc. Documentation tool for Pascal source code. — Режим доступа: <https://pasdoc.github.io/>
8. Milvus. Documentation. — Режим доступа: <https://milvus.io/docs>
9. OpenCode. Documentation. — Режим доступа: <https://opencode.ai/docs/>
10. Model Context Protocol. Introduction. — Режим доступа: <https://modelcontextprotocol.io/>
11. markdown-rag-mcp. Markdown RAG MCP. — Режим доступа: <https://github.com/mohllal/markdown-rag-mcp>
12. Obsidian. Help and documentation. — Режим доступа: <https://help.obsidian.md/>
13. pasls. Pascal Language Server. — Режим доступа: <https://github.com/genericptr/pasls>
14. xmlstarlet. XMLStarlet command line XML toolkit. — Режим доступа: <https://xmlstar.sourceforge.net/>
15. jq. jq Manual. — Режим доступа: <https://jqlang.github.io/jq/manual/>

16. uv. Documentation. — Режим доступа: <https://docs.astral.sh/uv/>
17. OpenRouter. API documentation. — Режим доступа: <https://openrouter.ai/docs>
18. OpenAI. API Reference. — Режим доступа: <https://platform.openai.com/docs/api-reference>
19. Embarcadero. RAD Studio documentation. — Режим доступа: <https://docwiki.embarcadero.com/RADStudio/>
20. Cline. Documentation. — Режим доступа: <https://docs.cline.bot/>
21. ГОСТ Р ИСО/МЭК ТО 9294-93. Информационная технология. Руководство по управлению документированием программного обеспечения. — Режим доступа: <https://normativ.kontur.ru/document?documentId=212171&moduleId=9>
22. Gao Y., Xiong Y., Gao X. и др. Retrieval-Augmented Generation for Large Language Models: A Survey. 2023. Режим доступа: <https://arxiv.org/abs/2312.10997>

ПРИЛОЖЕНИЕ 1. Листинг ai-descr.local.conf и rag-pipeline.local.conf

В приложении приведены конфигурационные файлы, используемые для настройки AI-обогащения описаний и общей цепочки генерации Markdown-wiki с последующей индексацией для RAG-поиска.

Листинг 1.1 — файл ai-descr.local.conf

```
# провайдер, который будет использоваться для AI-обогащения описаний.
AI_PROVIDER="openrouter"
# Указывается URL OpenAI-compatible API. В текущей конфигурации он задан для локального сервера, но при
использовании OpenRouter игнорируется.
AI_URL="http://127.0.0.1:11434/v1/chat/completions"
# Указывается API-ключ OpenRouter. Значение используется только при выборе AI_PROVIDER="openrouter".
OPENROUTER_API_KEY="sk-or-v1-"
# Указывается модель, используемая для генерации описаний.
AI_MODEL="poolside/laguna-m.1:free"
# Указывается температура генерации. Низкое значение уменьшает вариативность ответа модели.
AI_TEMPERATURE="0.1"
# Указывается максимальное количество токенов в ответе модели.
AI_MAX_TOKENS="800"
# режим работы AI-обогащение описаний для страниц unit.
AI_ENRICH_UNITS="true"
# режим работы AI-обогащение описаний для страниц классов.
AI_ENRICH_CLASSES="true"
# режим работы AI-обогащение описаний методов.
AI_ENRICH_METHODS="false"
# `ai-descr.local.conf` может содержать настройки сразу для обоих AI-provider-ов.
# Если `AI_PROVIDER="openrouter"`:
```

Листинг 1.2 — файл rag-pipeline.local.conf

```
# Локальная конфигурация clean pipeline. Этот файл задаёт пути для оркестратора.
# Задаётся корневой каталог проекта finalversion.
FINAL_ROOT="D:/Works/учёба/вкп/finalversion"
# Исходный Pascal/Delphi-проект.
# Задаётся путь к каталогу с исходными файлами обрабатываемого проекта.
PROJECT_ROOT="D:/Works/учёба/вкп/finalversion/data/input_project/src"
# Wiki generator пути.
# Задаётся каталог, в котором расположен код генератора Markdown-wiki.
WIKI_CODE_DIR="${FINAL_ROOT}/code/wiki"
# Задаётся путь к основному скрипту генерации Markdown-wiki.
WIKI_SCRIPT="${WIKI_CODE_DIR}/ai-descr-ai-v3-config-rag-md-v3.sh"
# Стабильный путь текущей wiki.
# Задаётся выходной каталог, в который будет сформирована актуальная Markdown-wiki.
WIKI_DIR="${FINAL_ROOT}/data/wiki_out/current"
# AI config для wiki generator.
# Задаётся путь к конфигурационному файлу AI-обогащения описаний.
AI_CONFIG="${FINAL_ROOT}/.config/ai-descr.local.conf"
# поле путей инструментов.
# Задаётся путь к исполняемому файлу PasDoc.
PASDOC_PROG="${FINAL_ROOT}/tools/pasdoc/bin/pasdoc.exe"
```

```

# Задаётся путь к исполняемому файлу xmlstarlet.
XMLSTARLET_BIN="${FINAL_ROOT}/tools/xmlstarlet/xmlstarlet-1.6.1/xml.exe"
# Задаётся путь к исполняемому файлу jq.
JQ_BIN="${FINAL_ROOT}/tools/bin/jq.exe"
# Задаётся путь к исполняемому файлу uv.
UV_EXE="C:/Users/Danii/AppData/Roaming/Python/Python313/Scripts/uv.exe"
# markdown-rag-mcp backend.
# Задаётся каталог, в котором расположен markdown-rag-mcp.
MARKDOWN_RAG_DIR="${FINAL_ROOT}/tools/markdown-rag-mcp"
# Задаётся путь к Docker Compose-файлу Milvus.
MILVUS_COMPOSE_FILE="${MARKDOWN_RAG_DIR}/docker/docker-compose.yml"
# MCP wrapper.
# Задаётся каталог wrapper-а, предоставляющего RAG-поиск через MCP.
MARKDOWN_RAG_WRAPPER_DIR="${FINAL_ROOT}/code/mcp/markdown-rag-mcp-wrapper"
# Задаётся путь к основному файлу MCP-сервера wrapper-а.
MARKDOWN_RAG_WRAPPER_SERVER="${MARKDOWN_RAG_WRAPPER_DIR}/server.py"
# Указывается, что далее задаются параметры pasls и OpenCode.
# pasls / OpenCode.
# Задаётся путь к исполняемому файлу Pascal Language Server.
PASLS_EXE="${FINAL_ROOT}/tools/pasls/pasls.exe"
# Задаётся путь к mcp-language-server, используемому для подключения pasls через MCP.
MCP_LANGUAGE_SERVER_EXE="${FINAL_ROOT}/tools/bin/mcp-language-server.exe"
# Logs.
# Задаётся каталог для сохранения журналов выполнения.
LOG_DIR="${FINAL_ROOT}/data/logs"
# Задаётся путь к файлу-маркеру, в котором хранится путь к последней сформированной wiki.
LAST_WIKI_DIR_FILE="${FINAL_ROOT}/data/wiki_out/last_wiki_dir.txt"
# Cache.
# Задаётся каталог для хранения кэшированных AI-описаний.
CACHE_DIR="${FINAL_ROOT}/data/cache/ai-descriptions"
# Указывается, что далее задаются параметры Docker и Milvus.
# Docker / Milvus.
# Включается автоматический запуск Milvus при выполнении pipeline.
AUTO_START_MILVUS="true"
# Задаётся имя контейнера Milvus.
MILVUS_CONTAINER_NAME="milvus-standalone"
# Задаётся максимальное время ожидания готовности Milvus в секундах.
MILVUS_HEALTH_TIMEOUT_SECONDS="120"
# Указывается, что далее задаются параметры выходного каталога wiki.
# Wiki output.
# Включается очистка каталога wiki перед новой генерацией.
CLEAN_WIKI_DIR="true"
# Указывается, что далее задаются параметры индексации RAG.
# RAG indexing.
# Отключается рекурсивная индексация вложенных каталогов wiki.
RAG_RECURSIVE="false"
# Включается принудительная переиндексация wiki.
RAG_FORCE_REINDEX="true"
# Указывается, что далее задаются параметры контрольного поискового запроса.
# RAG search sanity check.
# Задаётся тестовый запрос для проверки работоспособности RAG-поиска.
RAG_TEST_QUERY="FindBucket"

```

Задаётся количество результатов, возвращаемых при контрольном поиске.
RAG_SEARCH_LIMIT="1"

Задаётся минимальный порог близости для результатов RAG-поиска.
RAG_SEARCH_THRESHOLD="0.1"

ПРИЛОЖЕНИЕ 2. Листинг build-wiki-rag.sh

В приложении приведён скрипт build-wiki-rag.sh, выполняющий полный цикл подготовки Markdown-wiki и RAG-индекса: загрузку конфигурации, проверку окружения, запуск Milvus, генерацию wiki, индексацию и контрольный поисковый запрос.

```
# Указывается используемый командный интерпретатор.
#!/usr/bin/env sh
# Включается строгий режим выполнения: завершение при ошибке и при обращении к незадаанным
# переменным.
set -eu
# Определяется каталог, в котором расположен текущий скрипт.
SCRIPT_DIR=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
# Определяется корневой каталог проекта относительно расположения скрипта.
FINAL_ROOT_DEFAULT=$(CDPATH= cd -- "${SCRIPT_DIR}/.." && pwd)
# Определяется путь к конфигурационному файлу pipeline.
# Если переменная RAG_PIPELINE_CONFIG не задана, используется локальный конфиг по умолчанию.
CONFIG_FILE="${RAG_PIPELINE_CONFIG:-${FINAL_ROOT_DEFAULT}/.config/rag-pipeline.local.conf}"
# Объявляется функция аварийного завершения с выводом текста ошибки.
error_exit() {
    # Выводится сообщение об ошибке в stderr.
    echo "Ошибка: $1" >&2
    # Выполнение скрипта завершается с кодом ошибки.
    exit 1
}
# Объявляется функция вывода информационных сообщений pipeline.
info() {
    # Выводится сообщение с префиксом pipeline.
    echo "[pipeline] $1"
}
# Объявляется функция проверки существования файла.
require_file() {
    # Если файл не найден, выполнение завершается с ошибкой.
    [ -f "$1" ] || error_exit "$2: $1"
}
# Объявляется функция проверки существования каталога.
require_dir() {
    # Если каталог не найден, выполнение завершается с ошибкой.
    [ -d "$1" ] || error_exit "$2: $1"
}
# Объявляется функция проверки доступности команды.
require_cmd() {
    # Если команда не найдена в PATH, выполнение завершается с ошибкой.
    command -v "$1" >/dev/null 2>&1 || error_exit "команда не найдена: $1"
}
# Объявляется функция преобразования пути в Windows-формат.
to_win_path() {
    # Проверяется наличие утилиты cygpath.
    if command -v cygpath >/dev/null 2>&1; then
```

```

    # Если sygpath доступен, путь преобразуется в абсолютный Windows-формат.
    sygpath -am "$1"
# Обработывается случай отсутствия sygpath.
else
    # Если sygpath недоступен, путь возвращается без изменения.
    printf '%s\n' "$1"
# Завершается условная конструкция.
fi
}
# Объявляется функция загрузки и проверки конфигурации.
load_config() {
    # Проверяется наличие конфигурационного файла.
    [ -f "$CONFIG_FILE" ] || error_exit "конфиг не найден: $CONFIG_FILE"
    # shellcheck disable=SC1090
    # Подключается конфигурационный файл pipeline.
    . "$CONFIG_FILE"
    # Проверяется, что задан корневой каталог проекта.
    : "${FINAL_ROOT:?FINAL_ROOT не задан}"
    # Проверяется, что задан путь к исходному Pascal/Delphi-проекту.
    : "${PROJECT_ROOT:?PROJECT_ROOT не задан}"
    # Проверяется, что задан каталог генератора wiki.
    : "${WIKI_CODE_DIR:?WIKI_CODE_DIR не задан}"
    # Проверяется, что задан путь к скрипту генерации wiki.
    : "${WIKI_SCRIPT:?WIKI_SCRIPT не задан}"
    # Проверяется, что задан выходной каталог wiki.
    : "${WIKI_DIR:?WIKI_DIR не задан}"
    # Проверяется, что задан путь к конфигурации AI-обогащения.
    : "${AI_CONFIG:?AI_CONFIG не задан}"
    # Проверяется, что задан путь к PasDoc.
    : "${PASDOC_PROG:?PASDOC_PROG не задан}"
    # Проверяется, что задан путь к xmlstarlet.
    : "${XMLSTARLET_BIN:?XMLSTARLET_BIN не задан}"
    # Проверяется, что задан путь к jq.
    : "${JQ_BIN:?JQ_BIN не задан}"
    # Проверяется, что задан каталог markdown-rag-мсп.
    : "${MARKDOWN_RAG_DIR:?MARKDOWN_RAG_DIR не задан}"
    # Проверяется, что задан Docker Compose-файл Milvus.
    : "${MILVUS_COMPOSE_FILE:?MILVUS_COMPOSE_FILE не задан}"
    # Проверяется, что задан путь к uv.
    : "${UV_EXE:?UV_EXE не задан}"
    # Проверяется, что задан каталог логов.
    : "${LOG_DIR:?LOG_DIR не задан}"
    # Проверяется, что задан путь к файлу-маркеру последней wiki.
    : "${LAST_WIKI_DIR_FILE:?LAST_WIKI_DIR_FILE не задан}"
    # Проверяется, что задан каталог кэша.
    : "${CACHE_DIR:?CACHE_DIR не задан}"
    # Проверяется, что задан каталог MCP-wrapper-a.
    : "${MARKDOWN_RAG_WRAPPER_DIR:?MARKDOWN_RAG_WRAPPER_DIR не задан}"
    # Проверяется, что задан путь к серверному файлу MCP-wrapper-a.
    : "${MARKDOWN_RAG_WRAPPER_SERVER:?MARKDOWN_RAG_WRAPPER_SERVER не задан}"
}
# Объявляется функция запуска Milvus при необходимости.

```

```

start_milvus_if_needed() {
    # Проверяется, включён ли автоматический запуск Milvus.
    if[ "${AUTO_START_MILVUS:-true}" != "true" ]; then
        # Выводится сообщение о пропуске запуска Docker Compose.
        info "AUTO_START_MILVUS=false, пропускаю запуск Docker Compose"
        # Функция завершается без ошибки.
        return 0
    # Завершается условная конструкция.
    fi
    # Проверяется доступность команды docker.
    require_cmd docker
    # Проверяется наличие Docker Compose-файла Milvus.
    require_file "$MILVUS_COMPOSE_FILE" "docker-compose.yml для Milvus не найден"
    # Выводится сообщение о запуске или проверке Milvus.
    info "Запускаю/проверяю Milvus через Docker Compose"
    # Запускаются контейнеры Milvus через Docker Compose.
    docker compose -f "$MILVUS_COMPOSE_FILE" up -d
    # Определяется имя контейнера Milvus.
    container_name="${MILVUS_CONTAINER_NAME:-milvus-standalone}"
    # Определяется максимальное время ожидания готовности Milvus.
    timeout_seconds="${MILVUS_HEALTH_TIMEOUT_SECONDS:-120}"
    # Инициализируется счётчик прошедшего времени.
    elapsed=0
    # Запускается цикл ожидания готовности контейнера.
    while [ "$elapsed" -lt "$timeout_seconds" ]; do
        # Получается статус контейнера: health status или общий status.
        status=$(docker inspect --format '{{if .State.Health}}{{.State.Health.Status}}{{else}}{{.State.Status}}{{end}}'
"$container_name" 2>/dev/null || true)
        # Проверяется, перешёл ли контейнер в состояние healthy или running.
        if[ "$status" = "healthy" ] || [ "$status" = "running" ]; then
            # Выводится сообщение об успешной готовности Milvus.
            info "Milvus готов: ${container_name} (${status})"
            # Функция завершается без ошибки.
            return 0
        # Завершается условная конструкция.
        fi
        # Выполняется пауза перед следующей проверкой.
        sleep 2
        # Увеличивается счётчик времени ожидания.
        elapsed=$((elapsed + 2))
    # Завершается цикл ожидания.
    done
    # Выводится состояние Docker Compose-сервисов при неуспешном запуске.
    docker compose -f "$MILVUS_COMPOSE_FILE" ps || true
    # Выполнение завершается ошибкой, если Milvus не стал доступен за отведённое время.
    error_exit "Milvus не стал healthy за ${timeout_seconds} секунд"
}
# Объявляется функция подготовки выходных каталогов.
prepare_output_dirs() {
    # Создаётся каталог логов, если он ещё не существует.
    mkdir -p "$LOG_DIR"
    # Создаётся каталог кэша, если он ещё не существует.
    mkdir -p "$CACHE_DIR"
}

```

```

# Создаётся каталог для файла-маркера последней wiki.
mkdir -p "$(dirname "$LAST_WIKI_DIR_FILE")"
# Проверяется, включена ли очистка выходного каталога wiki.
if[ "${CLEAN_WIKI_DIR:-true}" = "true" ]; then
    # Выводится сообщение об очистке каталога wiki.
    info "Очищаю WIKI_DIR: $WIKI_DIR"
    # Удаляется текущий каталог wiki.
    rm -rf "$WIKI_DIR"
# Завершается условная конструкция.
fi
# Создаётся выходной каталог wiki.
mkdir -p "$WIKI_DIR"
}
# Объявляется функция генерации Markdown-wiki.
run_wiki_generation() {
    # Проверяется наличие исходного Pascal/Delphi-проекта.
    require_dir "$PROJECT_ROOT" "PROJECT_ROOT не найден"
    # Проверяется наличие каталога генератора wiki.
    require_dir "$WIKI_CODE_DIR" "WIKI_CODE_DIR не найден"
    # Проверяется наличие скрипта генерации wiki.
    require_file "$WIKI_SCRIPT" "wiki script не найден"
    # Проверяется наличие конфигурации AI-обогащения.
    require_file "$AI_CONFIG" "AI_CONFIG не найден"
    # Проверяется наличие исполняемого файла PasDoc.
    require_file "$PASDOC_PROG" "PASDOC_PROG не найден"
    # Проверяется наличие исполняемого файла xmlstarlet.
    require_file "$XMLSTARLET_BIN" "XMLSTARLET_BIN не найден"
    # Проверяется наличие исполняемого файла jq.
    require_file "$JQ_BIN" "JQ_BIN не найден"
    # Проверяется наличие каталога MCP-wrapper-a.
    require_dir "$MARKDOWN_RAG_WRAPPER_DIR" "MARKDOWN_RAG_WRAPPER_DIR не найден"
    # Проверяется наличие серверного файла MCP-wrapper-a.
    require_file "$MARKDOWN_RAG_WRAPPER_SERVER" "MARKDOWN_RAG_WRAPPER_SERVER не найден"
    # Формируется timestamp для имени лог-файла.
    now=$(date +%Y-%m-%d_%H-%M-%S)
    # Формируется путь к логу генерации wiki.
    WIKI_LOG="$LOG_DIR/wiki-build-${now}.log"
    # Выводится сообщение о запуске генерации wiki.
    info "Генерирую Markdown wiki"
    # Выводится путь к выходному каталогу wiki.
    info "WIKI_DIR: $WIKI_DIR"
    # Выводится путь к файлу лога генерации wiki.
    info "Лог wiki: $WIKI_LOG"
    # Запускается генерация wiki в отдельной subshell-среде.
    (
        # Выполняется переход в каталог генератора wiki.
        cd "$WIKI_CODE_DIR"
        # В PATH добавляются каталог jq и каталог генератора, после чего передаются переменные окружения и запускается скрипт генерации.
        PATH="$(dirname "$JQ_BIN");$WIKI_CODE_DIR:$PATH" \
        AI_CONFIG="$AI_CONFIG" \

```

```

PROJECT_ROOT="$PROJECT_ROOT" \
WIKI_DIR="$WIKI_DIR" \
PASDOC_PROG="$PASDOC_PROG" \
XMLSTARLET_BIN="$XMLSTARLET_BIN" \
JQ_BIN="$JQ_BIN" \
CACHE_DIR="$CACHE_DIR" \
sh "$WIKI_SCRIPT"
# stdout и stderr записываются одновременно в консоль и лог-файл.
) 2>&1 | tee "$WIKI_LOG"
# Проверяется, что каталог wiki был создан.
[ -d "$WIKI_DIR" ] || error_exit "созданная wiki-директория не найдена: $WIKI_DIR"
# Проверяется, что в wiki создана начальная страница Home.md.
[ -f "$WIKI_DIR/Home.md" ] || error_exit "Home.md не найден в wiki-директории: $WIKI_DIR"
# Путь к текущей wiki записывается в файл-маркер.
printf '%s\n' "$WIKI_DIR" > "$LAST_WIKI_DIR_FILE"
# Выводится сообщение об успешном создании wiki.
info "Wiki создана: $WIKI_DIR"
}
# Объявляется функция индексации wiki.
index_wiki() {
# Проверяется наличие каталога markdown-rag-mcp.
require_dir "$MARKDOWN_RAG_DIR" "MARKDOWN_RAG_DIR не найден"
# Проверяется наличие исполняемого файла uv.
require_file "$UV_EXE" "uv.exe не найден"
# Проверяется наличие каталога wiki.
require_dir "$WIKI_DIR" "WIKI_DIR не найден"
# Путь к wiki приводится к Windows-формату при необходимости.
WIKI_DIR_WIN=$(to_win_path "$WIKI_DIR")
# По умолчанию задаётся режим нерекурсивной индексации.
recursive_flag="--no-recursive"
# Если включена рекурсивная индексация, задаётся флаг --recursive.
[ "${RAG_RECURSIVE:-false}" = "true" ] && recursive_flag="--recursive"
# По умолчанию флаг принудительной переиндексации не задаётся.
force_flag=""
# Если принудительная переиндексация включена, задаётся флаг --force.
[ "${RAG_FORCE_REINDEX:-true}" = "true" ] && force_flag="--force"
# Формируется timestamp для имени лог-файла.
now=$(date +%Y-%m-%d_%H-%M-%S)
# Формируется путь к логу индексации.
INDEX_LOG="${LOG_DIR}/rag-index-${now}.log"
# Выводится сообщение о запуске индексации wiki.
info "Индексирую wiki в markdown-rag-mcp"
# Выводится путь к логу индексации.
info "Лог index: $INDEX_LOG"
(
# Выполняется переход в каталог markdown-rag-mcp.
cd "$MARKDOWN_RAG_DIR"
# Настраивается UTF-8 окружение, режим копирования uv и запускается команда индексирования.
PYTHONIOENCODING="utf-8" \
PYTHONUTF8="1" \
UV_LINK_MODE="copy" \
"$UV_EXE" run markdown-rag-mcp index \
--index-dir "$WIKI_DIR_WIN" \

```



```

$recursive_flag \
$force_flag
# stdout и stderr записываются одновременно в консоль и лог-файл.
) 2>&1 | tee "$INDEX_LOG"
# Проверяется текстовый вариант сообщения об успешной индексации без failed files.
if grep -q "Directory indexing complete: .* success, 0 failed" "$INDEX_LOG"; then
    # Выводится сообщение об успешной индексации.
    info "Индексация завершена без ошибок"
# Проверяется табличный вариант сообщения об отсутствии failed files.
elif grep -q "Files Failed.* / [[:space:]]*0[[:space:]]*" / " "$INDEX_LOG"; then
    # Выводится сообщение об успешной индексации.
    info "Индексация завершена без ошибок"
# Проверяется JSON-подобный вариант сообщения об отсутствии failed files.
elif grep -q '"failed_files"[[:space:]]*:[[space:]]*0' "$INDEX_LOG"; then
    # Выводится сообщение об успешной индексации.
    info "Индексация завершена без ошибок"
# Обрабатывается случай, когда успешная индексация не подтверждена.
else
    # Выполнение завершается ошибкой с указанием лога индексации.
    error_exit "индексация не подтвердила 0 failed files. Лог: $INDEX_LOG"
# Завершается условная конструкция.
fi
}
# Объявляется функция контрольного RAG-поиска.
check_rag_search() {
    # Определяется тестовый поисковый запрос.
    query="{RAG_TEST_QUERY:-FindBucket}"
    # Определяется порог близости для поиска.
    threshold="{RAG_SEARCH_THRESHOLD:-0.1}"
    # Определяется количество возвращаемых результатов.
    limit="{RAG_SEARCH_LIMIT:-1}"
    # Формируется timestamp для имени лог-файла.
    now=$(date +%Y-%m-%d_%H-%M-%S)
    # Формируется путь к логу контрольного поиска.
    SEARCH_LOG="{LOG_DIR}/rag-search-check-${now}.log"
    # Выводится сообщение о запуске контрольного поиска.
    info "Контрольный RAG search: ${query}"
    # Запускается контрольный поиск в отдельной subshell-среде.
    (
        # Выполняется переход в каталог markdown-rag-mcp.
        cd "$MARKDOWN_RAG_DIR"
        # Настраивается UTF-8 окружение, режим копирования ив и запускается команда поиска.
        PYTHONIOENCODING="utf-8" \
        PYTHONUTF8="1" \
        UV_LINK_MODE="copy" \
        "$UV_EXE" run markdown-rag-mcp search "$query" \
        --limit "$limit" \
        --threshold "$threshold" \
        --include-metadata \
        --format json
    # stdout и stderr записываются одновременно в консоль и лог-файл.
    ) 2>&1 | tee "$SEARCH_LOG"
    # Проверяется наличие поля results в JSON-ответе.

```

```

grep -q ""results"" "$SEARCH_LOG" || error_exit "контрольный search не вернул JSON results. Лог:
$SEARCH_LOG"
# Проверяется, не оказался ли список results пустым.
if grep -q ""results"[[:space:]]*:[[:space:]]*\[" "$SEARCH_LOG"; then
    # Выполнение завершается ошибкой при пустом результате поиска.
    error_exit "контрольный search вернул пустой results. Лог: $SEARCH_LOG"
# Завершается условная конструкция.
fi
# Выводится сообщение об успешном завершении контрольного поиска.
info "Контрольный search завершён. Лог: $SEARCH_LOG"
}
# Объявляется основная функция скрипта.
main() {
    # Загружается и проверяется конфигурация pipeline.
    load_config
    # Подготавливаются выходные каталоги.
    prepare_output_dirs
    # Выводится путь к используемому конфигурационному файлу.
    info "CONFIG_FILE: $CONFIG_FILE"
    # Выводится корневой каталог проекта.
    info "FINAL_ROOT: $FINAL_ROOT"
    # Выводится путь к исходному Pascal/Delphi-проекту.
    info "PROJECT_ROOT: $PROJECT_ROOT"
    # Выводится путь к выходной wiki.
    info "WIKI_DIR: $WIKI_DIR"
    # Запускается или проверяется Milvus.
    start_milvus_if_needed
    # Запускается генерация Markdown-wiki.
    run_wiki_generation
    # Запускается индексация сформированной wiki.
    index_wiki
    # Выполняется контрольный поисковый запрос.
    check_rag_search
    # Выводится пустая строка для отделения итогового сообщения.
    echo ""
    # Выводится сообщение об успешном завершении pipeline.
    echo "Готово."
    # Выводится путь к сформированной wiki.
    echo "Wiki: $WIKI_DIR"
    # Выводится путь к файлу-маркеру последней wiki.
    echo "Last wiki marker: $LAST_WIKI_DIR_FILE"
    # Выводится путь к RAG-backend-у.
    echo "RAG backend: $MARKDOWN_RAG_DIR"
    # Выводится путь к OpenCode MCP-wrapper-у.
    echo "OpenCode MCP wrapper: $MARKDOWN_RAG_WRAPPER_DIR"
}
# Запускается основная функция с передачей всех аргументов командной строки.
main "$@"

```

ПРИЛОЖЕНИЕ 3. Листинг ai-descr-ai-v3-config-rag-md-v3.sh

Здесь приводится исходный код создающего wiki скрипта.

```
# Указывается используемый командный интерпретатор.
#!/usr/bin/env sh
# Локальная генерация wiki из Pascal/Delphi файлов через PasDoc SimpleXML.
# Предполагается запуск в Git Bash / MSYS2.
# Включается строгий режим обращения к переменным.
set -u
# КОНФИГУРАЦИЯ
# По умолчанию скрипт читает локальный конфиг из текущей папки запуска.
# AI_CONFIG="/path/to/ai-descr.local.conf" ./ai-descr-local-ai-v3-config.sh
# Определяется каталог, в котором расположен текущий скрипт.
SCRIPT_DIR=$(CDPATH= cd -- "$(dirname -- "$0")" && pwd)
# Определяется путь к конфигурационному файлу AI-обогащения.
CONFIG_FILE="${AI_CONFIG:-${SCRIPT_DIR}/ai-descr.local.conf}"
# Фиксируется признак того, был ли загружен конфигурационный файл.
CONFIG_LOADED="false"
# Значения, переданные извне оркестратором, должны иметь приоритет над AI_CONFIG.
# AI_CONFIG может содержать старые пути, поэтому сохраняем внешние значения до source.
# Сохраняется внешний путь к PasDoc, если он был передан.
ENV_PASDOC_PROG="${PASDOC_PROG:-}"
# Сохраняется внешний путь к xmlstarlet, если он был передан.
ENV_XMLSTARLET_BIN="${XMLSTARLET_BIN:-}"
# Сохраняется внешний путь к jq, если он был передан.
ENV_JQ_BIN="${JQ_BIN:-}"
# Сохраняется внешний путь к исходному проекту, если он был передан.
ENV_PROJECT_ROOT="${PROJECT_ROOT:-}"
# Сохраняется внешний путь к выходной wiki, если он был передан.
ENV_WIKI_DIR="${WIKI_DIR:-}"
# Сохраняется внешний путь к каталогу кэша, если он был передан.
ENV_CACHE_DIR="${CACHE_DIR:-}"
# Проверяется наличие конфигурационного файла.
if [ -f "$CONFIG_FILE" ]; then
    # Подключается конфигурационный файл.
    . "$CONFIG_FILE"
    # Отмечается, что конфигурация была успешно загружена.
    CONFIG_LOADED="true"
# Проверяется случай, когда AI_CONFIG был указан явно, но файл не найден.
elif [ -n "${AI_CONFIG:-}" ]; then
    # Выводится сообщение об ошибке отсутствующего конфигурационного файла.
    echo "Ошибка: AI_CONFIG указан, но файл не найден: $CONFIG_FILE" >&2
    # Скрипт завершается с ошибкой.
    exit 1
fi
# Внешние значения имеют приоритет над AI_CONFIG.
# Если внешний путь к PasDoc задан, он подставляется после загрузки конфига.
[ -n "$ENV_PASDOC_PROG" ] && PASDOC_PROG="$ENV_PASDOC_PROG"
# Если внешний путь к xmlstarlet задан, он подставляется после загрузки конфига.
[ -n "$ENV_XMLSTARLET_BIN" ] && XMLSTARLET_BIN="$ENV_XMLSTARLET_BIN"
# Если внешний путь к jq задан, он подставляется после загрузки конфига.
[ -n "$ENV_JQ_BIN" ] && JQ_BIN="$ENV_JQ_BIN"
```

```

# Если внешний путь к исходному проекту задан, он подставляется после загрузки конфига.
[ -n "$ENV_PROJECT_ROOT" ] && PROJECT_ROOT="$ENV_PROJECT_ROOT"
# Если внешний путь к выходной wiki задан, он подставляется после загрузки конфига.
[ -n "$ENV_WIKI_DIR" ] && WIKI_DIR="$ENV_WIKI_DIR"
# Если внешний путь к каталогу кэша задан, он подставляется после загрузки конфига.
[ -n "$ENV_CACHE_DIR" ] && CACHE_DIR="$ENV_CACHE_DIR"
# НАСТРОЙКИ
# Задаётся путь к PasDoc по умолчанию, если он не был передан через окружение или конфиг.
PASDOC_PROG="{PASDOC_PROG:-D:/Works/учёба/BKP/code/wiki/pasdoc/bin/pasdoc.exe}"
# Задаётся путь к xmlstarlet по умолчанию, если он не был передан через окружение или конфиг.
XMLSTARLET_BIN="{XMLSTARLET_BIN:-D:/Works/учёба/BKP/code/wiki/xmlstarlet-1.6.1-win32/xmlstarlet-1.6.1/xml.exe}"
# Задаётся путь к jq, если он был передан через окружение или конфиг.
JQ_BIN="{JQ_BIN:-}"
# Задаётся путь к исходному проекту: сначала из аргумента, затем из переменной, затем значение по умолчанию.
PROJECT_ROOT="{1:-{PROJECT_ROOT:-D:/Works/учёба/BKP/GpDelphiUnits_mini/src}}"
# Формируется текущая дата и время для уникальных имён временных каталогов.
NOW=$(date +%Y-%m-%d_%H-%M-%S)
# Временные директории — в ASCII-пути, чтобы Windows exe не сталкивались с кириллицей.
# Создаётся корневой временный каталог обработки.
WORK_ROOT=$(mktemp -d /tmp/pasdoc_wiki_${NOW}_XXXXXX)
# Задаётся каталог для временных файлов PasDoc.
DOC_DIR="{WORK_ROOT}/docs_tmp"
# Задаётся staging-каталог для копий исходных файлов.
STAGE_DIR="{WORK_ROOT}/stage"
# Итоговая wiki — рядом со скриптом.
# Задаётся выходной каталог wiki.
WIKI_DIR="{WIKI_DIR:-{SCRIPT_DIR}/wiki_out_${NOW}}"
# Задаётся формат вывода PasDoc.
PASDOC_OUTPUT_FORMAT="simplexml"
# НАСТРОЙКИ ИИ-ОБОГАЩЕНИЯ
# AI_PROVIDER:
# openrouter — OpenRouter API: https://openrouter.ai/api/v1/chat/completions
# openai-compatible — любой OpenAI-compatible endpoint: Ollama/LM Studio/vLLM/etc.
# Задаётся AI-провайдер по умолчанию.
AI_PROVIDER="{AI_PROVIDER:-openrouter}"
# Задаётся URL AI API, если он передан через конфиг или окружение.
AI_URL="{AI_URL:-}"
# Задаётся модель AI-обогащения.
AI_MODEL="{AI_MODEL:-}"
# Задаётся температура генерации.
AI_TEMPERATURE="{AI_TEMPERATURE:-0.1}"
# Задаётся максимальное количество токенов в ответе модели.
AI_MAX_TOKENS="{AI_MAX_TOKENS:-1200}"
# По умолчанию обогащаем только крупные сущности. Методы можно включить отдельно,
# чтобы первый AI-прогон не превращался в десятки/сотни запросов.
# Включается AI-обогащение unit-страниц.
AI_ENRICH_UNITS="{AI_ENRICH_UNITS:-true}"
# Включается AI-обогащение страниц классов.
AI_ENRICH_CLASSES="{AI_ENRICH_CLASSES:-true}"
# Отключается AI-обогащение методов по умолчанию.
AI_ENRICH_METHODS="{AI_ENRICH_METHODS:-false}"

```

```

# Кэш зависит от источника, модели, provider-а, URL, параметров и версии промптов.
# Задаётся версия промптов для кэширования AI-описаний.
AI_PROMPT_VERSION="v2-short-descriptions"
# Задаётся версия схемы AI-кэша.
AI_CACHE_SCHEMA_VERSION="ai-v3"
# Задаётся каталог кэша AI-описаний.
CACHE_DIR="${CACHE_DIR:-${HOME}/.cache/pasdoc_ai_descriptions}"
# Безопасный разделитель полей вместо ' '.
# Создаётся служебный разделитель полей.
SEP=$(printf '\037')
# ПРОВЕРКИ
# Объявляется функция поиска jq.
resolve_jq_bin() {
    # Если JQ_BIN задан явно, используем его.
    if [ -n "${JQ_BIN:-}" ]; then
        # Проверяется наличие указанного jq-файла.
        if [ -x "$JQ_BIN" ] || [ -f "$JQ_BIN" ]; then
            # Функция завершается успешно.
            return 0
        fi
        # Выводится ошибка, если JQ_BIN указан, но файл не найден.
        echo "Ошибка: JQ_BIN указан, но файл не найден: $JQ_BIN" >&2
        # Скрипт завершается с ошибкой.
        exit 1
    fi
    # Частый случай для Git Bash/MSYS2: jq.exe лежит рядом со скриптом.
    # Проверяется наличие jq.exe рядом со скриптом.
    if [ -f "${SCRIPT_DIR}/jq.exe" ]; then
        # Используется jq.exe из каталога скрипта.
        JQ_BIN="${SCRIPT_DIR}/jq.exe"
        # Функция завершается успешно.
        return 0
    fi
    # Проверяется наличие jq рядом со скриптом.
    if [ -f "${SCRIPT_DIR}/jq" ]; then
        # Используется jq из каталога скрипта.
        JQ_BIN="${SCRIPT_DIR}/jq"
        # Функция завершается успешно.
        return 0
    fi
    # Проверяется наличие jq в PATH.
    if command -v jq >/dev/null 2>&1; then
        # Используется jq из PATH.
        JQ_BIN="jq"
        # Функция завершается успешно.
        return 0
    fi
    # Выводится ошибка, если jq не найден.
    echo "Ошибка: jq не найден. Положите jq.exe рядом со скриптом, добавьте jq в PATH или задайте JQ_BIN=/path/to/jq.exe" >&2
    # Скрипт завершается с ошибкой.
    exit 1
}

```

```

}
# Объявляется функция проверки зависимостей.
check_dependencies() {
    # Проверяется наличие PasDoc.
    [ -f "$PASDOC_PROG" ] || { echo "Ошибка: PasDoc не найден: $PASDOC_PROG"; exit 1; }
    # Проверяется наличие xmlstarlet.
    [ -f "$XMLSTARLET_BIN" ] || { echo "Ошибка: xmlstarlet не найден: $XMLSTARLET_BIN"; exit 1; }
    # Проверяется наличие исходного проекта.
    [ -d "$PROJECT_ROOT" ] || { echo "Ошибка: PROJECT_ROOT не найден: $PROJECT_ROOT"; exit 1; }
    # Проверяется наличие awk.
    command -v awk >/dev/null 2>&1 || { echo "Ошибка: awk не найден"; exit 1; }
    # Проверяется наличие curl.
    command -v curl >/dev/null 2>&1 || { echo "Ошибка: curl не найден, AI-версия не может работать без curl";
exit 1; }
    # Проверяется наличие cugpath.
    command -v cugpath >/dev/null 2>&1 || { echo "Ошибка: cugpath не найден"; exit 1; }
    # Проверяется наличие find.
    command -v find >/dev/null 2>&1 || { echo "Ошибка: find не найден"; exit 1; }
    # Проверяется наличие grep.
    command -v grep >/dev/null 2>&1 || { echo "Ошибка: grep не найден"; exit 1; }
    # Проверяется наличие md5sum.
    command -v md5sum >/dev/null 2>&1 || { echo "Ошибка: md5sum не найден"; exit 1; }
    # Выполняется поиск и проверка jq.
    resolve_jq_bin
    # Проверяется наличие mktmp.
    command -v mktmp >/dev/null 2>&1 || { echo "Ошибка: mktmp не найден"; exit 1; }
    # Проверяется наличие sed.
    command -v sed >/dev/null 2>&1 || { echo "Ошибка: sed не найден"; exit 1; }
}
# ВСПОМОГАТЕЛЬНОЕ
# Объявляется функция преобразования пути в Windows-формат.
to_win_path() {
    # Возвращается абсолютный Windows-путь.
    cugpath -am "$1"
}
# Объявляется функция вызова xmlstarlet с преобразованием пути к XML-файлу.
xmlstarlet_sel() {
    # Сохраняется путь к XML-файлу.
    xml_file="$1"
    # Удаляется первый аргумент, чтобы далее передать оставшиеся параметры в xmlstarlet.
    shift
    # Путь к XML-файлу преобразуется в Windows-формат.
    xml_file_win=$(to_win_path "$xml_file")
    # Выполняется команда xmlstarlet sel.
    "$XMLSTARLET_BIN" sel "$@" "$xml_file_win"
}
# Объявляется функция получения списка Pascal/Delphi-файлов.
get_pascal_file_list() {
    # Выполняется поиск файлов .pas и .pp внутри PROJECT_ROOT.
    find "$PROJECT_ROOT" -type f \( -iname "*.pas" -o -iname "*.pp" \) | sort
}
# Объявляется функция извлечения имени unit.
extract_unit_name() {

```

```

# Сохраняется путь к исходному файлу.
file_path="$1"
# Получается имя файла с расширением.
base_name=$(basename "$file_path")
# Удаляется расширение .pas.
base_name=${base_name%.pas}
# Удаляется расширение .pp.
base_name=${base_name%.pp}
# Из исходного файла извлекается объявление unit.
unit_name=$(grep -i "^[[:space:]]*unit[[:space:]]\+" "$file_path" \
  | head -1 \
  | sed -E 's/^[[:space:]]*[Uu][Nn][Ii][Tt][[:space:]]+//' \
  | sed 's/;.*//' \
  | xargs)
# Если объявление unit не найдено, используется имя файла.
[ -z "$unit_name" ] && unit_name="$base_name"
# Возвращается имя unit.
echo "$unit_name"
}

# Объявляется функция создания ASCII-имени staging-файла.
make_ascii_stage_file() {
# Сохраняется путь к исходному файлу.
src_path="$1"
# Получается имя исходного файла.
base_name=$(basename "$src_path")
# Формируется хэш исходного пути.
hash=$(printf '%s' "$src_path" | md5sum | cut -d' ' -f1)
# Возвращается путь к staging-файлу.
echo "${STAGE_DIR}/${hash}_${base_name}"
}

# Делает путь к исходнику переносимым для wiki.
# Например:
# D:/.../GpDelphiUnits_mini/src/GpStringHash.pas -> src/GpStringHash.pas
# Объявляется функция формирования отображаемого пути к исходнику.
make_source_path_display() {
# В пути файла обратные слэши заменяются на прямые.
file_path=$(printf '%s' "$1" | sed 's#\\#/#g')
# В пути PROJECT_ROOT обратные слэши заменяются на прямые.
root_path=$(printf '%s' "$PROJECT_ROOT" | sed 's#\\#/#g')
# Удаляется завершающий слэш из PROJECT_ROOT.
root_path=${root_path%/}
# Получается имя корневого каталога проекта.
root_base=$(basename "$root_path")
# Проверяется, находится ли файл внутри PROJECT_ROOT.
case "$file_path" in
"$root_path"/*)
# Получается относительный путь внутри PROJECT_ROOT.
rel_path=${file_path#"$root_path"/}
# Возвращается переносимый путь с именем корневого каталога.
printf '%s/%s\n' "$root_base" "$rel_path"
;;
*)
# Если файл почему-то вне PROJECT_ROOT, не тащим абсолютный путь в wiki.

```

```

        # Возвращается только имя файла.
        basename "$file_path"
    ;;
esac
}
# Объявляется функция формирования имени страницы класса.
class_page_name() {
    # Сохраняется имя unit.
    unit_name="$1"
    # Сохраняется имя класса.
    class_name="$2"
    # Возвращается имя страницы вида Unit.Class.
    echo "${unit_name}.${class_name}"
}
# Объявляется функция формирования wiki-ссылки на класс.
class_wikilink() {
    # Сохраняется имя unit.
    unit_name="$1"
    # Сохраняется имя класса.
    class_name="$2"
    # Формируется имя страницы класса.
    page_name=$(class_page_name "$unit_name" "$class_name")
    # Возвращается Obsidian-ссылка на страницу класса.
    echo "[[${page_name}|${class_name}]]"
}
# Объявляется функция объединения непустых строк в CSV.
join_nonempty_lines_csv() {
    # Непустые строки объединяются через запятую.
    awk '
        NF {
            if (out != "") out = out ", "
            out = out $0
        }
        END { print out }
    '
}
# Объявляется функция извлечения XML-значений в CSV.
xml_values_csv() {
    # Сохраняется путь к XML-файлу.
    xml_file="$1"
    # Сохраняется XPath для выбора узлов.
    xpath="$2"
    # Сохраняется выражение для извлечения значения.
    value_expr="$3"
    # Проверяется наличие XML-файла.
    [ -f "$xml_file" ] || error_exit "XML-файл не найден: $xml_file"
    # Создаётся временный файл для значений.
    values_file=$(mktemp "${WORK_ROOT}/xml_values_XXXXXX") || error_exit "не удалось создать временный файл xml_values"
    # Создаётся временный файл для ошибок.
    error_file=$(mktemp "${WORK_ROOT}/xml_values_err_XXXXXX") || error_exit "не удалось создать временный файл xml_values_err"

```



```

# Эти списки используются только для RAG-поисковых ключей.
# Отсутствие узлов здесь нормально: в unit может не быть types/variables/routines,
# а в class может не быть fields/properties/methods.
# Но настоящие ошибки xmlstarlet, например битый XPath или XML.
# Из XML извлекаются значения по заданному XPath.
xmlstarlet_sel "$xml_file" -t -m "$xpath" -v "$value_expr" -n > "$values_file" 2> "$error_file"
# Сохраняется код завершения xmlstarlet.
status=$?
# Проверяется, возникла ли ошибка при извлечении XML-значений.
if [ "$status" -ne 0 ] && [ -s "$error_file" ]; then
    # Выводится сообщение об ошибке XML-извлечения.
    echo "Ошибка: не удалось извлечь XML-значения: xpath=${xpath}, value=${value_expr}" >&2
    # Выводятся первые строки файла ошибок.
    sed -n '1,20p' "$error_file" >&2
    # Удаляются временные файлы.
    rm -f "$values_file" "$error_file"
    # Скрипт завершается с ошибкой.
    exit 1
fi
# Непустые значения объединяются в CSV-строку.
join_nonempty_lines_csv < "$values_file"
# Удаляются временные файлы.
rm -f "$values_file" "$error_file"
}
# Объявляется функция записи RAG-ключа при непустом значении.
write_rag_key_if_not_empty() {
    # Сохраняется путь к выходному Markdown-файлу.
    output_file="$1"
    # Сохраняется имя ключа.
    key="$2"
    # Сохраняется значение ключа.
    value="$3"
    # Если значение непустое, ключ записывается в Markdown-файл.
    [ -n "$value" ] && echo "- ${key}: ${value}" >> "$output_file"
}
# Объявляется функция добавления RAG-ключей unit.
add_unit_rag_search_keys() {
    # Сохраняется путь к Markdown-файлу unit.
    md_file="$1"
    # Сохраняется имя unit.
    unit_name="$2"
    # Сохраняется отображаемый путь к источнику.
    source_path="$3"
    # Сохраняется путь к XML-файлу.
    xml_file="$4"
    # Извлекается список классов unit.
    classes=$(xml_values_csv "$xml_file" "//structure[@type='class']" "normalize-space(@name)")
    # Извлекается список типов unit.
    types=$(xml_values_csv "$xml_file" "//type" "normalize-space(@name)")
    # Извлекается список переменных уровня unit.
    variables=$(xml_values_csv "$xml_file" "//variable[not(parent::structure)]" "normalize-space(@name)")
    # Извлекается список процедур и функций уровня unit.
    routines=$(xml_values_csv "$xml_file" "//routine[not(parent::structure)]" "normalize-space(@name)")

```

```

# В Markdown-файл записывается основной блок RAG-ключей unit.
cat >> "$md_file" <<EOF_MD
## RAG-поисковые ключи
- kind: unit
- id: ${unit_name}
- source_path: ${source_path}
EOF_MD
# При наличии записывается список классов.
write_rag_key_if_not_empty "$md_file" "classes" "$classes"
# При наличии записывается список типов.
write_rag_key_if_not_empty "$md_file" "types" "$types"
# При наличии записывается список переменных.
write_rag_key_if_not_empty "$md_file" "variables" "$variables"
# При наличии записывается список процедур и функций.
write_rag_key_if_not_empty "$md_file" "routines" "$routines"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$md_file"
}
# Объявляется функция добавления RAG-ключей класса.
add_class_rag_search_keys() {
# Сохраняется путь к Markdown-файлу класса.
class_md="$1"
# Сохраняется имя класса.
class_name="$2"
# Сохраняется имя unit.
unit_name="$3"
# Сохраняется отображаемый путь к исходнику.
source_path="$4"
# Сохраняется имя предка класса.
ancestor_name="$5"
# Сохраняется путь к XML-файлу.
xml_file="$6"
# Из XML извлекается тип структуры.
structure_type=$(xmlstarlet_sel "$xml_file" -t -v "normalize-space(//structure[@name='${class_name}']/@type)")
|| \
    error_exit "не удалось извлечь type для structure: ${unit_name}.${class_name}"
# Тип структуры нормализуется.
structure_type=$(normalize_inline "$structure_type")
# Проверяется, что тип структуры был получен.
[ -n "$structure_type" ] || error_exit "не удалось получить type для structure: ${unit_name}.${class_name}"
# Извлекается список методов класса.
methods=$(xml_values_csv "$xml_file" "//structure[@name='${class_name}']/routine" "normalize-space(@name)")
# Извлекается список полей класса.
fields=$(xml_values_csv "$xml_file" "//structure[@name='${class_name}']/variable" "normalize-space(@name)")
# Извлекается список свойств класса.
properties=$(xml_values_csv "$xml_file" "//structure[@name='${class_name}']/property" "normalize-space(@name)")
# В Markdown-файл записывается основной блок RAG-ключей класса.
cat >> "$class_md" <<EOF_MD
## RAG-поисковые ключи
- kind: class
- id: ${unit_name}.${class_name}
- unit: ${unit_name}

```

```

- structure_type: ${structure_type}
- source_path: ${source_path}
EOF_MD
# При наличии записывается имя предка класса.
write_rag_key_if_not_empty "$class_md" "ancestor" "$ancestor_name"
# При наличии записывается список методов.
write_rag_key_if_not_empty "$class_md" "methods" "$methods"
# При наличии записывается список полей.
write_rag_key_if_not_empty "$class_md" "fields" "$fields"
# При наличии записывается список свойств.
write_rag_key_if_not_empty "$class_md" "properties" "$properties"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$class_md"
}

# Добавляет к строкам два служебных поля:
# 1) порядковый номер элемента среди элементов с тем же именем;
# 2) общее количество элементов с тем же именем.
# Это нужно для простых ID вида Create#2 только при повторяющихся именах.
# Объявляется функция добавления счётчиков повторяющихся имён.
add_duplicate_counters() {
# Через awk к строкам добавляются порядковый номер и общее число повторов.
awk -v sep="$SEP" '
BEGIN { FS = sep; OFS = sep }
{
    line[NR] = $0
    name[NR] = $1
    count[$1]++
}
END {
    for (i = 1; i <= NR; i++) {
        current = name[i]
        seen[current]++
        print line[i], seen[current], count[current]
    }
}
'
}

# Объявляется функция очистки временного каталога.
cleanup() {
# Если временный каталог существует, он удаляется.
[ -d "$WORK_ROOT" ] && rm -rf "$WORK_ROOT"
}

# ОЧИСТКА ТЕКСТА ИЗ XML
# Объявляется функция декодирования XML/HTML-сущностей.
decode_entities() {
# Выполняется замена основных XML/HTML-сущностей на обычные символы.
printf '%s' "$1" | sed \
-e 's/</g' \
-e 's/>/g' \
-e 's/"/"/g' \
-e 's/'/'/g' \
-e 's/&/\&/g'
}

```

```

}
# Объявляется функция нормализации однострочного текста.
normalize_inline() {
    # Декодируются XML/HTML-сущности.
    decoded=$(decode_entities "$1")
    # Удаляются лишние пробелы, переводы carriage return и крайние пробелы.
    printf '%s' "$decoded" | sed -E \
        -e 's/\r//g' \
        -e 's/[[:space:]]+/ /g' \
        -e 's/^ //' \
        -e 's/ $/'
}

# Объявляется функция нормализации описания.
normalize_description() {
    # Декодируются XML/HTML-сущности.
    decoded=$(decode_entities "$1")
    # Удаляются XML/HTML-теги, лишние пробелы и крайние пробелы.
    printf '%s' "$decoded" | sed -E \
        -e 's/\r//g' \
        -e 's#</?[>]+># #g' \
        -e 's/[[:space:]]+/ /g' \
        -e 's/^ //' \
        -e 's/ $/'
}

# Объявляется функция возврата текста или заглушки.
text_or_placeholder() {
    # Сохраняется текстовое значение.
    value="$1"
    # Проверяется, пустое ли значение.
    if [ -z "$value" ]; then
        # Если значение пустое, возвращается заглушка.
        echo "Нет описания"
    # Обработывается непустое значение.
    else
        # Возвращается исходное значение.
        echo "$value"
    fi
}

# AI-ОБОГАЩЕНИЕ
# Объявляется функция аварийного завершения с ошибкой.
error_exit() {
    # Выводится сообщение об ошибке.
    echo "Ошибка: $1" >&2
    # Скрипт завершается с ошибкой.
    exit 1
}

# Объявляется функция проверки true/false-значения.
is_true_false() {
    # Проверяется, равно ли значение true или false.
    case "$1" in
        true|false) return 0 ;;
        *) return 1 ;;
    esac
}

```

```

}
# Объявляется функция проверки числового значения.
validate_number() {
    # Проверяется, что значение является целым или дробным числом.
    printf '%s' "$1" | grep -Eq '^[0-9]+([.][0-9]+)?$'
}
# Объявляется функция проверки положительного целого числа.
validate_positive_integer() {
    # Проверяется, что значение является положительным целым числом.
    printf '%s' "$1" | grep -Eq '^[1-9][0-9]*$'
}
# Объявляется функция определения URL AI-провайдера.
resolve_ai_url() {
    # Выбирается URL в зависимости от AI_PROVIDER.
    case "$AI_PROVIDER" in
        openrouter)
            # Для OpenRouter endpoint фиксирован. AI_URL из окружения/конфига здесь намеренно игнорируется,
            # чтобы не получить противоречивую конфигурацию вроде:
            # AI_PROVIDER=openrouter + AI_URL=http://127.0.0.1:11434/...
            # Возвращается фиксированный endpoint OpenRouter.
            echo "https://openrouter.ai/api/v1/chat/completions"
            ;;
        openai-compatible)
            # Проверяется наличие AI_URL для OpenAI-compatible backend-a.
            [ -n "$AI_URL" ] || error_exit "для AI_PROVIDER=openai-compatible нужно задать AI_URL"
            # Возвращается заданный пользователем AI_URL.
            echo "$AI_URL"
            ;;
        *)
            # Завершение при неизвестном AI-провайдере.
            error_exit "неизвестный AI_PROVIDER: $AI_PROVIDER. Допустимо: openrouter, openai-compatible"
            ;;
    esac
}
# Объявляется функция проверки AI-конфигурации.
validate_ai_config() {
    # Проверяется корректность флага AI_ENRICH_UNITS.
    is_true_false "$AI_ENRICH_UNITS" || error_exit "AI_ENRICH_UNITS должен быть true или false"
    # Проверяется корректность флага AI_ENRICH_CLASSES.
    is_true_false "$AI_ENRICH_CLASSES" || error_exit "AI_ENRICH_CLASSES должен быть true или false"
    # Проверяется корректность флага AI_ENRICH_METHODS.
    is_true_false "$AI_ENRICH_METHODS" || error_exit "AI_ENRICH_METHODS должен быть true или false"
    # Проверяется, что модель AI задана.
    [ -n "$AI_MODEL" ] || error_exit "AI_MODEL не задан. Укажи model id выбранной модели провайдера,
    например через export AI_MODEL='<model-id>'"
    # Проверяется корректность температуры генерации.
    validate_number "$AI_TEMPERATURE" || error_exit "AI_TEMPERATURE должен быть числом, например
    0.1"
    # Проверяется корректность максимального количества токенов.
    validate_positive_integer "$AI_MAX_TOKENS" || error_exit "AI_MAX_TOKENS должен быть
    положительным целым числом"
    # Определяется итоговый URL AI API.

```

```

AI_URL_RESOLVED=$(resolve_ai_url) || exit 1
# Проверяется конфигурация в зависимости от выбранного провайдера.
case "$AI_PROVIDER" in
    openrouter)
        # Для OpenRouter проверяется наличие API-ключа.
        [ -n "${OPENROUTER_API_KEY:-}" ] || error_exit "для AI_PROVIDER=openrouter нужно задать OPENROUTER_API_KEY"
        ;;
    openai-compatible)
        # Для OpenAI-compatible отдельная обязательная проверка ключа не требуется.
        :
        ;;
    esac
# Создаётся каталог кэша AI-описаний.
mkdir -p "$CACHE_DIR" || error_exit "не удалось создать CACHE_DIR: $CACHE_DIR"
}
# Объявляется функция хэширования строки.
hash_string() {
    # Возвращается md5-хэш строки.
    printf '%s' "$1" | md5sum | cut -d' ' -f1
}
# Объявляется функция получения хэша файла.
get_file_hash() {
    # Сохраняется путь к файлу.
    file_path="$1"
    # Проверяется наличие файла.
    if [ ! -f "$file_path" ]; then
        # Выводится ошибка при отсутствии файла.
        echo "Ошибка: не найден файл для AI/кэша: $file_path" >&2
        # Возвращается код ошибки.
        return 1
    fi
    # Возвращается md5-хэш файла.
    md5sum "$file_path" | cut -d' ' -f1
}
# Объявляется функция формирования безопасного компонента имени файла.
safe_cache_component() {
    # Все небезопасные символы заменяются на подчёркивания.
    printf '%s' "$1" | sed -E 's/[^A-Za-z0-9_-]+/_/g; s/_+/_/; s/_+$//'
}
# Объявляется функция формирования пути к файлу кэша.
get_cache_path() {
    # Сохраняется путь к исходному файлу.
    file_path="$1"
    # Сохраняется имя unit.
    unit_name="$2"
    # Сохраняется тип сущности.
    item_type="$3"

    # Сохраняется имя сущности.
    item_name="$4"
    # Сохраняется промпт.
    prompt="$5"

```

```

# Вычисляется хэш исходного файла.
file_hash=$(get_file_hash "$file_path") || return 1
# Вычисляется хэш промпта.
prompt_hash=$(hash_string "$prompt")
# Вычисляется хэш настроек AI-генерации.
settings_hash=$(hash_string
"${AI_CACHE_SCHEMA_VERSION}|${AI_PROMPT_VERSION}|${AI_PROVIDER}|${AI_URL_RESOLVED}
|${AI_MODEL}|${AI_TEMPERATURE}|${AI_MAX_TOKENS}|${prompt_hash}")
# Вычисляется итоговый хэш кэшируемого элемента.
item_hash=$(hash_string "${unit_name}|${item_type}|${item_name}|${file_hash}|${settings_hash}")
# Формируется читаемый безопасный префикс имени файла кэша.
safe_prefix=$(safe_cache_component "${unit_name}_${item_type}_${item_name}")
# Ограничиваем читаемый префикс, чтобы имена файлов кэша не становились слишком длинными.
# Префикс имени файла кэша ограничивается 80 символами.
safe_prefix=$(printf '%.80s' "$safe_prefix")
# Возвращается полный путь к файлу кэша.
echo "${CACHE_DIR}/${safe_prefix}_${item_hash}.cache"
}
# Объявляется функция чтения описания из кэша.
load_from_cache() {
# Сохраняется путь к файлу кэша.
cache_file="$1"
# Проверяется наличие кэш-файла.
if [ -f "$cache_file" ]; then
# Содержимое кэша выводится в stdout.
cat "$cache_file"
# Функция завершается успешно.
return 0
fi
# Возвращается код ошибки, если кэш отсутствует.
return 1
}
# Объявляется функция сохранения описания в кэш.
save_to_cache() {
# Сохраняется путь к файлу кэша.
cache_file="$1"
# Сохраняется текст описания.
description="$2"
# Описание записывается в файл кэша.
printf '%s\n' "$description" > "$cache_file" || error_exit "не удалось записать AI-кэш: $cache_file"
}
# Объявляется функция очистки ответа AI-модели.
strip_ai_response() {
# Удаляем возможный reasoning-блок у reasoning-моделей и грубые markdown-обёртки.
# Из ответа удаляются reasoning-блоки, markdown-обёртки и крайние пробелы.
sed -e '/<think>/,/</think>/d' \
-e 's/``[[:alnum:]]*/g' \
-e 's/``/g' \
-e 's/\r/g' \
-e 's/^[[:space:]]*/' \
-e 's/[[:space:]]*/'
}
# Объявляется функция формирования JSON-запроса к AI.

```

```

build_ai_request() {
    # Сохраняется путь к файлу запроса.
    request_file="$1"
    # Сохраняется промпт.
    prompt="$2"
    # Сохраняется путь к исходному файлу.
    file_path="$3"
    # Через jq формируется JSON-запрос к OpenAI-compatible API.
    "$JQ_BIN" -n \
        --arg model "$AI_MODEL" \
        --arg prompt "$prompt" \
        --rawfile source "$file_path" \
        --argjson temperature "$AI_TEMPERATURE" \
        --argjson max_tokens "$AI_MAX_TOKENS" \
        '{
            model: $model,
            messages: [
                {
                    role: "user",
                    content: ($prompt + "\n\nИсходный файл Delphi/Pascal:\n``pascal\n" + $source + "\n``")
                }
            ],
            temperature: $temperature,
            max_tokens: $max_tokens
        }' > "$request_file" || return 1
}

# Объявляется функция вызова AI-провайдера.
call_ai_provider() {
    # Сохраняется путь к файлу запроса.
    request_file="$1"
    # Сохраняется путь к файлу ответа.
    response_file="$2"
    # Выбирается способ HTTP-запроса в зависимости от AI_PROVIDER.
    case "$AI_PROVIDER" in
        openrouter)
            # Выполняется POST-запрос к OpenRouter.
            curl -sS -L -X POST "$AI_URL_RESOLVED" \
                -H "Content-Type: application/json" \
                -H "Authorization: Bearer ${OPENROUTER_API_KEY}" \
                -d "@$request_file" \
                -o "$response_file" \
                -w '%{http_code}'
            ;;
        openai-compatible)
            # Проверяется, задан ли API-ключ для OpenAI-compatible backend-a.
            if [ -n "${AI_API_KEY:-}" ]; then
                # Выполняется POST-запрос с авторизацией.
                curl -sS -L -X POST "$AI_URL_RESOLVED" \
                    -H "Content-Type: application/json" \
                    -H "Authorization: Bearer ${AI_API_KEY}" \
                    -d "@$request_file" \
                    -o "$response_file" \
                    -w '%{http_code}'
            fi
        esac
}

```



```

# Обработывается OpenAI-compatible backend без авторизации.
else
    # Выполняется POST-запрос без заголовка авторизации.
    curl -sS -L -X POST "$AI_URL_RESOLVED" \
        -H "Content-Type: application/json" \
        -d "@$request_file" \
        -o "$response_file" \
        -w '%{http_code}'

fi
;;
*)
    # Возвращается ошибка при неизвестном провайдере.
    return 1
;;
esac
}

# Объявляется функция извлечения текста ответа AI.
extract_ai_content() {
    # Сохраняется путь к файлу ответа.
    response_file="$1"

    # Через jq извлекается choices[0].message.content и очищается результат.
    "$JQ_BIN" -r '
        .choices[0].message.content as $content
        | if ($content | type) == "string" then $content
          elif ($content | type) == "array" then ($content | map(.text? // empty) | join(""))
          else empty end
        ' "$response_file" 2>/dev/null | strip_ai_response
}

# Объявляется функция генерации AI-описания.
generate_ai_description() {
    # Сохраняется промпт.
    prompt="$1"
    # Сохраняется путь к исходному файлу.
    file_path="$2"
    # Сохраняется имя unit.
    unit_name="$3"
    # Сохраняется тип сущности.
    item_type="$4"
    # Сохраняется имя сущности.
    item_name="$5"
    # Формируется путь к файлу кэша.
    cache_file=$(get_cache_path "$file_path" "$unit_name" "$item_type" "$item_name" "$prompt") || return 1
    # Проверяется наличие готового описания в кэше.
    if cached_desc=$(load_from_cache "$cache_file"); then
        # В stderr выводится сообщение об использовании кэша.
        echo "AI cache: ${unit_name}/${item_type}/${item_name}" >&2
        # Возвращается кэшированное описание.
        printf '%s\n' "$cached_desc"
        # Функция завершается успешно.
        return 0
    fi
    # В stderr выводится сообщение о запуске AI-генерации.

```

```

echo "AI generate: ${unit_name}/${item_type}/${item_name}" >&2
# Создаётся временный JSON-файл запроса.
request_file=$(mktemp "${WORK_ROOT}/ai_request_XXXXXX.json") || return 1
# Создаётся временный JSON-файл ответа.
response_file=$(mktemp "${WORK_ROOT}/ai_response_XXXXXX.json") || return 1
# Формируется JSON-запрос к AI.
build_ai_request "$request_file" "$prompt" "$file_path" || {
    echo "Ошибка: не удалось сформировать JSON-запрос к ИИ" >&2
    return 1
}
# Выполняется запрос к AI-провайдеру.
http_code=$(call_ai_provider "$request_file" "$response_file") || {
    echo "Ошибка: запрос к ИИ-провайдеру не выполнен" >&2
    return 1
}
# Проверяется HTTP-код ответа.
case "$http_code" in
    2*) ;;
    *)
        # Выводится ошибка при неуспешном HTTP-коде.
        echo "Ошибка: ИИ-провайдер вернул HTTP $http_code" >&2
        # Выводятся первые строки ответа провайдера.
        sed -n '1,20p' "$response_file" >&2
        # Возвращается ошибка.
        return 1
        ;;
esac
# Извлекается текст описания из ответа AI.
description=$(extract_ai_content "$response_file")
# Проверяется, что AI вернул непустое описание.
if [ -z "$description" ] || [ "$description" = "null" ]; then
    # Выводится ошибка пустого ответа.
    echo "Ошибка: ИИ-провайдер вернул пустое описание для ${unit_name}/${item_type}/${item_name}" >&2
    # Выводятся первые строки ответа провайдера.
    sed -n '1,20p' "$response_file" >&2
    # Возвращается ошибка.
    return 1
fi
# Описание сохраняется в кэш.
save_to_cache "$cache_file" "$description"
# Возвращается сгенерированное описание.
printf '%s\n' "$description"
}
# Объявляется функция проверки необходимости AI-описания.
needs_description() {
    # Сохраняется описание.
    desc="$1"

    # Сохраняется минимальная длина описания.
    min_length="${2:-20}"
    # Пустое описание требует обогащения.
    [ -z "$desc" ] && return 0
    # Заглушка требует обогащения.

```

```

[ "$desc" = "..." ] && return 0
# Текст "Нет описания" требует обогащения.
[ "$desc" = "Нет описания" ] && return 0
# Слишком короткое описание требует обогащения.
[ ${#desc} -lt "$min_length" ] && return 0
# В остальных случаях обогащение не требуется.
return 1
}

# Объявляется функция генерации описания unit.
generate_ai_unit_description() {
    # Сохраняется имя unit.
    unit_name="$1"
    # Сохраняется путь к исходному файлу.
    file_path="$2"
    # Формируется промпт для описания unit.
    prompt="Ты документируешь Pascal/Delphi-код для wiki разработчика. Дай краткое описание unit
${unit_name}. Опиши назначение модуля и основные сущности. Верни только готовое описание."
    # Запускается генерация AI-описания unit.
    generate_ai_description "$prompt" "$file_path" "$unit_name" "unit" "$unit_name"
}

# Объявляется функция генерации описания класса.
generate_ai_class_description() {
    # Сохраняется имя класса.
    class_name="$1"
    # Сохраняется имя unit.
    unit_name="$2"
    # Сохраняется путь к исходному файлу.
    file_path="$3"
    # Формируется промпт для описания класса.
    prompt="Ты документируешь Pascal/Delphi-код для wiki разработчика. Дай краткое описание класса
${class_name} из unit ${unit_name}. Опиши роль класса и его основную ответственность. Верни только
готовое описание."
    # Запускается генерация AI-описания класса.
    generate_ai_description "$prompt" "$file_path" "$unit_name" "class" "$class_name"
}

# Объявляется функция генерации описания метода.
generate_ai_method_description() {
    # Сохраняется имя метода.
    method_name="$1"
    # Сохраняется имя класса.
    class_name="$2"
    # Сохраняется имя unit.
    unit_name="$3"
    # Сохраняется объявление метода.
    declaration="$4"
    # Сохраняется путь к исходному файлу.
    file_path="$5"
    # Формируется ключ кэша метода с учётом объявления.
    method_cache_key="${class_name}.${method_name}:${declaration}"
    # Формируется промпт для описания метода.
    prompt="Ты документируешь Pascal/Delphi-код для wiki разработчика. Дай краткое описание метода
${class_name}.${method_name} из unit ${unit_name}. Опиши его роль и работу. Если метод перегружен,
ориентируйся именно на ${declaration} (объявление метода). Верни только готовое описание."
}

```

```

# Запускается генерация AI-описания метода.
generate_ai_description "$prompt" "$file_path" "$unit_name" "method" "$method_cache_key"
}
# Объявляется функция обогащения описания unit.
enrich_unit_description() {
    # Сохраняется имя unit.
    unit_name="$1"
    # Сохраняется исходное описание.
    desc="$2"
    # Сохраняется путь к исходному файлу.
    file_path="$3"
    # Проверяется, нужно ли генерировать AI-описание unit.
    if [ "$AI_ENRICH_UNITS" = true ] && needs_description "$desc" 20; then
        # Генерируется AI-описание unit.
        generate_ai_unit_description "$unit_name" "$file_path"
        # Возвращается код завершения генерации.
        return $?
    fi
    # Возвращается исходное описание или заглушка.
    text_or_placeholder "$desc"
}
# Объявляется функция обогащения описания класса.
enrich_class_description() {
    # Сохраняется имя класса.
    class_name="$1"
    # Сохраняется имя unit.
    unit_name="$2"
    # Сохраняется исходное описание.
    desc="$3"
    # Сохраняется путь к исходному файлу.
    file_path="$4"
    # Проверяется, нужно ли генерировать AI-описание класса.
    if [ "$AI_ENRICH_CLASSES" = true ] && needs_description "$desc" 20; then
        # Генерируется AI-описание класса.
        generate_ai_class_description "$class_name" "$unit_name" "$file_path"
        # Возвращается код завершения генерации.
        return $?
    fi
    # Возвращается исходное описание или заглушка.
    text_or_placeholder "$desc"
}
# Объявляется функция обогащения описания метода.
enrich_method_description() {
    # Сохраняется имя метода.
    method_name="$1"
    # Сохраняется имя класса.
    class_name="$2"
    # Сохраняется имя unit.
    unit_name="$3"
    # Сохраняется объявление метода.
    declaration="$4"
    # Сохраняется исходное описание.
    desc="$5"

```

```

# Сохраняется путь к исходному файлу.
file_path="$6"
# Проверяется, нужно ли генерировать AI-описание метода.
if [ "$AI_ENRICH_METHODS" = true ] && needs_description "$desc" 12; then
    # Генерируется AI-описание метода.
    generate_ai_method_description "$method_name" "$class_name" "$unit_name" "$declaration" "$file_path"
    # Возвращается код завершения генерации.
    return $?
fi
# Возвращается исходное описание или заглушка.
text_or_placeholder "$desc"
}
# GENERATE MARKDOWN
# Объявляется функция создания Markdown-страницы unit.
create_unit_markdown() {
    # Сохраняется путь к Markdown-файлу unit.
    md_file="$1"
    # Сохраняется имя unit.
    unit_name="$2"
    # Сохраняется описание unit.
    unit_desc="$3"
    # Сохраняется отображаемый путь к источнику.
    source_path="$4"
    # Сохраняется путь к XML-файлу.
    xml_file="$5"
    # В Markdown-файл записывается заголовок и путь к источнику.
    cat > "$md_file" <<EOF_MD
# Unit: ${unit_name}
**Исходник:** \`${source_path}\`
EOF_MD
    # Добавляются RAG-поисковые ключи unit.
    add_unit_rag_search_keys "$md_file" "$unit_name" "$source_path" "$xml_file"
    # В Markdown-файл записывается описание и раздел классов.
    cat >> "$md_file" <<EOF_MD
## Общее описание
${unit_desc}
---
## Структуры (классы)
EOF_MD
}
# Объявляется функция создания Markdown-страницы класса.
create_class_markdown() {
    # Сохраняется путь к Markdown-файлу класса.
    class_md="$1"
    # Сохраняется имя класса.
    class_name="$2"
    # Сохраняется описание класса.
    class_desc="$3"
    # Сохраняется имя предка класса.
    ancestor_name="$4"
    # Сохраняется имя unit.
    unit_name="$5"
    # Сохраняется путь к XML-файлу.

```

```

xml_file="$6"
# Сохраняется отображаемый путь к источнику.
source_path="$7"
# Сохраняется путь к staging-файлу для AI-обогащения.
ai_source_file="$8"
# Извлекается видимость класса.
class_vis=$(xmlstarlet_sel "$xml_file" -t -v "normalize-space(//structure[@name='${class_name}']/@visibility)"
2>/dev/null || true)
# Нормализуется значение видимости класса.
class_vis=$(normalize_inline "$class_vis")
# Если видимость не указана, используется public.
class_vis=${class_vis:-public}
# В Markdown-файл класса записывается заголовок и базовые сведения.
cat > "$class_md" <<EOF_MD
# Класс: ${class_name}
**Наследник:** ${ancestor_name:-нет}
**Видимость:** ${class_vis}
**Источник:** [${unit_name}]
**Исходник:** \ "${source_path}" \
EOF_MD
# Добавляются RAG-поисковые ключи класса.
add_class_rag_search_keys "$class_md" "$class_name" "$unit_name" "$source_path" "$ancestor_name"
"$xml_file"
# В Markdown-файл записываются описание и иерархия наследования.
cat >> "$class_md" <<EOF_MD
## Описание
${class_desc}
---
## Иерархия наследования
${class_name}${[ -n "${ancestor_name}" ] && echo " ← ${ancestor_name}"}
---
EOF_MD
# В Markdown-файл добавляется пустая строка.
echo "" >> "$class_md"
# В Markdown-файл добавляется раздел методов.
echo "## Методы" >> "$class_md"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$class_md"
# Создаётся временный файл для строк методов.
routine_rows=$(mktemp "${WORK_ROOT}/routine_rows_XXXXXX") || error_exit "не удалось создать
временный файл routine_rows"
# Создаётся временный файл для строк методов со счётчиками повторов.
routine_counted_rows=$(mktemp "${WORK_ROOT}/routine_counted_rows_XXXXXX") || error_exit "не
удалось создать временный файл routine_counted_rows"
# Из XML извлекаются сведения о методах класса.
xmlstarlet_sel "$xml_file" -t \
-m "//structure[@name='${class_name}']/routine" \
-v "normalize-space(@name)" -o "$SEP" \
-v "normalize-space(@declaration)" -o "$SEP" \
-v "normalize-space(@visibility)" -o "$SEP" \
-v "normalize-space(string(description/detailed))" -n 2>/dev/null > "$routine_rows"
# К строкам методов добавляются счётчики повторяющихся имён.
add_duplicate_counters < "$routine_rows" > "$routine_counted_rows"

```

```

# Запускается цикл записи методов класса в Markdown-файл.
while IFS="$SEP" read -r name decl vis desc item_index item_count; do
    # Нормализуется имя метода.
    name=$(normalize_inline "$name")
    # Нормализуется объявление метода.
    decl=$(normalize_inline "$decl")
    # Нормализуется видимость метода.
    vis=$(normalize_inline "$vis")
    # Нормализуется описание метода.
    desc=$(normalize_description "$desc")
    # При необходимости описание метода дополняется через AI.
    desc=$(enrich_method_description "$name" "$class_name" "$unit_name" "$decl" "$desc" "$ai_source_file") ||
exit 1
    # Пустые имена методов пропускаются.
    [ -z "$name" ] && continue
    # Формируется базовый ID метода.
    method_id="{unit_name}.${class_name}.${name}"
    # Для перегруженных методов к ID добавляется порядковый номер.
    [ "${item_count:-0}" -gt 1 ] && method_id="{method_id}#${item_index}"
    # В Markdown-файл записывается заголовок метода.
    echo "### Method: ${class_name}.${name}" >> "$class_md"
    # В Markdown-файл добавляется пустая строка.
    echo "" >> "$class_md"
    # В Markdown-файл записывается ID метода.
    echo "**ID:** \`${method_id}\`" >> "$class_md"
    # При наличии записывается объявление метода.
    [ -n "$decl" ] && echo "**Объявление:** \`${decl}\`" >> "$class_md"
    # При наличии записывается видимость метода.
    [ -n "$vis" ] && echo "**Видимость:** $vis" >> "$class_md"
    # В Markdown-файл добавляется пустая строка.
    echo "" >> "$class_md"
    # В Markdown-файл записывается описание метода.
    echo "$desc" >> "$class_md"
    # В Markdown-файл добавляется пустая строка.
    echo "" >> "$class_md"
    # Цикл читает строки методов из временного файла.
done < "$routine_counted_rows"
# Добавляется раздел полей класса.
generate_markdown_section "//structure[@name='${class_name}']/variable" "Поля" "$class_md" "$xml_file"
"declaration"
# Добавляется раздел свойств класса.
add_properties "$class_md" "$class_name" "$xml_file"
# В конец Markdown-файла класса записывается ссылка на исходный unit.
cat >> "$class_md" <<EOF_MD
---
**Источник:** [{unit_name}]
EOF_MD
}
# Объявляется функция добавления свойств класса.
add_properties() {
    # Сохраняется путь к Markdown-файлу класса.
    class_md="$1"
    # Сохраняется имя класса.

```

```

class_name="$2"
# Сохраняется путь к XML-файлу.
xml_file="$3"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$class_md"
# В Markdown-файл добавляется заголовок раздела свойств.
echo "## Свойства" >> "$class_md"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$class_md"
# Из XML извлекаются свойства класса.
xmlstarlet_sel "$xml_file" -t \
    -m "//structure[@name='${class_name}']/property" \
    -v "normalize-space(@name)" -o "$SEP" \
    -v "normalize-space(@type)" -o "$SEP" \
    -v "normalize-space(@reader)" -o "$SEP" \
    -v "normalize-space(@writer)" -o "$SEP" \
    -v "normalize-space(@visibility)" -o "$SEP" \
    -v "normalize-space(string(description/detailed))" -n 2>/dev/null | \
# К свойствам добавляются счётчики повторяющихся имён.
add_duplicate_counters | \
# Запускается цикл записи свойств в Markdown-файл.
while IFS="$SEP" read -r name type reader writer vis desc item_index item_count; do
    # Нормализуется имя свойства.
    name=$(normalize_inline "$name")
    # Нормализуется тип свойства.
    type=$(normalize_inline "$type")
    # Нормализуется reader свойства.
    reader=$(normalize_inline "$reader")
    # Нормализуется writer свойства.
    writer=$(normalize_inline "$writer")
    # Нормализуется видимость свойства.
    vis=$(normalize_inline "$vis")
    # Нормализуется описание свойства.
    desc=$(normalize_description "$desc")
    # Если описание пустое, подставляется заглушка.
    desc=$(text_or_placeholder "$desc")
    # Пустые имена свойств пропускаются.
    [ -z "$name" ] && continue
    # В Markdown-файл записывается заголовок свойства.
    echo "### $name" >> "$class_md"
    # В Markdown-файл добавляется пустая строка.
    echo "" >> "$class_md"
    # Для повторяющихся свойств записывается ID.
    [ "${item_count:-0}" -gt 1 ] && echo "**ID:** \`${name}${item_index}\`" >> "$class_md"
    # При наличии записывается тип свойства.
    [ -n "$type" ] && echo "**Тип:** $type" >> "$class_md"
    # При наличии записывается видимость свойства.
    [ -n "$vis" ] && echo "**Видимость:** $vis" >> "$class_md"
    # При наличии записывается reader свойства.
    [ -n "$reader" ] && echo "**Reader:** $reader" >> "$class_md"
    # При наличии записывается writer свойства.
    [ -n "$writer" ] && echo "**Writer:** $writer" >> "$class_md"
    # В Markdown-файл добавляется пустая строка.

```



```

echo "" >> "$class_md"
# В Markdown-файл записывается описание свойства.
echo "$desc" >> "$class_md"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$class_md"
# Завершается цикл обработки свойств.
done
}
# Объявляется функция генерации универсального Markdown-раздела.
generate_markdown_section() {
# Сохраняется XPath для выбора элементов.
xpath="$1"
# Сохраняется заголовок раздела.
title="$2"
# Сохраняется путь к выходному Markdown-файлу.
output_file="$3"
# Сохраняется путь к XML-файлу.
xml_file="$4"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$output_file"
# В Markdown-файл добавляется заголовок раздела.
echo "## $title" >> "$output_file"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$output_file"
# Из XML извлекаются элементы выбранного раздела.
xmlstarlet_sel "$xml_file" -t \
-m "$xpath" \
-v "normalize-space(@name)" -o "$SEP" \
-v "normalize-space(@declaration)" -o "$SEP" \
-v "normalize-space(@visibility)" -o "$SEP" \
-v "normalize-space(string(description/detailed))" -n 2>/dev/null | \
# К элементам добавляются счётчики повторяющихся имён.
add_duplicate_counters | \
# Запускается цикл записи элементов раздела.
while IFS="$SEP" read -r name decl vis desc item_index item_count; do
# Нормализуется имя элемента.
name=$(normalize_inline "$name")
# Нормализуется объявление элемента.
decl=$(normalize_inline "$decl")
# Нормализуется видимость элемента.
vis=$(normalize_inline "$vis")
# Нормализуется описание элемента.
desc=$(normalize_description "$desc")
# Если описание пустое, подставляется заглушка.
desc=$(text_or_placeholder "$desc")
# Пустые имена элементов пропускаются.
[ -z "$name" ] && continue
# В Markdown-файл записывается заголовок элемента.
echo "### $name" >> "$output_file"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$output_file"
# Для повторяющихся элементов записывается ID.
[ "${item_count:-0}" -gt 1 ] && echo "***ID:** \`${name}\#${item_index}\`" >> "$output_file"

```

```

# При наличии записывается объявление элемента.
[ -n "$decl" ] && echo "**Объявление:** \`${decl}\`" >> "$output_file"
# При наличии записывается видимость элемента.
[ -n "$vis" ] && echo "**Видимость:** $vis" >> "$output_file"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$output_file"
# В Markdown-файл записывается описание элемента.
echo "$desc" >> "$output_file"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$output_file"
# Завершается цикл обработки элементов раздела.
done
}
# Объявляется функция добавления типов и переменных уровня unit.
add_types_and_variables() {
# Сохраняется путь к XML-файлу.
xml_file="$1"
# Сохраняется путь к Markdown-файлу unit.
md_file="$2"
# Сохраняется отображаемый путь к исходнику.
source_path="$3"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$md_file"
# В Markdown-файл добавляется раздел типов.
echo "## Типы" >> "$md_file"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$md_file"
# Из XML извлекаются типы unit.
xmlstarlet_sel "$xml_file" -t \
-m "//type" \
-v "normalize-space(@name)" -o "$SEP" \
-v "normalize-space(@declaration)" -o "$SEP" \
-v "normalize-space(string(description/detailed))" -n 2>/dev/null | \
# К типам добавляются счётчики повторяющихся имён.
add_duplicate_counters | \
# Запускается цикл записи типов в Markdown-файл.
while IFS="$SEP" read -r name decl desc item_index item_count; do
# Нормализуется имя типа.
name=$(normalize_inline "$name")
# Нормализуется объявление типа.
decl=$(normalize_inline "$decl")
# Нормализуется описание типа.
desc=$(normalize_description "$desc")
# Если описание пустое, подставляется заглушка.
desc=$(text_or_placeholder "$desc")
# Пустые имена типов пропускаются.
[ -z "$name" ] && continue
# Проверяется наличие повторов имени типа.
if [ "$item_count:-0" -gt 1 ]; then
# Для повторяющегося типа записывается ID с порядковым номером.
echo "- **$name** \`${name}#${item_index}\` — \`${decl}\`" >> "$md_file"
# Обрабатывается уникальный тип.
else

```

```

# Для уникального типа записывается имя и объявление.
echo "- **$name** — \`${decl}\`" >> "$md_file"
fi
# В Markdown-файл записывается описание типа.
echo " - $desc" >> "$md_file"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$md_file"
# Завершается цикл обработки типов.
done
# В Markdown-файл добавляется пустая строка.
echo "" >> "$md_file"
# В Markdown-файл добавляется раздел переменных уровня unit.
echo "## Переменные уровня unit" >> "$md_file"
# В Markdown-файл добавляется пустая строка.
echo "" >> "$md_file"
# Из XML извлекаются переменные уровня unit.
xmlstarlet_sel "$xml_file" -t \
-m "//variable[not(parent::structure)]" \
-v "normalize-space(@name)" -o "$SEP" \
-v "normalize-space(@declaration)" -o "$SEP" \
-v "normalize-space(@visibility)" -o "$SEP" \
-v "normalize-space(string(description/detailed))" -n 2>/dev/null | \
# К переменным добавляются счётчики повторяющихся имён.
add_duplicate_counters | \
# Запускается цикл записи переменных уровня unit.
while IFS="$SEP" read -r name decl vis desc item_index item_count; do
# Нормализуется имя переменной.
name=$(normalize_inline "$name")
# Нормализуется объявление переменной.
decl=$(normalize_inline "$decl")
# Нормализуется видимость переменной.
vis=$(normalize_inline "$vis")
# Нормализуется описание переменной.
desc=$(normalize_description "$desc")
# Если описание пустое, подставляется заглушка.
desc=$(text_or_placeholder "$desc")
# Пустые имена переменных пропускаются.
[ -z "$name" ] && continue
# Проверяется наличие повторов имени переменной.
if [ "${item_count:-0}" -gt 1 ]; then
# Для повторяющейся переменной записывается ID с порядковым номером.
echo "- **$name** \`${name}#${item_index}\` (\`${vis:-public}\`): \`${decl}\`" >> "$md_file"
# Обрабатывается уникальная переменная.
else
# Для уникальной переменной записывается имя, видимость и объявление.
echo "- **$name** (\`${vis:-public}\`): \`${decl}\`" >> "$md_file"
fi
# В Markdown-файл записывается описание переменной.
echo " - $desc" >> "$md_file"
# Завершается цикл обработки переменных.
done
# В Markdown-файл добавляется пустая строка.
echo "" >> "$md_file"

```

```

# В Markdown-файл добавляется разделитель.
echo "---" >> "$md_file"
# В Markdown-файл записывается отметка об исходном файле.
echo "*Сгенерировано из: ${source_path}*" >> "$md_file"
}
# Объявляется функция обработки классов unit.
process_classes() {
    # Сохраняется путь к XML-файлу.
    xml_file="$1"
    # Сохраняется путь к каталогу wiki.
    wiki_dir="$2"
    # Сохраняется имя unit.
    unit_name="$3"
    # Сохраняется отображаемый путь к источнику.
    source_path="$4"
    # Сохраняется путь к Markdown-файлу unit.
    md_file="$5"
    # Сохраняется путь к staging-файлу для AI.
    ai_source_file="$6"
    # Создаётся временный файл для строк классов.
    class_rows=$(mktemp "${WORK_ROOT}/class_rows_XXXXXX") || error_exit "не удалось создать временный
файл class_rows"
    # Из XML извлекаются классы, их описания и предки.
    xmlstarlet_sel "$xml_file" -t \
        -m "//structure[@type='class']" \
        -v "normalize-space(@name)" -o "$SEP" \
        -v "normalize-space(string(description/detailed))" -o "$SEP" \
        -v "normalize-space(string(ancestor/@name))" -n 2>/dev/null > "$class_rows"
    # Запускается цикл обработки классов.
    while IFS="$SEP" read -r class_name class_desc ancestor_name; do
        # Нормализуется имя класса.
        class_name=$(normalize_inline "$class_name")
        # Нормализуется описание класса.
        class_desc=$(normalize_description "$class_desc")
        # При необходимости описание класса дополняется через AI.
        class_desc=$(enrich_class_description "$class_name" "$unit_name" "$class_desc" "$ai_source_file") || exit 1
        # Нормализуется имя предка класса.
        ancestor_name=$(normalize_inline "$ancestor_name")
        # Пустые имена классов пропускаются.
        [ -z "$class_name" ] && continue
        # Формируется имя страницы класса.
        page_name=$(class_page_name "$unit_name" "$class_name")
        # Формируется wiki-ссылка на класс.
        class_link=$(class_wikilink "$unit_name" "$class_name")
        # Формируется путь к Markdown-файлу класса.
        class_md="$wiki_dir/${page_name}.md"
        # Имя class-файла уже содержит unit, поэтому коллизии одноимённых классов из разных unit не
        сливаются.
        # Создаётся Markdown-страница класса.
        create_class_markdown "$class_md" "$class_name" "$class_desc" "$ancestor_name" "$unit_name" "$xml_file"
        "$source_path" "$ai_source_file"
        # В Markdown-файл unit добавляется краткая карточка класса.
        {

```

```

    echo "### Class: ${class_name}"
    echo ""
    echo "**Ссылка:** ${class_link}"
    echo "**Наследник:** ${ancestor_name:-нет}"
    echo "${class_desc}"
    echo ""
} >> "$md_file"
# Завершается цикл обработки классов.
done < "$class_rows"
}
# Объявляется функция обработки XML-файла unit.
process_xml_file() {
    # Сохраняется путь к XML-файлу.
    xml_file="$1"
    # Сохраняется путь к каталогу wiki.
    wiki_dir="$2"
    # Сохраняется имя unit.
    unit_name="$3"
    # Сохраняется путь к исходному Pascal/Delphi-файлу.
    pascal_file="$4"
    # Сохраняется путь к staging-файлу для AI.
    ai_source_file="$5"
    # Формируется отображаемый путь к исходному файлу.
    source_path=$(make_source_path_display "$pascal_file")
    # Формируется путь к Markdown-файлу unit.
    md_file="$wiki_dir/${unit_name}.md"
    # Из XML извлекается описание unit.
    unit_desc=$(xmlstarlet_sel "$xml_file" -t -v "normalize-space(string(/unit/description/detailed))" 2>/dev/null ||
true)
    # Описание unit нормализуется.
    unit_desc=$(normalize_description "$unit_desc")
    # При необходимости описание unit дополняется через AI.
    unit_desc=$(enrich_unit_description "$unit_name" "$unit_desc" "$ai_source_file") || exit 1
    # Создаётся Markdown-страница unit.
    create_unit_markdown "$md_file" "$unit_name" "$unit_desc" "$source_path" "$xml_file"
    # Обработываются классы unit.
    process_classes "$xml_file" "$wiki_dir" "$unit_name" "$source_path" "$md_file" "$ai_source_file"
    # Добавляются типы и переменные уровня unit.
    add_types_and_variables "$xml_file" "$md_file" "$source_path"
}
# Объявляется функция обработки одного Pascal/Delphi-файла.
process_single_pascal_file() {
    # Сохраняется путь к исходному файлу.
    file_path="$1"
    # Сохраняется каталог временной документации.
    doc_dir="$2"
    # Сохраняется каталог wiki.
    wiki_dir="$3"
    # Извлекается имя unit.
    unit_name=$(extract_unit_name "$file_path")
    # Получается имя файла с расширением.
    base_name=$(basename "$file_path")
    # Удаляется расширение .pas.

```

```

base_name=${base_name%.pas}
# Удаляется расширение .pp.
base_name=${base_name%.pp}
# Формируется путь к staging-копии файла.
staged_file=$(make_ascii_stage_file "$file_path")
# Исходный файл копируется в staging-каталог.
cp "$file_path" "$staged_file" || error_exit "не удалось скопировать файл в staging: $file_path"
# Формируется путь к списку файлов для PasDoc.
pasdoc_list="$doc_dir/temp_${base_name}.inc"
# Формируется каталог XML-вывода PasDoc.
xml_output_dir="$doc_dir/xml_temp_${base_name}"
# Формируется путь к логу PasDoc.
pasdoc_log="$xml_output_dir/pasdoc.log"
# Создаётся каталог XML-вывода PasDoc.
mkdir -p "$xml_output_dir"
# Путь к staging-файлу преобразуется в Windows-формат.
staged_file_win=$(to_win_path "$staged_file")
# Путь к списку файлов PasDoc преобразуется в Windows-формат.
pasdoc_list_win=$(to_win_path "$pasdoc_list")
# Путь к каталогу XML-вывода преобразуется в Windows-формат.
xml_output_dir_win=$(to_win_path "$xml_output_dir")
# В список PasDoc записывается путь к staging-файлу.
printf '%s\n' "$staged_file_win" > "$pasdoc_list"
# Запускается PasDoc для генерации SimpleXML.
"$PASDOC_PROG" \
    --output "$xml_output_dir_win" \
    --format="$PASDOC_OUTPUT_FORMAT" \
    --source "$pasdoc_list_win" \
    --define HAS_GRAPHVIZ=false > "$pasdoc_log" 2>&1 || true
# Ищется первый XML-файл, созданный PasDoc.
xml_file=$(find "$xml_output_dir" -type f -name "*.xml" | head -1)
# Проверяется, был ли XML-файл создан.
if [ -f "$xml_file" ]; then
    # XML-файл преобразуется в Markdown-страницы wiki.
    process_xml_file "$xml_file" "$wiki_dir" "$unit_name" "$file_path" "$staged_file"
    # Выводится сообщение об успешной обработке unit.
    echo "OK: $unit_name"
# Обрабатывается ситуация, когда XML не был создан.
else
    # Выводится предупреждение об отсутствии XML.
    echo "WARN: XML не сгенерирован для $file_path"
    # Проверяется наличие лога PasDoc.
    if [ -f "$pasdoc_log" ]; then
        # Выводится заголовок лога PasDoc.
        echo "----- PasDoc log: $pasdoc_log -----"
        # Выводятся первые строки лога PasDoc.
        sed -n '1,80p' "$pasdoc_log"
        # Выводится закрывающий разделитель лога.
        echo "-----"
    fi
fi
# Удаляется временный список файлов PasDoc.
rm -f "$pasdoc_list"

```

```

# Удаляется staging-копия исходного файла.
rm -f "$staged_file"
# Удаляется временный каталог XML-вывода.
rm -rf "$xml_output_dir"
}
# Объявляется функция создания индексной страницы Home.md.
create_index_file() {
    # Сохраняется путь к каталогу wiki.
    wiki_dir="$1"
    # Формируется путь к Home.md.
    home_file="$wiki_dir/Home.md"
    # В Home.md записывается заголовок и раздел unit.
    cat > "$home_file" <<EOF_MD
# Документация проекта
## Unit'ы (модули)
EOF_MD
    # По всем Markdown-файлам wiki выполняется поиск страниц unit.
    find "$wiki_dir" -maxdepth 1 -type f -name "*.md" ! -name "Home.md" | sort | while read -r md; do
        # Проверяется, является ли файл страницей unit.
        if grep -q '^# Unit:' "$md"; then
            # Получается имя страницы без расширения.
            name=$(basename "$md" .md)
            # При наличии имени в Home.md добавляется ссылка на unit.
            [ -n "$name" ] && echo "- [[$name]]" >> "$home_file"
        fi
    done
    # В Home.md добавляется раздел классов.
    cat >> "$home_file" <<EOF_MD
## Классы
EOF_MD
    # По всем Markdown-файлам wiki выполняется поиск страниц классов.
    find "$wiki_dir" -maxdepth 1 -type f -name "*.md" ! -name "Home.md" | sort | while read -r md; do
        # Проверяется, является ли файл страницей класса.
        if grep -q '^# Класс:' "$md"; then
            # Получается имя страницы класса.
            page_name=$(basename "$md" .md)
            # Извлекается заголовок класса.
            class_title=$(sed -n 's/^# Класс: //p' "$md" | head -1)
            # Если заголовок не найден, используется имя страницы.
            class_title=${class_title:-$page_name}
            # В Home.md добавляется ссылка на страницу класса.
            echo "- [[$page_name] | ${class_title}]" >> "$home_file"
        fi
    done
}
# Объявляется функция проверки описаний.
validate_descriptions() {
    # Сохраняется путь к каталогу wiki.
    wiki_dir="$1"
    # Выводится сообщение о запуске проверки описаний.
    echo "Проверка описаний..."
    # Перебираются Markdown-файлы wiki, кроме Home.md.
    find "$wiki_dir" -maxdepth 1 -type f -name "*.md" ! -name "Home.md" ! -name ".md" | while read -r md; do

```

```

# Проверяется наличие заглушки "Нет описания".
if grep -q 'Нет описания' "$md"; then
    # Выводится предупреждение о пропущенных описаниях.
    echo " ПРЕДУПРЕЖДЕНИЕ: $md содержит пропущенные описания"
fi
done
}
# Объявляется функция проверки структуры wiki.
validate_wiki_structure() {
    # Сохраняется путь к каталогу wiki.
    wiki_dir="$1"
    # Выводится сообщение о запуске проверки структуры wiki.
    echo "Проверка структуры wiki..."
    # Подсчитывается количество пустых Markdown-файлов.
    empty_md_count=$(find "$wiki_dir" -maxdepth 1 -type f -name "*.md" -size 0 | wc -l | xargs)
    # Проверяется наличие пустых Markdown-файлов.
    if [ "$empty_md_count" -gt 0 ]; then
        # Выводится предупреждение о пустых Markdown-файлах.
        echo " ПРЕДУПРЕЖДЕНИЕ: найдены пустые markdown-файлы: $empty_md_count"
    fi
    # Подсчитывается количество страниц unit.
    unit_count=$(find "$wiki_dir" -maxdepth 1 -type f -name "*.md" ! -name "Home.md" -exec grep -l '^# Unit:' {} \; | wc -l | xargs)
    # Подсчитывается количество страниц классов.
    class_count=$(find "$wiki_dir" -maxdepth 1 -type f -name "*.md" ! -name "Home.md" -exec grep -l '^# Класс:' {} \; | wc -l | xargs)
    # Выводится количество unit-страниц.
    echo " Unit-файлов: $unit_count"
    # Выводится количество class-страниц.
    echo " Class-файлов: $class_count"
}
# ОСНОВНАЯ ЛОГИКА
# Проверяются необходимые зависимости.
check_dependencies
# Проверяется конфигурация AI-обогащения.
validate_ai_config
# Создаются временные, выходные и кэш-каталоги.
mkdir -p "$DOC_DIR" "$STAGE_DIR" "$WIKI_DIR" "$CACHE_DIR"
# Регистрируется очистка временных файлов при завершении скрипта.
trap cleanup EXIT
# Нормализуем PROJECT_ROOT после проверки существования, чтобы относительные пути были стабильнее.
# В PROJECT_ROOT обратные слэши заменяются на прямые.
PROJECT_ROOT=$(printf '%s' "$PROJECT_ROOT" | sed 's#\\#/#g')
# Из PROJECT_ROOT удаляется завершающий слэш.
PROJECT_ROOT=${PROJECT_ROOT%/}
# Выводится путь к конфигурационному файлу.
echo "CONFIG_FILE: $CONFIG_FILE"
# Выводится признак загрузки конфигурации.
echo "CONFIG_LOADED: $CONFIG_LOADED"
# Выводится путь к исходному проекту.
echo "PROJECT_ROOT: $PROJECT_ROOT"
# Выводится путь к PasDoc.

```



```

echo "PASDOC_PROG: $PASDOC_PROG"
# Выводится путь к xmlstarlet.
echo "XMLSTARLET_BIN: $XMLSTARLET_BIN"
# Выводится путь к jq.
echo "JQ_BIN: $JQ_BIN"
# Выводится путь к временному корневому каталогу.
echo "WORK_ROOT: $WORK_ROOT"
# Выводится путь к временному каталогу документации.
echo "DOC_DIR: $DOC_DIR"
# Выводится путь к staging-каталогу.
echo "STAGE_DIR: $STAGE_DIR"
# Выводится путь к выходной wiki.
echo "WIKI_DIR: $WIKI_DIR"
# Выводится выбранный AI-провайдер.
echo "AI_PROVIDER: $AI_PROVIDER"
# Выводится итоговый URL AI API.
echo "AI_URL: $AI_URL_RESOLVED"
# Выводится выбранная AI-модель.
echo "AI_MODEL: $AI_MODEL"
# Выводится режим AI-обогащения unit.
echo "AI_ENRICH_UNITS: $AI_ENRICH_UNITS"
# Выводится режим AI-обогащения классов.
echo "AI_ENRICH_CLASSES: $AI_ENRICH_CLASSES"
# Выводится режим AI-обогащения методов.
echo "AI_ENRICH_METHODS: $AI_ENRICH_METHODS"
# Выводится путь к каталогу кэша.
echo "CACHE_DIR: $CACHE_DIR"
# Выводится пустая строка.
echo ""
# Инициализируется счётчик обработанных файлов.
total=0
# Создаётся временный список Pascal/Delphi-файлов.
pascal_file_list=$(mktemp "${WORK_ROOT}/pascal_files_XXXXXX") || error_exit "не удалось создать
временный список Pascal-файлов"
# Временный список заполняется найденными Pascal/Delphi-файлами.
get_pascal_file_list > "$pascal_file_list" || error_exit "не удалось получить список Pascal-файлов"
# Запускается цикл обработки Pascal/Delphi-файлов.
while read -r pascal_file; do
    # Пустые строки списка пропускаются.
    [ -z "$pascal_file" ] && continue
    # Увеличивается счётчик обработанных файлов.
    total=$((total + 1))
    # Выводится сообщение о текущем обрабатываемом файле.
    echo "[ $total ] Обработка: $pascal_file"
    # Обрабатывается один Pascal/Delphi-файл.
    process_single_pascal_file "$pascal_file" "$DOC_DIR" "$WIKI_DIR"
# Цикл читает список файлов из временного файла.
done < "$pascal_file_list"
# Создаётся индексная страница Home.md.
create_index_file "$WIKI_DIR"
# Проверяется наличие пропущенных описаний.
validate_descriptions "$WIKI_DIR"
# Проверяется структура сформированной wiki.

```

```
validate_wiki_structure "$WIKI_DIR"  
# Выводится пустая строка.  
echo ""  
# Выводится сообщение об успешном завершении генерации wiki.  
echo "Готово. Wiki создана в: $WIKI_DIR"
```

ПРИЛОЖЕНИЕ 4. Листинг pyproject.toml и server.py

Листинг pyproject.toml

```
# Объявляется секция с описанием Python-проекта.
[project]
# Задаётся имя проекта MCP-wrapper-a.
name = "markdown-rag-mcp-wrapper"
# Задаётся версия проекта.
version = "0.1.0"
# Задаётся краткое описание назначения проекта.
description = "Thin MCP wrapper around markdown-rag-mcp CLI search"
# Задаётся минимальная поддерживаемая версия Python.
requires-python = ">=3.12"
# Объявляется список зависимостей проекта.
dependencies = [
    # Подключается backend для RAG-поиска по Markdown.
    "markdown-rag-mcp",
    # Подключается библиотека MCP для реализации MCP-сервера.
    "mcp>=1.10.0",
]
# Объявляется секция настроек uv.
[tool.uv]
# Отключается сборка проекта как устанавливаемого Python-пакета.
package = false
# Объявляется секция локальных источников зависимостей uv.
[tool.uv.sources]
# Зависимость markdown-rag-mcp берётся из локального каталога и подключается в editable-режиме.
markdown-rag-mcp = { path = "../markdown-rag-mcp", editable = true }
```

Листинг server.py

```
# Включается отложенная обработка аннотаций типов.
from __future__ import annotations
# Импортируется модуль для асинхронного выполнения.
import asyncio
# Импортируется модуль с context manager-утилитами.
import contextlib
# Импортируется модуль для работы с переменными окружения.
import os
# Импортируется модуль для работы с путями импорта Python.
import sys
# Импортируется класс даты и времени для логирования.
from datetime import datetime
# Импортируется объектный интерфейс для работы с путями.
from pathlib import Path
# Импортируется универсальный тип для значений произвольной структуры.
from typing import Any
# Импортируется FastMCP для создания MCP-сервера.
from mcp.server.fastmcp import FastMCP
# Создаётся MCP-сервер с именем markdown-rag-mcp-wrapper.
mcp = FastMCP("markdown-rag-mcp-wrapper")
# Задаётся путь к backend-у markdown-rag-mcp по умолчанию.
DEFAULT_BACKEND_DIR = "D:/Works/учёба/вкpf/finalversion/tools/markdown-rag-mcp"
```

```

# Задаётся максимальный размер текста найденной секции по умолчанию.
DEFAULT_MAX_SECTION_CHARS = 4000
# Объявляется собственный тип ошибки wrapper-a.
class WrapperError(RuntimeError):
    """Raised for deterministic wrapper failures."""
# Объявляется глобальная переменная для кэшированного экземпляра RAGEngine.
_engine: Any | None = None
# Объявляется глобальная блокировка для безопасной инициализации RAGEngine.
_engine_lock: asyncio.Lock | None = None
# Объявляется функция определения корневого каталога finalversion.
def _default_final_root() -> Path:
    """Return finalversion root based on this file location.
    Expected location:
    finalversion/code/mcp/markdown-rag-mcp-wrapper/server.py
    """
    # Возвращается каталог finalversion относительно расположения server.py.
    return Path(__file__).resolve().parents[3]
# Объявляется функция формирования пути к debug-логу по умолчанию.
def _default_debug_log_path() -> str:
    # Возвращается путь к файлу лога wrapper-a внутри data/logs.
    return str(_default_final_root() / "data" / "logs" / "markdown-rag-mcp-wrapper.log")
# Объявляется функция чтения переменной окружения с fallback-значением.
def _env(name: str, default: str) -> str:
    # Из окружения читается значение переменной и очищается от крайних пробелов.
    value = os.environ.get(name, "").strip()
    # Возвращается значение из окружения или значение по умолчанию.
    return value if value else default
# Объявляется функция получения каталога backend-a.
def _backend_dir() -> str:
    # Возвращается MARKDOWN_RAG_BACKEND_DIR или каталог backend-a по умолчанию.
    return _env("MARKDOWN_RAG_BACKEND_DIR", DEFAULT_BACKEND_DIR)
# Объявляется функция получения пути к debug-логу.
def _debug_log_path() -> str:
    # Возвращается MARKDOWN_RAG_DEBUG_LOG_PATH или путь к debug-логу по умолчанию.
    return _env("MARKDOWN_RAG_DEBUG_LOG_PATH", _default_debug_log_path())
# Объявляется функция получения ограничения размера найденной секции.
def _max_section_chars() -> int:
    # Читается значение MARKDOWN_RAG_MAX_SECTION_CHARS или значение по умолчанию.
    raw = _env("MARKDOWN_RAG_MAX_SECTION_CHARS", str(DEFAULT_MAX_SECTION_CHARS))
    # Выполняется попытка преобразовать значение в целое число.
    try:
        # Значение преобразуется в int.
        value = int(raw)
    except ValueError as exc:
        # Обработывается ошибка преобразования.
        # Возвращается управляемая ошибка wrapper-a при некорректном значении.
        raise WrapperError(f"MARKDOWN_RAG_MAX_SECTION_CHARS must be an integer, got: {raw!r}") from exc
    # Проверяется, что ограничение больше нуля.
    if value <= 0:
        # Возвращается управляемая ошибка wrapper-a при неположительном значении.
        raise WrapperError("MARKDOWN_RAG_MAX_SECTION_CHARS must be > 0")
    # Возвращается корректное числовое ограничение.

```

```

    return value
# Объявляется функция записи диагностического сообщения в лог.
def _debug_log(message: str) -> None:
    """Write a diagnostic line to a log file.
    Logging must never break the MCP tool itself.
    """

# Перехватываются любые ошибки логирования, чтобы они не нарушали работу MCP-инструмента.
try:
    # Получается путь к debug-логу.
    log_path = Path(_debug_log_path())
    # Создаётся родительский каталог лога.
    log_path.parent.mkdir(parents=True, exist_ok=True)
    # Формируется строка лога с текущим временем.
    line = f"{datetime.now().isoformat(timespec='seconds')} {message}\n"
    # Открывается файл лога для добавления записи.
    with log_path.open("a", encoding="utf-8", errors="replace") as f:
        # Записывается строка лога.
        f.write(line)
# Ошибки логирования игнорируются.
except Exception:
    # Ничего не выполняется, чтобы логирование не ломало основной сценарий.
    pass
# Объявляется context manager для перехвата stdout и stderr backend-a.
@contextlib.contextmanager
def _capture_backend_output(label: str):
    """Redirect noisy backend stdout/stderr away from MCP stdio.
    MCP stdio uses stdout for protocol messages. Any logs/progress bars printed
    by markdown-rag-mcp, sentence-transformers, tqdm, rich, etc. can corrupt the
    MCP JSON-RPC stream. Therefore backend output must go to a log file.
    """

    # Получается путь к debug-логу.
    log_path = Path(_debug_log_path())
    # Создаётся родительский каталог лога.
    log_path.parent.mkdir(parents=True, exist_ok=True)
    # Открывается файл лога для записи вывода backend-a.
    with log_path.open("a", encoding="utf-8", errors="replace") as f:
        # Записывается отметка начала перехвата вывода.
        f.write(f"{datetime.now().isoformat(timespec='seconds')} capture start: {label}\n")
        # Принудительно сбрасывается буфер записи.
        f.flush()
        # stdout и stderr временно перенаправляются в файл лога.
        with contextlib.redirect_stdout(f), contextlib.redirect_stderr(f):
            # Выполняется защищённый блок вызывающего кода.
            try:
                # Управление передаётся в тело with-блока.
                yield
            # Блок finally выполняется при любом завершении context manager-a.
            finally:
                # Записывается отметка окончания перехвата вывода.
                f.write(f"{datetime.now().isoformat(timespec='seconds')} capture end: {label}\n")
                # Принудительно сбрасывается буфер записи.
                f.flush()
# Объявляется функция проверки структуры backend-каталога.

```

```

def _validate_backend_paths() -> None:
    # Получается путь к backend-каталогу.
    backend = Path(_backend_dir())
    # Проверяется существование backend-каталога.
    if not backend.exists():
        # Возвращается управляемая ошибка, если каталог backend-а отсутствует.
        raise WrapperError(f"Backend directory does not exist: {backend}")
    # Проверяется наличие pyproject.toml backend-а.
    if not (backend / "pyproject.toml").exists():
        # Возвращается ошибка, если каталог не похож на markdown-rag-mcp.
        raise WrapperError(f"Backend directory does not look like markdown-rag-mcp: {backend}")
    # Проверяется наличие исходного пакета markdown_rag_mcp.
    if not (backend / "src" / "markdown_rag_mcp").exists():
        # Возвращается ошибка, если пакет backend-а не найден.
        raise WrapperError(f"Backend src package not found: {backend / 'src' / 'markdown_rag_mcp'}")
# Объявляется функция добавления backend-а в sys.path.
def _ensure_backend_import_path() -> None:
    """Add backend src to sys.path as a fallback.
    Preferred runtime mode:
    uv --directory <MARKDOWN_RAG_BACKEND_DIR> run python <wrapper/server.py>
    In that mode markdown_rag_mcp is already installed in the backend venv.
    """
    # Формируется путь к src-каталогу backend-а.
    backend_src = str(Path(_backend_dir()) / "src")
    # Проверяется, присутствует ли путь backend-а в sys.path.
    if backend_src not in sys.path:
        # Путь backend-а добавляется в начало sys.path.
        sys.path.insert(0, backend_src)
# Объявляется функция обрезки длинного текста.
def _truncate(text: str, max_chars: int) -> str:
    # Если текст не превышает лимит, он возвращается без изменений.
    if len(text) <= max_chars:
        # Возвращается исходный текст.
        return text
    # Возвращается сокращённый текст с пометкой об обрезке.
    return text[: max_chars - 20].rstrip() + "\n... [truncated]"
# Объявляется функция приведения результата поиска к компактному словарю.
def _compact_result(result: Any, max_chars: int) -> dict[str, Any]:
    # Из результата извлекаются metadata.
    metadata = getattr(result, "metadata", None)
    # Проверяется, что metadata является словарём.
    if not isinstance(metadata, dict):
        # Если metadata не является словарём, используется пустой словарь.
        metadata = {}
    # Из результата извлекается текст секции.
    section_text = str(getattr(result, "section_text", "") or "")
    # Возвращается компактное представление результата поиска.
    return {
        # Сохраняется оценка уверенности результата.
        "confidence_score": getattr(result, "confidence_score", None),
        # Сохраняется заголовок найденной секции.
        "section_heading": getattr(result, "section_heading", None),
        # Сохраняется текст секции с ограничением длины.
    }

```

```

"section_text": _truncate(section_text, max_chars),
# Сохраняется путь к файлу, в котором найден результат.
"file_path": getattr(result, "file_path", None),
# Сохраняется подмножество служебных metadata.
"metadata": {
    # Сохраняется идентификатор документа.
    "document_id": metadata.get("document_id"),
    # Сохраняется идентификатор секции.
    "section_id": metadata.get("section_id"),
    # Сохраняется заголовок секции из metadata.
    "section_heading": metadata.get("section_heading"),
    # Сохраняется индекс chunk-а.
    "chunk_index": metadata.get("chunk_index"),
    # Сохраняется позиция начала chunk-а.
    "start_position": metadata.get("start_position"),
    # Сохраняется позиция конца chunk-а.
    "end_position": metadata.get("end_position"),
    # Сохраняется количество токенов.
    "token_count": metadata.get("token_count"),
},
}
# Объявляется асинхронная функция получения RAGEngine.
async def _get_engine() -> Any:
    """Initialize and cache RAGEngine inside the MCP process."""
    # Подключаются глобальные переменные движка и блокировки.
    global _engine, _engine_lock
    # В лог записывается вход в функцию получения движка.
    _debug_log("_get_engine enter")
    # Проверяется, создана ли блокировка инициализации.
    if _engine_lock is None:
        # В лог записывается создание блокировки.
        _debug_log("_get_engine create lock")
        # Создаётся асинхронная блокировка.
        _engine_lock = asyncio.Lock()
    # В лог записывается состояние перед входом в блокировку.
    _debug_log("_get_engine before lock")
    # Инициализация движка выполняется под блокировкой.
    async with _engine_lock:
        # В лог записывается вход в критическую секцию.
        _debug_log("_get_engine after lock")
        # Проверяется, был ли движок уже создан ранее.
        if _engine is not None:
            # В лог записывается использование кэшированного движка.
            _debug_log("_get_engine cached engine hit")
            # Возвращается кэшированный движок.
            return _engine
        # В лог записывается начало проверки путей backend-а.
        _debug_log("_get_engine validate backend paths start")
        # Проверяется корректность расположения backend-а.
        _validate_backend_paths()
        # В лог записывается успешная проверка путей backend-а.
        _debug_log("_get_engine validate backend paths success")
        # В лог записывается начало добавления backend-а в sys.path.

```

```

_debug_log("_get_engine ensure backend import path start")
# Добавляется путь к backend-у в sys.path.
_ensure_backend_import_path()
# В лог записывается успешное добавление backend-а в sys.path.
_debug_log("_get_engine ensure backend import path success")
# Выполняется импорт компонентов backend-а.
try:
    # В лог записывается начало импорта config.
    _debug_log("_get_engine import markdown_rag_mcp.config start")
    # Импортируется функция получения конфигурации backend-а.
    from markdown_rag_mcp.config import get_config
    # В лог записывается успешный импорт config.
    _debug_log("_get_engine import markdown_rag_mcp.config success")
    # В лог записывается начало импорта RAGEngine.
    _debug_log("_get_engine import markdown_rag_mcp.core start")
    # Импортируется основной движок RAG-поиска.
    from markdown_rag_mcp.core import RAGEngine
    # В лог записывается успешный импорт RAGEngine.
    _debug_log("_get_engine import markdown_rag_mcp.core success")
# Обрабатывается ошибка импорта backend-а.
except Exception as exc:
    # В лог записывается ошибка импорта.
    _debug_log(f"_get_engine import failed {type(exc).__name__}: {exc}")
    # Возвращается управляемая ошибка wrapper-а.
    raise WrapperError(
        "Failed to import markdown_rag_mcp. "
        "Run wrapper through the markdown-rag-mcp backend environment. "
        f"Original error: {type(exc).__name__}: {exc}"
    ) from exc
# Выполняется создание и инициализация RAGEngine.
try:
    # В лог записывается начало инициализации RAGEngine.
    _debug_log("RAGEngine initialization start")
    # В лог записывается начало получения конфигурации.
    _debug_log("get_config start")
    # Получается конфигурация backend-а.
    config = get_config()
    # В лог записывается успешное получение конфигурации.
    _debug_log("get_config success")
    # В лог записывается начало создания RAGEngine.
    _debug_log("RAGEngine constructor start")
    # Создаётся экземпляр RAGEngine.
    engine = RAGEngine(config)
    # В лог записывается успешное создание RAGEngine.
    _debug_log("RAGEngine constructor success")
    # В лог записывается начало асинхронной инициализации движка.
    _debug_log("engine.initialize start")
    # Выполняется асинхронная инициализация RAGEngine.
    await engine.initialize()
    # В лог записывается успешная инициализация движка.
    _debug_log("engine.initialize success")
    # Инициализированный движок сохраняется в глобальный кэш.
    _engine = engine

```



```

# В лог записывается успешное завершение инициализации RAGEngine.
_debug_log("RAGEngine initialization success")
# Возвращается инициализированный движок.
return _engine
# Обработывается ошибка инициализации RAGEngine.
except Exception as exc:
    # Глобальный кэш движка сбрасывается.
    _engine = None
    # В лог записывается ошибка инициализации.
    _debug_log(f"RAGEngine initialization failed {type(exc).__name__}: {exc}")
    # Возвращается управляемая ошибка wrapper-a.
    raise WrapperError(f"Failed to initialize RAGEngine: {type(exc).__name__}: {exc}") from exc
# Объявляется асинхронная функция остановки RAGEngine.
async def _shutdown_engine() -> None:
    # Подключается глобальная переменная движка.
    global _engine
    # Проверяется, существует ли кэшированный движок.
    if _engine is None:
        # Если движок не создан, функция завершается.
        return
    # Выполняется попытка корректно остановить движок.
    try:
        # В лог записывается начало остановки RAGEngine.
        _debug_log("RAGEngine shutdown start")
        # Вывод backend-a при shutdown перенаправляется в debug-лог.
        with _capture_backend_output("shutdown"):
            # Выполняется асинхронная остановка движка.
            await _engine.shutdown()
        # В лог записывается успешная остановка RAGEngine.
        _debug_log("RAGEngine shutdown success")
    # Блок finally выполняется независимо от результата остановки.
    finally:
        # Глобальный кэш движка очищается.
        _engine = None
# Регистрируется MCP-инструмент прогрева RAGEngine.
@mcp.tool()
async def markdown_rag_warmup() -> dict[str, Any]:
    """Initialize backend imports and cached RAGEngine before real search."""
    # Выполняется попытка прогреть backend.
    try:
        # В лог записывается начало прогрева.
        _debug_log("warmup start")
        # Вывод backend-a при прогреве перенаправляется в debug-лог.
        with _capture_backend_output("warmup"):
            # Получается или создаётся RAGEngine.
            engine = await _get_engine()
        # В лог записывается успешное завершение прогрева.
        _debug_log("warmup success")
        # Возвращается успешный ответ MCP-инструмента.
        return {
            # Указывается успешный статус.
            "status": "success",
            # Возвращается текстовое сообщение о готовности RAGEngine.

```

```

        "message": "RAGEngine is initialized",
        # Возвращается признак наличия кэшированного движка.
        "engine_cached": engine is not None,
    }
# Обработывается ошибка прогрева.
except Exception as exc:
    # В лог записывается ошибка прогрева.
    _debug_log(f"warnup error {type(exc).__name__}: {exc}")
    # Возвращается структурированный ответ об ошибке.
    return {
        # Указывается статус ошибки.
        "status": "error",
        # Возвращается тип ошибки.
        "error_type": type(exc).__name__,
        # Возвращается текст ошибки.
        "error": str(exc),
    }
# Регистрируется MCP-инструмент поиска по Markdown-wiki.
@mcp.tool()
async def markdown_rag_search(
    # Принимается текст поискового запроса.
    query: str,
    # Принимается максимальное количество результатов.
    limit: int = 5,
    # Принимается порог близости результата.
    threshold: float = 0.1,
    # Принимается флаг включения metadata.
    include_metadata: bool = True,
) -> dict[str, Any]:
    """Search the indexed Markdown wiki through markdown-rag-mcp Python API.
    The wiki must be indexed beforehand by the build/orchestration pipeline.
    This tool is intentionally read-only.
    """
    # Проверяется, что поисковый запрос не пустой.
    if not query.strip():
        # Возвращается ошибка пустого запроса.
        return {
            # Указывается статус ошибки.
            "status": "error",
            # Возвращается описание ошибки.
            "error": "query must not be empty",
            # Возвращается исходный запрос.
            "query": query,
            # Возвращается переданный limit.
            "limit": limit,
            # Возвращается переданный threshold.
            "threshold": threshold,
        }
    # Проверяется, что limit больше нуля.
    if limit <= 0:
        # Возвращается ошибка некорректного limit.
        return {
            # Указывается статус ошибки.

```

```

    "status": "error",
    # Возвращается описание ошибки.
    "error": "limit must be > 0",
    # Возвращается исходный запрос.
    "query": query,
    # Возвращается переданный limit.
    "limit": limit,
    # Возвращается переданный threshold.
    "threshold": threshold,
}
# Проверяется, что threshold находится в допустимом диапазоне.
if threshold < 0.0 or threshold > 1.0:
    # Возвращается ошибка некорректного threshold.
    return {
        # Указывается статус ошибки.
        "status": "error",
        # Возвращается описание ошибки.
        "error": "threshold must be in range [0.0, 1.0]",
        # Возвращается исходный запрос.
        "query": query,
        # Возвращается переданный limit.
        "limit": limit,
        # Возвращается переданный threshold.
        "threshold": threshold,
    }
# Выполняется попытка поиска.
try:
    # В лог записывается начало поиска.
    _debug_log(f"search start query={query!r} limit={limit} threshold={threshold}")
    # Вывод backend-а при поиске перенаправляется в debug-лог.
    with _capture_backend_output("search"):
        # В лог записывается состояние перед получением движка.
        _debug_log("search before _get_engine")
        # Получается или создаётся RAGEngine.
        engine = await _get_engine()
        # В лог записывается состояние после получения движка.
        _debug_log("search after _get_engine")
        # В лог записывается состояние перед вызовом поиска backend-а.
        _debug_log("search before engine.search")
        # Выполняется поиск через API markdown-rag-мсп.
        results_raw = await engine.search(
            query=query,
            limit=limit,
            similarity_threshold=threshold,
            include_metadata=include_metadata,
        )
        # В лог записывается состояние после поиска backend-а.
        _debug_log("search after engine.search")
    # Получается ограничение размера текста секции.
    max_chars = _max_section_chars()
    # Результаты backend-а приводятся к компактному виду.
    results = [_compact_result(item, max_chars) for item in results_raw]
    # В лог записывается успешное завершение поиска.

```

```

_debug_log(f"search success query={query!r} result_count={len(results)}")
# Возвращается успешный результат поиска.
return {
    # Указывается успешный статус.
    "status": "success",
    # Возвращается исходный запрос.
    "query": query,
    # Возвращается использованный limit.
    "limit": limit,
    # Возвращается использованный threshold.
    "threshold": threshold,
    # Возвращается количество найденных результатов.
    "result_count": len(results),
    # Возвращается список найденных результатов.
    "results": results,
}
# Обрабатывается ошибка поиска.
except Exception as exc:
    # В лог записывается ошибка поиска.
    _debug_log(f"search error {type(exc).__name__}: {exc}")
    # Возвращается структурированная ошибка поиска.
    return {
        # Указывается статус ошибки.
        "status": "error",
        # Возвращается тип ошибки.
        "error_type": type(exc).__name__,
        # Возвращается текст ошибки.
        "error": str(exc),
        # Возвращается исходный запрос.
        "query": query,
        # Возвращается использованный limit.
        "limit": limit,
        # Возвращается использованный threshold.
        "threshold": threshold,
    }
# Регистрируется MCP-инструмент сброса кэшированного RAGEngine.
@mcp.tool()
async def markdown_rag_reload_engine() -> dict[str, Any]:
    """Shutdown cached RAGEngine so the next search reinitializes it."""
    # Выполняется попытка сбросить кэш движка.
    try:
        # В лог записывается начало reload_engine.
        _debug_log("reload_engine start")
        # Выполняется остановка и очистка кэшированного RAGEngine.
        await _shutdown_engine()
        # В лог записывается успешное завершение reload_engine.
        _debug_log("reload_engine success")
        # Возвращается успешный ответ.
        return {
            # Указывается успешный статус.
            "status": "success",
            # Возвращается сообщение об очистке кэша.
            "message": "RAGEngine cache cleared",

```

```

    }
    # Обрабатывается ошибка сброса движка.
except Exception as exc:
    # В лог записывается ошибка reload_engine.
    _debug_log(f"reload_engine error {type(exc).__name__}: {exc}")
    # Возвращается структурированная ошибка.
    return {
        # Указывается статус ошибки.
        "status": "error",
        # Возвращается тип ошибки.
        "error_type": type(exc).__name__,
        # Возвращается текст ошибки.
        "error": str(exc),
    }

# Регистрируется MCP-инструмент проверки доступности wrapper-a.
@mcp.tool()
def markdown_rag_ping() -> dict[str, Any]:
    """Fast diagnostic tool to verify OpenCode can call this MCP server."""
    # В лог записывается вызов ping.
    _debug_log("ping")
    # Возвращается успешный диагностический ответ.
    return {
        # Указывается успешный статус.
        "status": "success",
        # Возвращается сообщение о доступности wrapper-a.
        "message": "markdown-rag-mcp-wrapper is reachable",
    }

# Проверяется, запущен ли файл как основной модуль.
if __name__ == "__main__":
    # Запускается MCP-сервер.
    mcp.run()

```

ПРИЛОЖЕНИЕ 5. Листинг `opcode.fragment_example.json` и `AGENTS.md`

Листинг `opcode.fragment_example.json`

```
{
  // Подключается схема конфигурационного файла OpenCode.
  "$schema": "https://opcode.ai/config.json",
  // Задаётся блок экспериментальных настроек OpenCode.
  "experimental": {
    // Устанавливается увеличенное время ожидания MCP-инструментов.
    "mcp_timeout": 100000
  },
  // Подключаются файлы с инструкциями для агента.
  "instructions": [
    // Указывается путь к файлу AGENTS.md.
    "D:/Works/учёба/вкп/finalversion/AGENTS.md"
  ],
  // Задаётся блок MCP-серверов, доступных агенту.
  "mcp": {
    // Описывается MCP-сервер pasls для точной навигации по Pascal/Delphi-коду.
    "pasls": {
      // Указывается локальный тип запуска MCP-сервера.
      "type": "local",
      // Задаётся команда запуска mcp-language-server с подключением pasls.
      "command": [
        // Указывается путь к исполняемому файлу mcp-language-server.
        "D:/Works/учёба/вкп/finalversion/tools/bin/mcp-language-server.exe",
        // Передаётся параметр рабочей директории проекта.
        "-workspace",
        // Указывается путь к исходной Pascal/Delphi-кодовой базе.
        "D:/Works/учёба/вкп/finalversion/data/input_project/src",
        // Передаётся параметр LSP-сервера.
        "-lsp",
        // Указывается путь к исполняемому файлу pasls.
        "D:/Works/учёба/вкп/finalversion/tools/pasls/pasls.exe"
      ],
    },
    // Задаётся время ожидания ответа pasls MCP-сервера.
    "timeout": 100000
  },
  // Описывается MCP-сервер wrapper-a для поиска по Markdown-wiki.
  "markdown_rag_wrapper": {
    // Указывается локальный тип запуска MCP-сервера.
    "type": "local",
    // Задаётся команда запуска MCP-wrapper-a через uv.
    "command": [
      // Указывается путь к исполняемому файлу uv.
      "C:/Users/Danii/AppData/Roaming/Python/Python313/Scripts/uv.exe",
      // Передаётся параметр рабочей директории uv.
      "--directory",
      // Указывается каталог backend-a markdown-rag-mcp.
      "D:/Works/учёба/вкп/finalversion/tools/markdown-rag-mcp",
      // Передаётся команда запуска через uv.
      "run",
    ],
  },
}
```

```

// Указывается интерпретатор Python.
"python",
// Указывается путь к серверному файлу MCP-wrapper-a.
"D:/Works/учёба/вкр/finalversion/code/mcp/markdown-rag-mcp-wrapper/server.py"
],
// Включается использование MCP-wrapper-a.
"enabled": true,
// Задаётся увеличенное время ожидания ответа wrapper-a.
"timeout": 100000
}
}
}

```

Листинг AGENTS.md

Ты интеллектуальный помощник и работаешь с Pascal/Delphi-кодовой базой и Markdown wiki.

Доступные источники

`markdown\_rag\_wrapper` — позволяет получить информацию о структуре проекта;

`pasls` — code intelligence по Pascal/Delphi;

встроенные инструменты OpenCode — чтение и изменение файлов проекта.

Правила работы

1. Если вопрос касается архитектуры, unit/class/method, назначения сущностей или связей в проекте — сначала используй `markdown\_rag\_wrapper.markdown\_rag\_search`. он позволяет получить информацию о структуре и комментариях сущности

2. Рекомендации по применению:

- для точных имён методов/классов используй короткие query с идентификаторами. достаточные параметры `limit = 4`, `threshold = 0.1`, `include\_metadata = true`;

- для смысловых вопросов используй query на русском или английском по смыслу. достаточные параметры `limit = 5-7`, `threshold = 0.1`, `include\_metadata = true`;

3. Используй результат как навигационный контекст, но не считай его единственным источником истины.

4. Для точного ответа по реализации проверяй исходный код через `pasls` или чтение файлов. перед использованием `pasls` инструментов требуется запустить tool diagnostics. не используй `pasls` инструменты редактирования и tool references.

5. Не выполняй широкий поиск по домашней папке пользователя или всему диску. Если нужно читать исходный код, ограничивайся папкой проекта из конфига: `data/input\_project/src`.

6. Если в wiki написано `Нет описания`, это не ошибка поиска. Используй ID, объявление, видимость и `source\_path`, затем уточняй по исходному коду.

7. Не выдумывай детали реализации. Ответ должен строиться на данных, а не догадках. Если информации недостаточно, скажи, что нужно проверить исходник.

8. Не изменяй файлы, если пользователь просит только объяснение, анализ или поиск.

9. Если пользователь просит изменить код, сначала кратко объясни план изменения, затем вноси минимальные правки.

10. Отвечай на русском языке.

11. В ответах по коду указывай:

- найденную сущность;
- unit/class/method;
- краткое назначение;
- при необходимости — где смотреть исходный код.

12. до использования `markdown\_rag\_search` нужно один раз использовать `markdown\_rag\_warmup`. результат будет ошибкой ожидания. это нормально

Стиль ответа

- кратко, но достаточно конкретно;
- без лишней теории;
- с опорой на найденные сущности проекта;

- если есть сомнение — явно обозначай его.